

Machine Learning

Passerini Andrea

a.y. 2025/2026

University of Trento

Davide Donà

July 2026

Notes Repository

Contents

1	Introduction	1
1.1	Machine Learning Systems	1
1.2	Learning settings	3
1.3	Tasks in Machine Learning	4
1.4	Choice of learning algorithms	4
2	Decision Tree Learning	6
2.1	Appropriate problems	6
2.2	Learning decision tree	7
2.2.1	Choosing the best attribute	7
2.3	Issues in decision tree learning	8
2.3.1	Overfitting avoidance	8
2.3.2	Dealing with continuous-valued attributes:	9
2.3.3	Alternative attributes test measure	9
2.3.4	Handling Attributes with missing values	10
2.4	Random Forests	11
3	K-Nearest Neighbors	12
3.1	Metric function	12
3.2	Classification	13
3.3	Regression	13
3.4	Characteristics	14
3.5	Distance-weighted K-Nearest Neighbor	14
4	Evaluation	16
4.1	Binary classification	16
4.1.1	Accuracy	17
4.1.2	Precision and Recall	17
4.1.3	Precision-Recall curve	18
4.2	Multi-class classification	19
4.3	Regression	20
4.4	Performance estimation	20
4.4.1	Hold-out procedure	20
4.4.2	K-Fold Cross-validation	20
4.5	Hypothesis testing	21
4.5.1	T-test	22
4.5.2	Example: comparing two algorithms	22
5	Parameter Estimation	24
5.1	Maximum Likelihood Estimation	25
5.1.1	Maximizing the log-likelihood	26
5.1.2	Example: MLE for Gaussian distribution	26
5.2	Bayesian Estimation	28

5.2.1	Example: Bayesian estimation for Gaussian distribution .	29
5.3	Sufficient statistics	31
5.4	Conjugate priors	32
5.5	Example of parameter estimation	32
5.5.1	Bernoulli distribution	32
5.5.2	Multinomial distribution	34
6	Bayesian Networks	35
6.1	Bayesian Networks structure	35
6.1.1	Independency Map	36
6.1.2	Factorization	36
6.2	Bayesian Network definition	37
6.3	D-separation	38
6.3.1	Independence and conditional independence	38
6.3.2	Tail-to-tail connection	39
6.3.3	Head-to-tail connection	40
6.3.4	Head-to-head connection	41
6.3.5	General D-separation criterion	43
6.4	Bayesian Network Independencies	44
6.5	Equivalence Classes	45
6.6	Building Bayesian Networks	46
7	Learning Bayesian Networks	47
7.1	Parameter Learning	47
7.1.1	Complete Data	47
7.1.2	Adding priors	49
7.1.3	Incomplete Data	50
7.2	Structure Learning	50
8	Naive Bayes	52
8.1	Naive Bayes assumption	52
8.2	Naive Bayes classifier	52
8.2.1	Parameter estimation	53
8.3	Example - Text Classification	53
9	Linear Discriminant Functions	55
9.1	Advantages and disadvantages of discriminative models	55
9.2	Definition	55
9.3	Linear Binary Classifier	56
9.4	Multiclass Classification	58
9.4.1	One-vs-All (OvA)	58
9.4.2	One-vs-One (OvO)	58
9.4.3	One-vs-One (OvO) vs One-vs-All (OvA)	58

10 Perceptron	59
10.1 Biological Motivation	59
10.2 Single Neuron Architecture	59
10.2.1 Augmented feature / weight vector	60
10.2.2 Representational power	60
10.3 Parameter learning	60
10.3.1 Gradient descent	60
10.3.2 Stochastic Gradient Descent	62
10.4 Perceptron Regression	62
10.4.1 Error function	63
10.4.2 Closed form solution	63
10.4.3 Gradient descent	64
11 Support Vector Machines	65
11.1 Maximum Margin Classifier	65
11.2 Hard Margin SVM	66
11.2.1 Learning problem	67
11.2.2 Karush-Kuhn-Tucker (KKT) Approach	67
11.2.3 Lagrangian of the SVM problem	68
11.2.4 Observations	69
11.3 Soft Margin SVM	70
11.3.1 Learning problem	71
11.3.2 Regularization theory	71
11.3.3 Lagrangian of the Soft Margin SVM	72
11.3.4 Observations	73
11.4 Large Scale SVM	74
11.5 Margin Error Bound	75
12 Kernel Machines	76
12.1 Feature Map	76
12.2 Linear separation in feature space	77
12.3 Kernel trick	78
12.3.1 Examples of kernels	78
12.4 Valid Kernels	79
12.4.1 Basic Kernels	80
12.4.2 Kernel Combinations	81
12.5 Kernel on Graphs	82
12.5.1 Graph Isomorphism	82
12.5.2 Weisfeiler-Lehman Test	83
12.5.3 Weisfeiler-Lehman Kernel	83
13 Neural Networks	85
13.1 Multi Layer Perceptron	85
13.1.1 Activation Functions	86
13.1.2 Output Layer	87
13.1.3 Representational power of MLPs	87

13.2	Training Multi Layer Perceptrons	88
13.2.1	Loss Functions	88
13.2.2	Stochastic Gradient Descent	89
13.2.3	Backpropagation	89
13.3	Modular Structure of Neural Networks	91
13.4	Remarks on Training Neural Networks	92
13.4.1	Stopping Criteria	92
13.4.2	Vanishing gradients	92
13.5	Popular architectures	95
14	Unsupervised Learning - Clustering	97
14.1	K-Means Clustering	97
14.2	Quality of Clustering	98
14.3	Gaussian Mixture Models (GMM)	98
14.4	Choose the number of clusters k	100
14.4.1	Elbow Method	100
14.4.2	Silhouette Method	101
14.4.3	Hierarchical Clustering	102
15	Reinforcement Learning	104
15.1	Learning Settings	104
15.2	Markov Decision Process (MDP)	104
15.3	Utilities	105
15.4	Policy	105
15.4.1	Bellman Equations	106
15.4.2	Value Iteration	106
15.4.3	Policy Iteration	107
15.5	Dealing with Partial Knowledge	108
15.6	Policy Evaluation	108
15.6.1	Adaptive Dynamic Programming (ADP)	108
15.6.2	Temporal-Difference Learning (TD Learning)	109
15.6.3	Differences	110
15.7	Policy Improvement	110
15.7.1	A simple solution	110
15.7.2	Trade off strategies	110
15.7.3	Utilities of an action:	111
15.8	Scaling to Large Problems	113
15.8.1	TD learning: State Utilities	113
15.8.2	TD learning: Action Utilities	113
16	Ensemble methods	114
16.1	Bagging	114
16.1.1	Bootstrap resampling	114
16.1.2	Combining methods	115
16.2	Stacking	116
16.3	Boosting	116

16.3.1 AdaBoost	117
A Principal Component Analysis	118
A.1 PCA Algorithm	118

1 Introduction

What is Machine Learning: a computer **program** is said to **learn** from **experience** E , with respect to a **class of task** T and a **performance measure** P , if its performance at the task T , as measured by P , increases with the experience E .

- **Task** T : the problem to be addressed (resolved) by the computer;
- **Performance measure** P : evaluate the learned system. Can be tricky to design in cases of data generation;
- **Training experience** E : data, used to train the system.

1.1 Machine Learning Systems

To design a **Machine Learning System**, we have to follow these steps:

1. **Formalizing the learning task;**
2. **Collecting the data;**
3. **Features extraction;**
4. **Choose the class of the learning model;**
5. **Train** the model;
6. **Evaluate** the model;

Going more in depth for each step, we can say:

Formalizing the learning task: we are going to define the task that should be addressed by the learning system. A learning problem is often composed of many related tasks. During this phase, also an appropriate performance measure for the learning system should be defined.

Collecting the data: a set of training examples must be collected in a machine-readable format. The data could be of two types:

- **Labeled data**, used for supervised learning, requires manual intervention for its generation;
- **Unlabeled data**: used in unsupervised learning or in semi-supervised learning.

This is one of the most critical phases of the whole process.

Extracting Features: from the raw data, a relevant set of features (a subset of all the features of the data, only containing relevant information) must be extracted before feeding it as input to the learning system.

- Too **few features** can miss relevant information contained in the data, preventing the system from learning the task with reasonable performance.
- Too **many features** (usually done in deep learning) requires more training data to achieve the same levels of generalization.

Choosing learning model: Based on the task:

- **Simple linear model** is easy to train, but might not be enough for non-linearly separable data.

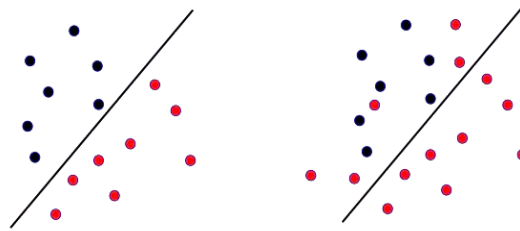


Figure 1: Linear model applied to Linear and Non-Linear data

- **Complex model** can learn even non-linearly separable data. A too complex model could learn from noise in the data, failing to generalize on new unseen data.

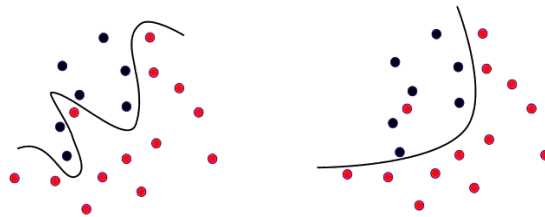


Figure 2: Complex model learning noise on the left, and generalizing well on the right

It's important to note that, when a linear model can be applied, it's usually preferred due to its simplicity and efficiency. In such cases, there is no need to resort to more complex models.

Training the model: this implies searching through the space of possible models, trying to fit the available training examples, according to the chosen performance measure. The learned model should generalize well on unseen data, not just memorize the training examples (overfitting).

Evaluating the model : applying **performance measures** on the **learned model**, using unseen data. This phase can provide insights into the model's weaknesses, giving suggestions for refining it.

1.2 Learning settings

In this section we will analyze the learning setting for different types of learning.

Supervised Learning In supervised learning the learner is provided with a set of input/output pairs $(x_i, y_i) \in X \times Y$, where:

- X : input space (set of possible feature vectors);
- Y : output space (set of possible labels or target values);
- x_i : the i -th input feature vector, with dimension X ;
- y_i : the corresponding output (label or target), with dimension Y .

The learned model $f : X \rightarrow Y$ should map input into their outputs. Typically, a domain expert is involved in labeling the training data.

Unsupervised Learning The learner is provided with a set of input examples $x_i \in X$, with no labeling information. The task is to model training examples, for example by grouping them into clusters according to their similarity.

Semi-supervised Learning As in the Supervised Learning, the learner is provided with a set of input/output pairs. Also, a much larger group of unlabeled data $x_i \in X$ is provided. Like in supervised learning, the learned model $f : X \rightarrow Y$ should map input examples into their outputs.

The additional unlabeled data can be exploited to improve performances of the model (forcing the model to produce similar outputs for similar inputs or to learn the structure of the input data).

Reinforcement Learning The learner is provided with a set of possible **STATES** S , and for each state, a set of possible **actions** A , moving it to a next state. While performing an action a , the learner is provided with an immediate reward $r(s, a)$. The task is to learn a **POLICY**, that given a state s , selects an action a that maximize the overall reward (including future moves).

1.3 Tasks in Machine Learning

Supervised Learning tasks We can have different types of tasks in supervised learning:

- **CLASSIFICATION**: learning a discrete label (finite number of options). Could be:
 - **Binary**: assign one of two possible classes;
 - **Multiclass**: assign one of $n > 2$ possible classes;
 - **Multi-Label**: assign a *subset* of size $m \leq n$ of all the possible labels. Each data point can be associated with multiple labels simultaneously.
- **REGRESSION**: assigning a *real* value to an example;
- **RANKING**: ordering a set of examples, according to their importance with the task.

Unsupervised Learning tasks

- **Dimensionality reduction**: reduce the dimensionality of the data, maintaining as much information as possible. A common example is **Principal Component Analysis** (PCA);
- **Clustering**: clustering the data into **homogeneous groups**, according to their similarity;
- **Anomaly detection**: finding instances in the given examples, that contains errors. This can be used in example for the recognition of anomalous network traffic;

Probabilistic reasoning Reasoning in presence of uncertainty. It can be used for:

- evaluating the effect of a certain piece of evidence on other related variables;
- Estimate probability and relations between variables from a set of observation.

1.4 Choice of learning algorithms

Based on the information about the task available to us, we can choose the learning algorithms accordingly:

- **Full knowledge** of distribution of data: **Bayesian decision theory**. If we know the distribution of data and the parameters, we can build Bayesian Networks to model the distribution;

- Form of **distribution known, parameters unknown**: parameter estimation from training data. Once learned, we can use **generative methods** to predict the output, given the input;
- **Distribution unknown, training examples available: discriminative methods**, learn a function predicting the output, given the input. There are various methods for this, including:
 - Decision Trees;
 - Support Vector Machines;
 - Neural Networks;
 - etc.
- **Distribution unknown, training examples unavailable: unsupervised methods**.

All of these methods will be analyzed in the following sections.

2 Decision Tree Learning

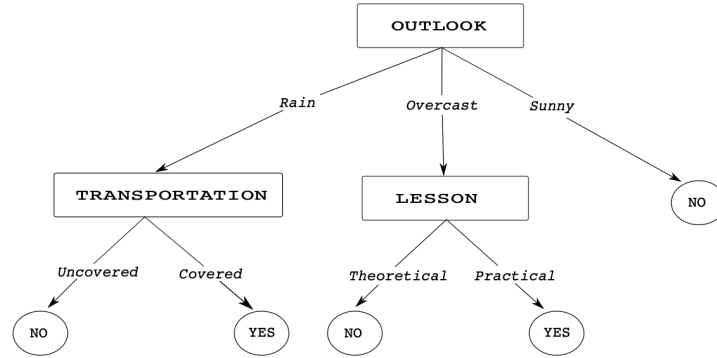


Figure 3: An example of a **learned decision tree**, *Go to lesson*

A **decision Tree** represents a *disjunction* ($\vee$: represent the different path to reach the same outcome) of *conjunction* ($\wedge$: represent the **and** between the conditions on the path to reach the leafs) of constraints over attribute values. As just described, it's used to encode *DNF* logical formulas:

- Each path from the root to a leaf is a **conjunction** of the **constraints** specified on the node along it. For example, in **fig3**, the leftmost path can be read as:

$$\text{OUTLOOK} = \text{RAIN} \wedge \text{TRANSPORTATION} = \text{Uncovered}$$

- The leaf contains the **label** to be assigned to instances reaching it;
- The **disjunction** of all paths is the logical formula represented by the tree.

2.1 Appropriate problems

There are several tasks where decision trees can be effectively applied. In particular, the task must have these features:

- Binary or multi-class **classification**;
- Instances are represented as **attribute value pair** (tabular data);
- Different explanation for the same class are possible (disjunction of different paths);
- Some instances have missing attributes;
- There is a **need** for an **interpretable explanation** for the output. In fact, decision trees can be easily visualized and understood by humans.

2.2 Learning decision tree

To learn **decision trees**, there is a simple **greedy top-down** strategy. Starting from the **root node** with the full training set, the algorithm follow these steps:

1. Choose best attribute to be evaluated (**fig3: outlook** is the first chosen attribute);
2. Add a child node for each attribute value (**fig3: rain, overcast and sunny** are **all possible values** for outlook);
3. Split the node training set into children nodes, according to the attribute values;
4. **Stop splitting** a node if all its training set belongs to a single class (**homogeneous leaf**), or there are no more attributes to test

2.2.1 Choosing the best attribute

To understand how we can choose the best attribute to be evaluated, we first have to define **entropy**.

Entropy: a measure of the **amount of information** contained in a **collection of instances** S which can take a number of c possible values (labels in our case). In simpler words, entropy quantifies the amount of uncertainty (or impurity) in a dataset:

- If all examples belong to the same class (homogeneous group), then entropy = 0
- If examples are evenly split among classes (uncertainty is maximal), then entropy is maximal.

It's defined as follows:

$$H(S) = - \sum_{i=1}^c p_i \log_2 p_i \quad (2.1)$$

Where:

- S is the collection of training examples, the dataset or a subset of it;
- p_i is the fraction of S , with label i . It represents the probability of finding the label i , inside S .

Information Gain: represents the **expected reduction of entropy**, obtained by partitioning S over an attribute A . It's defined as follows:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \quad (2.2)$$

Where:

- $H(S)$: entropy of the original set S (before the split);
- $\text{Values}(A)$: set of possible values, taken by the attribute A ;
- S_v : the subset of S , taking value v at attribute A ;
- The second term represents the **sum of entropies** of all the subsets S_v , obtained by partitioning over the attribute A , weighted by their respective sizes.

We can derive that an attribute with **high Information Gain** IG tends to produce homogeneous groups in terms of labels, thus favoring their classification.

2.3 Issues in decision tree learning

2.3.1 Overfitting avoidance

Requiring that each leaf contains only examples of a certain class can lead to very complex trees. A complex tree can easily overfit the training set, incorporating noise in the data.

To solve this problem, it's possible to accept impure leaves, assigning them the label of the majority of their training examples. This process is called **pruning**. There are two possible strategies to prune a decision tree:

- **Pre-pruning:** decide whether to stop splitting a node, even if it contains training examples with different labels.
- **Post-pruning:** learn the full tree and successively prune it, removing subtrees.

Reduces Error Pruning: a **post-pruning** strategy, using the validation set. The procedure follows these next steps:

1. for each node in the tree:
 - evaluate the performance on the validation set, after removing the subtree rooted at it;
2. If all node removals made the performance worse, STOP;
3. Choose the node that gave the best performance improvement;
4. Replace the subtree rooted at it with a leaf;

5. Assign to the lead the majority label of all the example in the subtree;
6. Return to 1.

2.3.2 Dealing with continuous-valued attributes:

Continuous valued attributes must be discretized in order to be used as internal node tests, or otherwise, we could end up with an infinite decision tree. The procedure follows these next steps:

1. Examples are **sorted** according to their continuous value;
2. For **each pair of successive examples**, having **different labels**, a **candidate threshold** is placed at the average of the two values;
3. For **each candidate threshold**, the **IG** obtained by splitting the examples, at the given threshold, is computed;
4. The threshold with the **highest IG** is used to discretize the attribute;

Example: consider the dataset in **table 1**:

Example	Attribute A	Class
1	2.5	+
2	3.0	+
3	4.5	-
4	5.0	-
5	6.0	+

Table 1: Example dataset with continuous attribute A

The dataset is already sorted according to attribute A. The candidate thresholds are:

- Between example 2 and 3: $(3.0 + 4.5) / 2 = 3.75$;
- Between example 4 and 5: $(5.0 + 6.0) / 2 = 5.5$;

We can then compute the IG for both thresholds, choosing the one with the highest value.

2.3.3 Alternative attributes test measure

The previously defined **Information Gain** criterion tends to **prefer attributes** with a **larger number of possible values**, since it will cause data to be split into very small subsets.

As an extreme case, if we consider a unique attribute in our data, such as an ID, it will cause the data to be **perfectly split into singletons**. This may look like a perfect split, but offers no generalization on new examples.

Attribute Value Entropy: To resolve this issue, a measure of the entropy of the dataset S , with respect to the values of an attribute A is defined:

$$H_A(S) = - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|} \quad (2.3)$$

- If, for a certain attribute (ex: ID), every value is unique, then $H_A(S)$ is very high;
- If a certain attribute has only a few values (ex: binary attribute), then $H_A(S)$ is very low;

Information gain Ratio: We can then define an **Information Gain Ratio** measure, that **down weights** the **IG** by the **Attribute Value Entropy**

$$IGR(S, A) = \frac{IG(S, A)}{H_A(S)} \quad (2.4)$$

Obviously, we have that:

- If the attribute A has many unique values, then $H_A(S)$ is high, resulting in a lower **IGR**, for the attribute A ;
- If the attribute A has few unique values, then $H_A(S)$ is low, resulting in a higher **IGR**.

2.3.4 Handling Attributes with missing values

Assuming an **example** x , belonging to **class** $c(x)$ has a missing value for the attribute A . When the attribute A is to be tested at node n , we can apply two solutions:

- **SIMPLE SOLUTION:** Assign, to the example x , the most common attribute A values among the training examples at node n ;
- **COMPLEX SOLUTION:** propagate x to each child of n , with a fractional value equal to the proportion of examples with the corresponding attribute value.
At test times for each candidate class (since more leaf nodes are reached), all fraction belonging to the same class are summed. The example is assigned the class with the highest overall value.

Example of complex solution: Consider the decision tree in **fig 3** and an example x with missing value for attribute **outlook**.

Suppose that, at node **outlook**, x is propagated to all three children:

- To the left child (**rain**): with fraction $\frac{5}{10}$ (in the dataset, 5 examples out of 10 have outlook = rain);

- To the middle child (**overcast**): with fraction $\frac{4}{10}$;
- To the right child (**sunny**): with fraction $\frac{1}{10}$;

Continuing the propagation, we reach the leafs:

- Leftmost leaf (**yes**): receives fraction $\frac{5}{10}$;
- Middle leaf (**yes**): receives fraction $\frac{4}{10}$;
- Rightmost leaf (**no**): receives fraction $\frac{1}{10}$;

Then, summing the fractions for each class:

- Class **yes**: total fraction = $\frac{5}{10} + \frac{4}{10} = \frac{9}{10}$;
- Class **no**: total fraction = $\frac{1}{10}$;

The example x is then assigned to class **yes**, since it has the highest total fraction.

2.4 Random Forests

Random Forests are an *ensemble learning method*, that combines multiple decision trees to improve classification or regression performance.

Training To train a Random Forest, the following steps are performed:

1. Given a training set of N examples, sample N examples with replacement (each example can be selected multiple times);
2. Train a decision tree on the sampled data, but at each split, choose the best split among a random subset of features, rather than all features. This approach allows to introduce more diversity among the trees;
3. Repeat steps 1 and 2 to create a forest of M trees.

Prediction To make predictions with a Random Forest, the following steps are performed:

- For classification tasks, each tree in the forest makes a prediction for the input example;
- The final prediction is made by majority voting among all trees.

A more in depth explanation of ensemble methods can be found in **section 16**.

3 K-Nearest Neighbors

To better understand this **non-parametric** machine learning algorithm, we will first analyze its **1-Nearest Neighbor** variant.

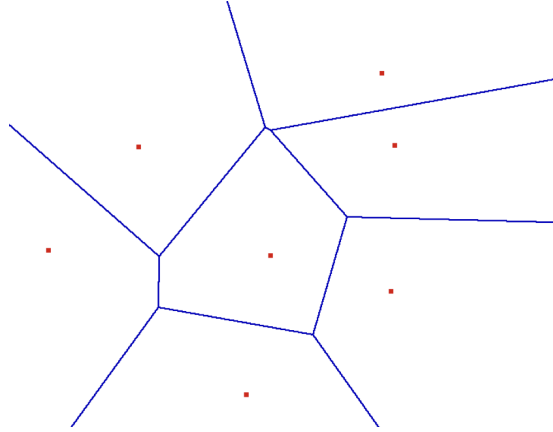


Figure 4: Example of a 1-Nearest Neighbor

- Each **red dot** represents one instance of the **training set**. Each instance has two features, and every label is unique;
- The **blue lines** represent the **decision boundaries**. Each region contains the set of points that are **closer** to the **red dot inside that region** than to any other red dot;
- For every instance of the test set, we can determine in **which region** it falls, and the predicted label will be the one associated with the red dot of that region.

3.1 Metric function

To implement this algorithm, we first have to define a **metric** to measure the **distance between instances**.

Given a set $\mathcal{X}$, a function $d : \mathcal{X} \times \mathcal{X} \rightarrow R_0^+$ is a **metric** iff for any $x, y, z \in \mathcal{X}$ the following properties are satisfied:

- **reflexivity**: $d(x, y) = 0$ iff $x = y$
- **symmetry**: $d(x, y) = d(y, x)$
- **triangle inequality**: $d(x, y) + d(y, z) \geq d(x, z)$

An example of **metric** is the **Euclidean distance in R^n** . It's defined as follows:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (3.1)$$

3.2 Classification

Algorithm 1 *k*-Nearest Neighbor Classification

```

1: for all test examples  $x$  do
2:   for all training examples  $(x_i, y_i)$  do
3:     compute distance  $d(x, x_i)$ 
4:   end for
5:   select the  $k$ -nearest neighbors of  $x$ 
6:   return class of  $x$  as majority class among neighbors:

```

$$\arg \max_y \sum_{i=1}^k \delta(y, y_i)$$

```

7: end for

```

Where:

$$\delta(x, y) = \begin{cases} 1 & \text{if } x = y \\ 0 & \text{otherwise} \end{cases}$$

Assigns a class to a test example by looking at the k training examples closest to it. Given a test example x , the predicted class of x is the majority class y among its K -nearest neighbors.

3.3 Regression

Algorithm 2 *k*-Nearest Neighbor Regression

```

1: for all test examples  $x$  do
2:   for all training examples  $(x_i, y_i)$  do
3:     compute distance  $d(x, x_i)$ 
4:   end for
5:   select the  $k$ -nearest neighbors of  $x$ 
6:   return the average output value among neighbors:

```

$$\frac{1}{K} \sum_{i=1}^k y_i$$

```

7: end for

```

Here, given a test example x , the predicted value of x is the mean of the values among its K -nearest neighbors.

3.4 Characteristics

- **Instance-based learning:** the model used for prediction is calibrated for test example to be processed. This model doesn't build a real general representation for data, but it's limited to the example on the training set.
- **Lazy learning:** computation is mostly deferred to the classification phase. In fact for every new example, we compare it with every other example in the training set. There is no real training phase.
- **Local learner:** assumes prediction should be mainly influenced by nearby instances.
- **Uniform feature weighting:** every feature contribute at the same way on computing distance. So there are not parameters to be learned.

3.5 Distance-weighted K-Nearest Neighbor

The just presented algorithm can be enhanced with its **Distance-Weighted K-Nearest Neighbor** variation.

Take as an example this case here, using 3-Nearest Neighbors:

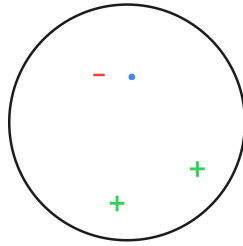


Figure 5: Example of issue with K-Nearest Neighbors

Even though the **blue dot**, representing the testing instance, is closer to the **- label**, the algorithm will still assign the **+ label**, since it represents the majority label among its neighbors.

We can weight the **influence** that each neighbor has on the label, based on its distance from the testing instance, giving closer neighbors a higher impact on the final classification.

We first define a new function:

$$w_i = \frac{1}{d(x, x_i)}$$

The algorithms now present as follows:

Classification

$$\arg \max_y \sum_{i=1}^k w_i \delta(y, y_i)$$

Regression

$$\frac{\sum_{i=1}^k w_i y_i}{\sum_{i=1}^k w_i}$$

4 Evaluation

In this section we will discuss *Evaluation*, the process of assessing a model's performance, reliability, and generalizability using specific quantitative metrics and qualitative techniques.

- Requires to define a **performance measures** to be optimized. These vary based on the task we are addressing;
- Performance of learning algorithms cannot be evaluated on the entire domain. We have to introduce some approximations;
- Performance evaluation is needed for
 - **Tune** the hyperparameters of learning methods;
 - **Evaluating** the performance of the learned model;
 - **Computing statistical significance** of results.

The performance measure usually differs from the loss function used during training. In fact:

- The **loss function** simply measures the cost paid for predicting $f(x)$ when the true output is y . It usually has to be differentiable (smooth) to allow optimization;
- The **performance measure** quantifies how well the model performs on a given task. Usually, different performance measures are used to test different aspects of the model.

4.1 Binary classification

The typical way to evaluate a binary classifier is by means of a **confusion matrix**: Where:

		prediction	
		P	N
ground truth	P	TP	FN
	N	FP	TN

Table 2: Confusion matrix for binary classification

- The rows represent the **actual** classes (ground truth);
- The columns represent the **predicted** classes;
- Each entry in the matrix represents the number of examples that fall into that category:

- **TP** (True Positive): number of positive examples correctly classified as positive;
- **TN** (True Negative): number of negative examples correctly classified as negative;
- **FN** (False Negative): number of positive examples incorrectly classified as negative;
- **FP** (False Positive): number of negative examples incorrectly classified as positive;

From the confusion matrix we can derive several performance measures.

4.1.1 Accuracy

Accuracy

Accuracy is the fraction of correctly labelled example among all predictions.

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \quad (4.1)$$

Accuracy metric is not very reliable when the classes are imbalanced. For example, if in a dataset the positive class represents a small fraction of the examples, a classifier that always predicts negative will achieve a high accuracy, but it wouldn't be able to predict any positive example.

A possible solution consists in **rebalancing costs**, making each positive example worth more than a negative one (e.g., each positive counts $\frac{N}{P}$, where N = number of negative examples, and P = number of positive examples).

4.1.2 Precision and Recall

Precision

Precision is the fraction of correctly predicted positive examples among all predicted positive examples.

$$Prec = \frac{TP}{TP + FP} \quad (4.2)$$

This metric measures the precision of the learner in predicting positive examples. A problem with precision is that the learner can achieve a high precision by predicting only a few positive examples, only when 100% sure.

Recall

Recall (or sensitivity) is the fraction of correctly predicted positive examples among all actual positive examples.

$$Rec = \frac{TP}{TP + FN} \quad (4.3)$$

It measures the ability of the learner to find all positive examples.

Since these last two metrics are complementary, we can combine them into a single one, called **F-measure**.

F-measure

F-measure combines **precision** and **recall**, balancing their trade-off.

$$F_{\beta} = \frac{(1 + \beta^2) \cdot Prec \cdot Rec}{\beta^2 \cdot Prec + Rec} \quad (4.4)$$

F_1 is usually a good performance measure in the case of unbalanced datasets.

4.1.3 Precision-Recall curve

The **Precision-Recall curve** is a graphical representation that illustrates the trade-off between precision and recall for different threshold values.

Many classifiers output a confidence (probability score) for each prediction. All the evaluation metrics seen so far measure the performance of the classifier at a specific threshold. We can vary this threshold from min to max to obtain different pairs of precision and recall values, which can be plotted to create the Precision-Recall curve.

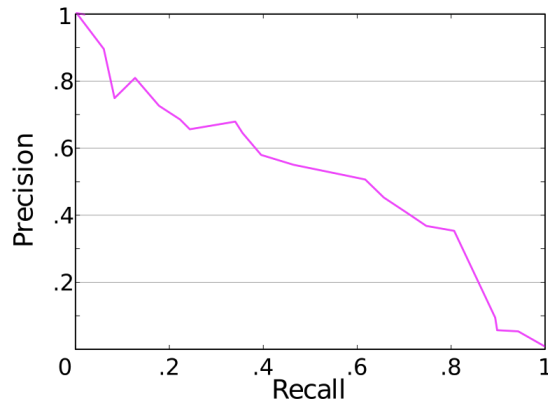


Figure 6: Precision-Recall curve example

Also, a single value can be computed by computing the **area under the curve**. It combines the performance of the classifier across all thresholds into a single metric.

4.2 Multi-class classification

In the case of multi-class classification, we can simply extend the concepts of precision, recall, and F-measure to multiple classes. We first redefine the **confusion matrix** as a generalization of the binary case:

		prediction		
		y_1	y_2	y_3
ground truth	y_1	n_{11}	n_{12}	n_{13}
	y_2	n_{21}	n_{22}	n_{23}
	y_3	n_{31}	n_{32}	n_{33}

Table 3: Confusion matrix for multi-class classification (3 classes example)

Where:

- n_{ij} is the number of examples of class y_i predicted as class y_j ;
- The **main diagonal** elements n_{ii} represent the number of correctly classified examples for each class, the **true positives**;
- The **sum** of off-diagonal elements in a **column** j ($\sum_{i \neq j} n_{ij}$) represents the number of **false positives** for class y_j ;
- The **sum** of off-diagonal elements in a **row** i ($\sum_{j \neq i} n_{ij}$) represents the number of **false negatives** for class y_i ;

From this confusion matrix, we can compute precision and recall for each class y_i as follows:

- **Precision** for class y_i :

$$Prec_i = \frac{n_{ii}}{\sum_j n_{ji}}$$

- **Recall** for class y_i :

$$Rec_i = \frac{n_{ii}}{\sum_j n_{ij}}$$

In addition, we can extend the definition of accuracy to the multiclass setting.

Multiclass accuracy

Multiclass accuracy is the fraction of correctly predicted examples among

all examples.

$$MAcc = \frac{\sum_i n_{ii}}{\sum_i \sum_j n_{ij}} \quad (4.5)$$

4.3 Regression

The typical performance measures in the regression domain is the **Root Mean Squared Error** (RMSE). It represents the **distance** between the predicted values $f(x_i)$ and the actual values y_i .

Root Mean Squared Error

Root Mean Squared Error is defined as:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (f(x_i) - y_i)^2} \quad (4.6)$$

For a dataset $\mathcal{D}$ with $|\mathcal{D}| = N$ examples.

4.4 Performance estimation

Estimating the performance of a learning algorithm on training data is optimistically biased because, as the model has already seen those examples. To correctly estimate the performance, we need to evaluate the model on unseen data. To ensure this, we can use two main strategies.

4.4.1 Hold-out procedure

Given a dataset $\mathcal{D}$, we split it into three disjoint subsets:

- **Training set** $\mathcal{D}_{train}$: used to train the model;
- **Validation set** $\mathcal{D}_{val}$: used to tune hyper-parameters and select the best model;
- **Test set** $\mathcal{D}_{test}$: used to evaluate the final performance of the selected model.

This procedure is simple and effective, but can be applied only when we have a sufficiently large dataset available.

4.4.2 K-Fold Cross-validation

The **k-fold** cross validation approach **splits** the dataset $\mathcal{D}$ into k disjoint subsets (folds) of equal size. Then, for each $i \in [1, k]$:

- **Train** a predictor $\mathcal{L}$ using dataset $T_i = \mathcal{D} \setminus \mathcal{D}_i$ (all folds except the i -th one);
- **Compute score** S_i of the predictor $\mathcal{L}(T_i)$:

$$S_i = S_{\mathcal{D}_i}[\mathcal{L}(T_i)]$$

After this iterative procedure, we can compute the **average score** over all folds:

$$\bar{S} = \frac{1}{k} \sum_{i=1}^k S_i$$

When applying k-fold cross-validation we obviously get k different performances scores $S_1, S_2, \dots, S_k$. The average score $\bar{S}$ is a good estimate of the model's performance, but it is also useful to compute the **variance** of these scores to understand the stability of the model across different data splits.

Assuming that each fold is independent, we can compute the variance as:

$$\text{Var}(\bar{S}) = \text{Var}\left[\frac{S_1, S_2, \dots, S_k}{k}\right]$$

We can take out the constant $\frac{1}{k}$ as $\text{Var}(aX) = a^2 \text{Var}(X)$:

$$\text{Var}(\bar{S}) = \frac{1}{k^2} \sum_{j=1}^k \text{Var}(S_j)$$

We cannot compute $\text{Var}(S_j)$ directly, but we can estimate it using the sample variance formula:

$$\text{Var}(S_j) \approx \frac{1}{k-1} \sum_{i=1}^k (S_i - \bar{S})^2$$

Combining the two equations, we get:

$$\begin{aligned} \text{Var}(\bar{S}) &\approx \frac{1}{k^2} \sum_{j=1}^k \underbrace{\left(\frac{1}{k-1} \sum_{i=1}^k (S_i - \bar{S})^2 \right)}_{\text{Constant wrt } j} \\ &= \frac{1}{k^2} \cdot k \cdot \left(\frac{1}{k-1} \sum_{i=1}^k (S_i - \bar{S})^2 \right) = \frac{1}{k(k-1)} \sum_{i=1}^k (S_i - \bar{S})^2 \end{aligned}$$

4.5 Hypothesis testing

When comparing two learning algorithms, we want to determine if the observed difference in performance is statistically significant or if it could have occurred by chance. Hypothesis testing provides a framework to make this determination.

Given the hypotheses that we want to test (e.g. "Algorithm A is better than Algorithm B "), we first need to define:

- **Null hypothesis** (H_0): the one rejecting our claim. It usually states that there is no difference between the two algorithms;
- **Test statistic**: a function T over samples of random variables that summarizes the data. Its value is used to decide whether to reject the null hypothesis.

4.5.1 T-test

In the **T-test**, the test statistic is given by the standardized mean, defined as:

$$T = \frac{\bar{X} - \mu_0}{\sqrt{\text{Var}[\bar{X}]}} \quad (4.7)$$

Assuming that the samples come from an unknown Normal distribution, the test statistic T follows a t-distribution with $k - 1$ degrees of freedom (k is the number of samples).

The null hypothesis H_0 is rejected iff the computed statistic T is greater than a critical value $t_{\alpha, k-1}$. Formally, in a two-tailed test:

$$H_0 \text{ rejected iff } |T| > t_{\alpha/2, k-1} \quad (4.8)$$

Where α is the significance level (usually set to 0.05, means that we accept a 5% chance of rejecting the null hypothesis when it's actually true).

4.5.2 Example: comparing two algorithms

Suppose we have two learning algorithms A and B , and we want to compare their performance using k -fold cross-validation. We perform the following steps:

- Run *k-fold cross-validation* for both algorithms, obtaining two sets of performance scores;
- Compute the **mean difference** $\hat{\delta}$:

$$\hat{\delta} = \frac{1}{k} \sum_{i=1}^k \delta_i$$

Where δ_i is the difference in performance scores for fold i . It is computed as:

$$\delta_i = S_{D_i}[L_A(T_i)] - S_{D_i}[L_B(T_i)]$$

This is simply the difference between the scores of algorithm A and B on fold i .

- The null hypothesis H_0 states that there is no difference in performance between the two algorithms, meaning that $\mu_0 = 0$;

- Compute the **variance** of the differences:

$$Var[\hat{\delta}] = \frac{1}{k(k-1)} \sum_{i=1}^k (\delta_i - \hat{\delta})^2$$

- Compute the **test statistic** T :

$$T = \frac{\hat{\delta} - 0}{\sqrt{Var[\hat{\delta}]} } = \frac{\hat{\delta}}{\sqrt{Var[\hat{\delta}]}}$$

- Compare $|T|$ with the critical value $t_{\alpha/2, k-1}$, that can be found in t-distribution tables. If $|T| > t_{\alpha/2, k-1}$, we reject the null hypothesis H_0 and conclude that there is a statistically significant difference between the two algorithms.

5 Parameter Estimation

Setting In order to understand what we will be doing in this section, we need to first introduce the setting of the environment we are working in. We have:

- A **collection of data** sampled from a **probability distribution** $p(x, y)$;
- The **probability distribution** $p(x, y)$ is **known**, but the parameters θ of the distribution are **unknown**;
- There is a **training set** $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ of N examples, sampled **i.i.d.** from the distribution $p(x, y)$;

I.I.D.

Independent and Identically Distributed (i.i.d.) refers to a set of random variables that are all drawn from the same probability distribution and are mutually independent.

The **task** is to estimate the parameters θ of the distribution $p(x, y)$ using the training set $\mathcal{D}$, in order to make predictions on new examples.

Multiclass classification setting:

- The training set $\mathcal{D}$ can be divided into c subsets $\mathcal{D}_1, \dots, \mathcal{D}_c$, where each $\mathcal{D}_i = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ contains examples of class y_i ;
- For any new example $\mathbf{x}$, we want to be able to compute the posterior probability for each class y_i (what's the probability of y_i given $\mathbf{x}$) using Bayes' theorem:

$$p(y_i|\mathbf{x}, \mathcal{D}) = \frac{p(\mathbf{x}|y_i, \mathcal{D})p(y_i|\mathcal{D})}{p(\mathbf{x}|\mathcal{D})} \quad (5.1)$$

A posteriori probability

The probability of a class y_i given the features $\mathbf{x}$ and the training set $\mathcal{D}$, denoted as $p(y_i|\mathbf{x}, \mathcal{D})$, is called the posterior probability. It represents the probability of x belonging to class y_i after observing the data $\mathcal{D}$.

Simplification: Starting from the equation 5.1, we can simplify it by making some assumptions:

- We assume $\mathbf{x}$ is independent of $\mathcal{D}_j (j \neq i)$, so:

$$P(y_i|\mathbf{x}, \mathcal{D}) = \frac{p(\mathbf{x}|y_i, \mathcal{D}_i)p(y_i|\mathcal{D})}{p(\mathbf{x}|\mathcal{D})} \quad (5.2)$$

- Without additional knowledge, $P(y_i|\mathcal{D})$ can be computed as the fraction of examples in class y_i in the training set:

$$P(y_i|\mathcal{D}) = \frac{|\mathcal{D}_i|}{|\mathcal{D}|}$$

- The denominator $p(\mathbf{x}|\mathcal{D})$ can be computed using the **law of total probability**:

$$p(\mathbf{x}|\mathcal{D}) = \sum_{i=1}^c p(\mathbf{x}|y_i, \mathcal{D})p(y_i|\mathcal{D})$$

At this point the only factor of equation 5.2 that remains unknown is $p(\mathbf{x}|y_i, \mathcal{D}_i)$. In order to estimate it, we need to estimate the class dependent parameter θ_i of the distribution.

This can be done in two ways:

1. **Maximum Likelihood Estimation (MLE):** we assume that the parameter θ_i is a fixed but unknown values. We estimate it by maximizing the likelihood of the observed data $\mathcal{D}_i$. Obtained values are used to compute probabilities of new examples:

$$p(\mathbf{x}|y_i, \mathcal{D}_i) \approx p(\mathbf{x}|\theta_i)$$

2. **Bayesian estimation:** we assume that the parameter θ_i is a random variable with a prior distribution $p(\theta_i)$. Observed data turns the prior into a posterior distribution. The prediction for new examples is obtained by integrating over all possible values of θ_i :

$$p(\mathbf{x}|y_i, \mathcal{D}_i) = \int_{\theta_i} p(\mathbf{x}, \theta_i|y_i, \mathcal{D}_i) d\theta_i$$

5.1 Maximum Likelihood Estimation

Maximum Likelihood Estimation (**MLE**) is a method used to estimate the parameters of a statistical model. Given a set of training data $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, the estimation of the parameter θ is done by finding the parameters with the highest likelihood of generating the observed data (highest posterior probability).

$$\theta_i^* = \operatorname{argmax}_{\theta} p(\theta_i|\mathcal{D}_i, y_i) = \operatorname{argmax}_{\theta_i} \frac{p(\mathcal{D}_i, y_i|\theta_i)p(\theta_i)}{p(\mathcal{D}_i, y_i)} \quad (5.3)$$

Since the denominator is independent of θ_i , we can ignore it for the purpose of maximization:

$$\theta_i^* = \operatorname{argmax}_{\theta_i} p(\mathcal{D}_i, y_i | \theta_i) p(\theta_i)$$

This equation assumes that a prior distribution $p(\theta_i)$ is given.

The most common form of MLE, which doesn't take into account the prior, but only the training samples, is given by:

$$\theta^* = \operatorname{argmax}_{\theta} p(\mathcal{D}, y | \theta) \quad (5.4)$$

Since the examples are **i.i.d.**, the joint probability over D can be expressed as the product of the individual probabilities:

$$\theta^* = \operatorname{argmax}_{\theta} \prod_{j=1}^n p(\mathbf{x}_j | \theta) \quad (5.5)$$

To find the value of θ that maximizes the likelihood, we would have to compute the derivative of the likelihood function and set it to zero. This is often difficult to compute, so instead we work with the **log-likelihood**.

5.1.1 Maximizing the log-likelihood

The **log-likelihood** is defined as the logarithm of the likelihood function (equation 5.5):

$$\theta^* = \operatorname{argmax}_{\theta} \sum_{j=1}^n \ln p(\mathbf{x}_j | \theta) \quad (5.6)$$

The logarithm is a **monotonically increasing function**, so maximizing the log-likelihood is equivalent to maximizing the likelihood itself. The logarithm transforms the product of probabilities into a sum, making it easier to compute the derivative.

In order to **find the maximum**, we compute the derivative of the log-likelihood wrt θ and set it to zero:

$$\nabla_{\theta} \sum_{j=1}^n \ln p(\mathbf{x}_j | \theta) = 0 \quad (5.7)$$

Solving this equation will give us a local or global maximum of the log-likelihood function, depending on the shape of the function.

5.1.2 Example: MLE for Gaussian distribution

Assuming that the data is generated from a Gaussian distribution, we can estimate the parameters μ and σ^2 using MLE.

First, the probability density function of a Gaussian distribution is given by:

$$p(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Considering its logarithm, we have:

$$\sum_{j=1}^n \ln p(\mathbf{x}_j|\theta) = \sum_{j=1}^n \ln \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(\mathbf{x}_j - \mu)^2}{2\sigma^2} \right) \right] = \sum_{j=1}^n \ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{(\mathbf{x}_j - \mu)^2}{2\sigma^2}$$

Estimating μ :

$$\frac{\partial \log p(\mathbf{x}|\theta)}{\partial \mu} = \frac{\partial}{\partial \mu} \sum_{j=1}^n \left[\ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{(\mathbf{x}_j - \mu)^2}{2\sigma^2} \right]$$

The derivative of the first term is zero, so we only need to compute the derivative of the second term, which is:

$$= \sum_{j=1}^n -\frac{1}{2\sigma^2} \cdot 2(\mathbf{x}_j - \mu)(-1) = \sum_{j=1}^n \frac{(\mathbf{x}_j - \mu)}{\sigma^2}$$

Setting this quantity to zero, we have:

$$\begin{aligned} \sum_{j=1}^n \frac{(\mathbf{x}_j - \mu)}{\sigma^2} &= 0 \\ \sum_{j=1}^n (\mathbf{x}_j - \mu) &= 0 \\ \sum_{j=1}^n \mathbf{x}_j - \sum_{j=1}^n \mu &= 0 \implies \sum_{j=1}^n \mathbf{x}_j - n\mu = 0 \\ \mu &= \frac{1}{n} \sum_{j=1}^n \mathbf{x}_j \end{aligned}$$

Which is the **sample mean** of the data.

Estimating σ : Repeating the same process for σ , we have:

$$\frac{\partial \log p(\mathbf{x}|\theta)}{\partial \sigma} = \frac{\partial}{\partial \sigma} \sum_{j=1}^n \left[\ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{(\mathbf{x}_j - \mu)^2}{2\sigma^2} \right]$$

Focusing on the first term, we rewrite it as:

$$\ln \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) = \ln([2\pi\sigma^2]^{-\frac{1}{2}}) = -\frac{1}{2} \ln(2\pi\sigma^2)$$

Using the properties of logarithms, we can separate the terms:

$$= -\frac{1}{2} \ln(2\pi) - \frac{1}{2} \ln(\sigma^2)$$

Since the term $-\frac{1}{2} \ln(2\pi)$ is independent of σ , its derivative is zero. We only need to compute the derivative of:

$$\begin{aligned} &= \frac{\partial}{\partial \sigma} \sum_{j=1}^n \left[-\frac{1}{2} \ln(\sigma^2) - \frac{(\mathbf{x}_j - \mu)^2}{2} (\sigma^{-2}) \right] \\ &= \sum_{j=1}^n -\frac{1}{\sigma} + \frac{(\mathbf{x}_j - \mu)^2}{\sigma^3} \end{aligned}$$

if we set the quantity to zero, we have:

$$\sum_{j=1}^n -\frac{1}{\sigma} + \frac{(\mathbf{x}_j - \mu)^2}{\sigma^3} = 0$$

Multiplying by σ^3 , we have:

$$\begin{aligned} \sum_{j=1}^n -\sigma^2 + (\mathbf{x}_j - \mu)^2 &= 0 \\ \sigma^2 &= \frac{1}{n} \sum_{j=1}^n (\mathbf{x}_j - \mu)^2 \end{aligned}$$

Which is the **sample variance** of the data.

Conclusion: By this two derivations, we have found that the maximum likelihood estimators for the parameters of a Gaussian distribution are simply their estimates over the samples:

- Gaussian mean μ is estimated by the sample mean;
- Gaussian variance σ^2 is estimated by the sample variance.

5.2 Bayesian Estimation

With this approach we focus on the estimation of the parameters of our distribution $p(\mathbf{x}, y)$ a priori, meaning that we update our knowledge about the parameters θ based on the observed data.

We start by assuming that the parameters θ_i are **random variables** with a known **prior distribution**.

Predictions for new examples are obtained by integrating over all possible values of θ_i :

$$p(\mathbf{x}|y_i, \mathcal{D}_i) = \int_{\theta_i} p(\mathbf{x}, \theta_i | y_i, \mathcal{D}_i) d\theta_i$$

Since probability of $\mathbf{x}$ given each class y_i is independent of the other classes, we can simplify the equation as:

$$p(\mathbf{x}|\mathcal{D}) = \int_{\theta} p(\mathbf{x}, \theta | \mathcal{D}) d\theta$$

Using the definition of conditional probability, we can rewrite the equation as:

$$p(\mathbf{x}|\mathcal{D}) = \int_{\theta} p(\mathbf{x}|\theta, \mathcal{D})p(\theta|\mathcal{D})d\theta$$

In computing $p(\mathbf{x}|\theta, \mathcal{D})$ we can assume that $\mathbf{x}$ is independent of $\mathcal{D}$ given θ , since the parameter θ captures all the information about the distribution. Thus, we have:

$$p(\mathbf{x}|\mathcal{D}) = \int_{\theta} p(\mathbf{x}|\theta)p(\theta|\mathcal{D})d\theta$$

- The term $p(\mathbf{x}|\theta)$ can be easily computed since both the distribution and the initial parameters θ are known;
- The term $p(\theta|\mathcal{D})$ is the **posterior distribution** of the parameter θ given the data $\mathcal{D}$.

Using Bayes' theorem, we can express the posterior distribution as:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})} \quad (5.8)$$

Where $p(\mathcal{D})$ is a constant independent of θ , so we can ignore it for the purpose of estimation.

5.2.1 Example: Bayesian estimation for Gaussian distribution

Unknown μ , known σ : Assuming that the data is drawn from a Gaussian distribution $p(\mathbf{x}|\mu) \sim \mathcal{N}(\mu, \sigma^2)$.

The prior distribution for $\theta = \mu$ is also a Gaussian distribution: $p(\mu) \sim \mathcal{N}(\mu_0, \sigma_0^2)$. Where:

- μ_0 is the prior mean (our initial guess for the mean of the data);
- σ_0^2 is the prior variance (our initial uncertainty about the mean of the data).

Using Bayes' theorem, we can compute the posterior distribution for μ given the data $\mathcal{D}$. Starting from the equation 5.8 we have:

$$p(\mu|\mathcal{D}) = \frac{p(\mathcal{D}|\mu)p(\mu)}{p(\mathcal{D})}$$

Since we have already seen that $\frac{1}{p(\mathcal{D})}$ is independent of μ , we can replace it with a constant α . Since the examples are i.i.d we can derive:

$$p(\mu|\mathcal{D}) = \alpha p(\mathcal{D}|\mu)p(\mu) = \alpha \prod_{j=1}^n p(\mathbf{x}_j|\mu)p(\mu)$$

From here we can substitute the two Gaussian distributions:

1. The distribution of the data given μ ;

$$p(\mathbf{x}_j|\mu) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\mathbf{x}_j - \mu)^2}{2\sigma^2}\right)$$

2. The prior distribution of μ :

$$p(\mu) = \frac{1}{\sqrt{2\pi}\sigma_0} \exp\left(-\frac{(\mu - \mu_0)^2}{2\sigma_0^2}\right)$$

Substituting these into our equation for the posterior distribution, we get:

$$p(\mu|\mathcal{D}) = \alpha \prod_{j=1}^n \left(\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\mathbf{x}_j - \mu)^2}{2\sigma^2}\right) \right) \left(\frac{1}{\sqrt{2\pi}\sigma_0} \exp\left(-\frac{(\mu - \mu_0)^2}{2\sigma_0^2}\right) \right)$$

Which simplifies to:

$$\begin{aligned} p(\mu|\mathcal{D}) &= \alpha \prod_{j=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2} \frac{(\mathbf{x}_j - \mu)^2}{\sigma^2}\right] \frac{1}{\sqrt{2\pi}\sigma_0} \exp\left[-\frac{1}{2} \frac{(\mu - \mu_0)^2}{\sigma_0^2}\right] \\ &= \alpha' \exp\left[-\frac{1}{2} \left(\sum_{j=1}^n \left(\frac{\mu - \mathbf{x}_j}{\sigma} \right)^2 + \left(\frac{\mu - \mu_0}{\sigma_0} \right)^2 \right)\right] \\ &= \alpha'' \exp\left[-\frac{1}{2} \left[\left(\frac{n}{\sigma^2} + \frac{1}{\sigma_0^2} \right) \mu^2 - 2 \left(\frac{1}{\sigma^2} \sum_{j=1}^n \mathbf{x}_j + \frac{\mu_0}{\sigma_0^2} \right) \mu \right]\right] \end{aligned}$$

We can in fact see that this composition still behaves like a normal distribution itself.

Solving for μ_n and σ_n^2 we have:

$$\mu_n = \frac{n\sigma_0^2}{n\sigma_0^2 + \sigma^2} \hat{\mu}_n + \frac{\sigma^2}{n\sigma_0^2 + \sigma^2} \mu_0$$

Where $\hat{\mu}_n$ is the sample mean of the data;

$$\sigma_n^2 = \frac{\sigma_0^2 \sigma^2}{n\sigma_0^2 + \sigma^2}$$

Interpretation: The Interpretation of the results are the following:

- the **mean** of the posterior distribution μ_n is a **linear combination** of the prior mean μ_0 and the sample mean $\hat{\mu}_n$;
- the **more training examples** (n) are seen, the more the sample mean $\hat{\mu}_n$ influences the posterior mean μ_n (the sample mean has n both in the numerator and denominator, while the prior mean has only constants in the denominator);
- the more training examples are seen, the **smaller** the variance σ_n^2 of the posterior distribution becomes, making the distribution sharper around the mean.

Computing the conditional density Finally, we can compute the conditional density $p(\mathbf{x}|\mathcal{D})$ using the posterior distribution:

$$p(\mathbf{x}|\mathcal{D}) = \int p(\mathbf{x}|\mu)p(\mu|\mathcal{D})d\mu$$

$$\sim \mathcal{N}(\mu_n, \sigma^2 + \sigma_n^2)$$

Where:

- μ_n is the posterior mean computed before (the estimation of the mean after seeing the data);
- $\sigma^2 + \sigma_n^2$ is the sum of the variance of the original distribution and the variance of the posterior distribution.

5.3 Sufficient statistics

Any function on a set of examples $\mathcal{D}$ is called a statistic.

Sufficient statistic

A statistic $\mathbf{s} = \phi(\mathcal{D})$ is said to be sufficient for a parameter θ if:

$$P(\mathcal{D}|\mathbf{s}, \theta) = P(\mathcal{D}|\mathbf{s})$$

If θ is a random variable, then a **sufficient statistic** contains all the information that the sample $\mathcal{D}$ provides about the parameter θ .

$$p(\theta|\mathcal{D}, \mathbf{s}) = p(\theta|\mathbf{s})$$

A sufficient statistic allows to summarize a sample $\mathcal{D}$ into a smaller set of values $\mathbf{s}$, without losing information. For example **sample mean** and **covariance** are sufficient statistics for the parameters of a Gaussian distribution.

5.4 Conjugate priors

Conjugate prior

Given:

- A likelihood function $p(x|\theta)$;
- A prior distribution on the parameter θ , $p(\theta)$;

The prior distribution $p(\theta)$ is said to be a **conjugate prior** for the likelihood function $p(x|\theta)$ if the posterior distribution $p(\theta|x)$ is in the same family as the prior distribution $p(\theta)$.

This means that if we start with a prior distribution that is conjugate to the likelihood function, after observing data and updating our beliefs, the posterior distribution will also be in the same family as the prior.

5.5 Example of parameter estimation

In this section we will see an example of both MLE and Bayesian estimation for a Bernoulli distribution.

5.5.1 Bernoulli distribution

Consider a **Bernoulli distribution** with events $x = 1$ for success and $x = 0$ for failure. Parameters θ is the probability of success. The **probability mass function** is given by:

$$P(x|\theta) = \theta^x (1 - \theta)^{1-x}$$

The **conjugate prior** is a **Beta distribution** that depends on α_t (number of successes) and α_h (number of failures):

$$P(\theta|\alpha_t, \alpha_h) = \frac{\Gamma(\alpha)}{\Gamma(\alpha_h)\Gamma(\alpha_t)} \theta^{\alpha_t-1} (1 - \theta)^{\alpha_h-1}$$

In the following, we will be using both MLE and Bayesian estimation to estimate the parameter θ of a Bernoulli distribution.

Maximum likelihood estimation: Suppose that a dataset $\mathcal{D} = \{H, H, T, T, T, H, H\}$ of n realizations is given.

Since every toss is independent, the likelihood of the data is given by the product of the individual probabilities:

$$P(\mathcal{D}|\theta) = \theta \cdot \theta \cdot (1 - \theta) \cdot (1 - \theta) \cdot (1 - \theta) \cdot \theta \cdot \theta = \theta^h (1 - \theta)^t$$

Where h is the number of heads and t is the number of tails in the dataset. The maximum likelihood estimation of θ is given by:

$$\begin{aligned} \frac{\partial}{\partial \theta} \ln p(\mathcal{D}|\theta) = 0 &\implies \frac{\partial}{\partial \theta} (h \ln \theta + t \ln(1 - \theta)) = 0 \\ \implies \frac{h}{\theta} - \frac{t}{1 - \theta} = 0 &\implies h(1 - \theta) = t\theta \implies \theta = \frac{h}{h + t} \end{aligned}$$

The maximum likelihood estimation of θ is simply the fraction of heads in the dataset. t, h are sufficient statistics for the Bernoulli distribution.

Bayesian estimation: As we said before, the posterior distribution is given by:

$$P(\theta|\mathcal{D}) \propto P(\mathcal{D}|\theta)P(\theta)$$

Substituting $P(\mathcal{D}|\theta)$ and $P(\theta)$ we have:

$$P(\theta|\mathcal{D}) \propto \underbrace{\theta^h(1 - \theta)^t}_{P(\mathcal{D}|\theta)} \cdot \underbrace{\theta^{\alpha_h - 1}(1 - \theta)^{\alpha_t - 1}}_{P(\theta)}$$

The prediction on a new example is given by:

$$\begin{aligned} P(x|\mathcal{D}) &= \int P(x|\theta)P(\theta|\mathcal{D})d\theta \\ &= \int \theta P(\theta|\mathcal{D})d\theta \end{aligned}$$

This integral is the expected value of θ under the posterior distribution. Since the posterior distribution is a Beta distribution with parameters $(\alpha_h + h, \alpha_t + t)$, the expected value is given by:

$$E[\theta|\mathcal{D}] = \frac{\alpha_h + h}{\alpha_h + \alpha_t + h + t}$$

This is basically the same as the MLE, but with the addition of the prior counts α_h and α_t . Analyzing our result, we can see that:

- Our prior knowledge about θ is encoded in $\alpha_h + \alpha_t$ imaginary observations;
- As the number of observations $h + t$ increases, the influence of the prior decreases;
- When $\alpha_h = \alpha_t = 0$, the Bayesian estimation reduces to the MLE.

5.5.2 Multinomial distribution

Similarly to the Bernoulli distribution, we can consider a **Multinomial distribution** with r possible outcomes. The parameter θ is a vector of probabilities for each outcome, where $\sum_{i=1}^r \theta_i = 1$. The **probability mass function** is given by:

$$P(x|\theta) = \prod_{k=1}^r \theta_k^{x_k}$$

Where x_k is the number of times outcome k is observed.

The conjugate prior is a **Dirichlet distribution** that depends on parameters $\alpha_1, \alpha_2, \dots, \alpha_r$:

$$P(\theta|\alpha) = \frac{\Gamma(\alpha)}{\prod_{k=1}^r \Gamma(\alpha_k)} \prod_{k=1}^r \theta_k^{\alpha_k - 1}$$

Ignoring the derivation steps, we can summarize the results for both MLE and Bayesian estimation:

Maximum likelihood estimation: The maximum likelihood estimation of θ_k is given by:

$$\theta_k = \frac{N_k}{N}$$

Where N_k is the number of times outcome k is observed and N is the total number of observations.

Bayesian estimation: Again, the integral for the prediction on a new example is equivalent to the expected value of θ_k under the posterior distribution.

$$E[\theta_k|\mathcal{D}] = \frac{N_k + \alpha_k}{N + \alpha}$$

6 Bayesian Networks

All probabilistic inference and learning methods are based on the repeated application of basic rules of probability theory. However, as the number of random variables increases, the complexity of representing and manipulating probability distributions grows exponentially. Probabilistic graphical models provide a convenient framework to represent and manipulate complex probability distributions. Bayesian Networks (BNs) are a formalism to represent probabilistic relationships among a set of random variables. They allow one to:

- Visualize the structure of a probabilistic model;
- Discover properties of the model (e.g., conditional independencies);
- Efficiently perform probabilistic inference and learning, using the structure of the graph;
- Represent multiple distributions with the same graph, independently of their qualitative aspects (e.g., discrete versus continuous variables).

6.1 Bayesian Networks structure

A Bayesian Network (BN) structure $\mathcal{G}$ is a directed acyclic graph (DAG) where:

- Each node represents a random variable;
- Each directed edge represents a direct dependency between the connected variables;

It is important to note that the absence of a direct edge between two nodes does not imply that the variables are unconditionally independent.

The given structure $\mathcal{G}$ of a BN encodes these independence assumptions:

$$\mathcal{I}_l(\mathcal{G}) = \{\forall i \mathbf{x}_i \perp \text{NonDescendants}_{\mathbf{x}_i} \mid \text{Parents}_{\mathbf{x}_i}\} \quad (6.1)$$

Each variable $\mathbf{x}_i$ is independent of its non-descendants given its parents.

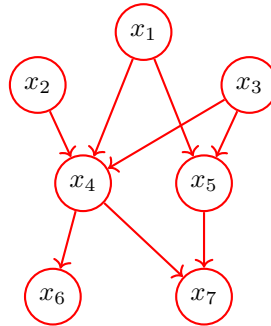


Figure 7: Example of Bayesian Network.

6.1.1 Independency Map

Independency Map

- Let p be a distribution over a set of random variables $\mathbf{X}$;
- Let $\mathcal{I}(p)$ be the set of independencies that hold in p ;

We say that $\mathcal{G}$ is an **Independency Map** (I-map) for p if p satisfies the local independencies encoded in $\mathcal{G}$:

$$\mathcal{I}_l(\mathcal{G}) \subseteq \mathcal{I}(p)$$

This means that all independencies encoded in the graph $\mathcal{G}$ hold in the distribution p . There can be additional independencies in p that are not encoded in $\mathcal{G}$.

6.1.2 Factorization

We say that a distribution p **factorizes** according to a BN structure $\mathcal{G}$ if p can be expressed as:

$$p(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m) = \prod_{i=1}^m p(\mathbf{x}_i \mid \text{Parents}_{\mathbf{x}_i})$$

We have that $\mathcal{G}$ is an I-map for p if and only if p factorizes according to $\mathcal{G}$.

Proof. (*I-Map* $\Rightarrow$ *Factorization*)

- Assume that $\mathcal{G}$ is an I-map for p ;
- We know that p satisfies the local independencies encoded in $\mathcal{G}$:

$$\{\forall i \ \mathbf{x}_i \perp \text{NonDescendants}_{\mathbf{x}_i} \mid \text{Parents}_{\mathbf{x}_i}\}$$

- We can order the variables in a topological order, such that each variable always comes after its parents;
- We can decompose the joint distribution using the chain rule of probability:

$$p(x_1, x_2, \dots, x_m) = \prod_{i=1}^m p(x_i \mid x_1, x_2, \dots, x_{i-1})$$

- Since we ordered the variables in a topological order, now $\text{Parents}_{x_i} \subseteq \{x_1, x_2, \dots, x_{i-1}\}$. We derive that:

$$p(x_i \mid x_1, x_2, \dots, x_{i-1}) = p(x_i \mid \text{Parents}_{x_i})$$

Combining the last two equations, we obtain the factorization property:

$$p(x_1, x_2, \dots, x_m) = \prod_{i=1}^m p(x_i \mid \text{Parents}_{x_i})$$

(Factorization $\Rightarrow$ I-Map)

- If p factorizes according to $\mathcal{G}$, then the joint distribution can be expressed as:

$$p(x_1, x_2, \dots, x_m) = \prod_{i=1}^m p(x_i \mid \text{Parents}_{x_i})$$

- Consider now the variable x_m . By the product rule ($p(X, Y) = p(Y \mid X)p(X)$), we can express the conditional distribution as:

$$p(x_m \mid x_1, x_2, \dots, x_{m-1}) = \frac{p(x_1, \dots, x_m)}{p(x_1, \dots, x_{m-1})}$$

- We can apply the sum rule to the denominator, marginalizing over x_m :

$$p(x_m \mid x_1, \dots, x_{m-1}) = \frac{p(x_1, \dots, x_m)}{\sum_{x_m} p(x_1, \dots, x_m)}$$

- Applying the factorization property, we obtain:

$$= \frac{\prod_{i=1}^m p(x_i \mid \text{Parents}_{x_i})}{\sum_{x_m} \prod_{i=1}^m p(x_i \mid \text{Parents}_{x_i})}$$

- We can rearrange the equation isolating the term for x_m :

$$= \frac{p(x_m \mid \text{Parents}_{x_m}) \prod_{i=1}^{m-1} p(x_i \mid \text{Parents}_{x_i})}{\sum_{x_m} p(x_m \mid \text{Parents}_{x_m}) \prod_{i=1}^{m-1} p(x_i \mid \text{Parents}_{x_i})}$$

We can subsequently simplify the equation, since $\sum_{x_m} p(x_m \mid \text{Parents}_{x_m}) = 1$, obtaining:

$$p(x_m \mid x_1, \dots, x_{m-1}) = p(x_m \mid \text{Parents}_{x_m})$$

□

6.2 Bayesian Network definition

A Bayesian Network (BN) is a pair $\mathcal{B} = (\mathcal{G}, \mathcal{P})$, where:

- $\mathcal{G}$ is a directed acyclic graph (DAG) representing the structure of the BN;
- $\mathcal{P}$ is a set of conditional probability distributions (CPDs) associated with the nodes in the graph.

P must factorize according to $\mathcal{G}$:

$$P(x_1, x_2, \dots, x_m) = \prod_{i=1}^m P(x_i \mid \text{Parents}_{x_i})$$

Example: Consider the following BN structure:

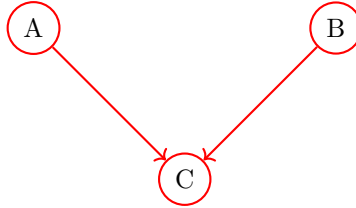


Figure 8: Example of Bayesian Network structure.

- Gene *A* and Gene *B* are independent;
- Gene *C* can be influenced by both Gene *A* and Gene *B*;

Gene	Value	P(value)
A	active	0.3
A	inactive	0.7

Table 4: CPD for Gene A.

Gene	Value	P(value)
B	active	0.3
B	inactive	0.7

Table 5: CPD for Gene B.

		A			
		active		inactive	
		B		B	
		active	inactive	active	inactive
C	active	0.9	0.6	0.7	0.1
	inactive	0.1	0.4	0.3	0.9

Table 6: Conditional Probability Table for Gene C.

6.3 D-separation

Before introducing the concept of D-separation, we need to better understand independence and conditional independence. We will then present the basic structures that can appear in a BN.

6.3.1 Independence and conditional independence

Independence

Two variables a, b are independent (written $a \perp b \mid \emptyset$) if:

$$P(a, b) = P(a)P(b)$$

Conditional independence

We say that two variables a, b are conditionally independent given a third variable c (written $a \perp b \mid c$) if:

$$P(a, b \mid c) = P(a \mid c)P(b \mid c)$$

Both independence and conditional independence can be verified by applying basic rules of probability. Instead, BNs allow one to verify these properties through the concept of D-separation.

6.3.2 Tail-to-tail connection

In a tail-to-tail connection, two variables a and b are connected through a common parent variable c . It is called tail-to-tail because the node c has two outgoing edges (tails) pointing to a and b .

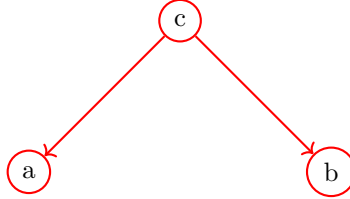


Figure 9: Tail-to-tail connection.

The joint distribution can be expressed as:

$$P(a, b, c) = P(a \mid c)P(b \mid c)P(c) \quad (6.2)$$

Given this, we want to understand the independency relationships between a and b :

- **Independent:** we need to check that $P(a, b) = P(a)P(b)$; To do this, we marginalize over c , applying the sum rule:

$$P(a, b) = \sum_c P(a, b, c)$$

We can then substitute the joint distribution from eq 6.2:

$$P(a, b) = \sum_c P(a \mid c)P(b \mid c)P(c) \neq P(a)P(b)$$

So a and b are **not independent**.

- **Conditionally independent:** we need to check that $P(a, b | c) = P(a | c)P(b | c)$; We can apply the product rule:

$$P(a, b | c) = \frac{P(a, b, c)}{P(c)}$$

We can then substitute the joint distribution from eq 6.2:

$$P(a, b | c) = \frac{P(a | c)P(b | c)P(c)}{P(c)} = P(a | c)P(b | c)$$

So a and b are **conditionally independent** given c .

We derive that, if c is observed, a and b become independent.

Example: A classic example of tail-to-tail connection is the relationship between COVID infection c , and two symptoms: fever a and cough b . If we observe the COVID infection status, knowing whether a patient has a fever provides no additional information about whether they have a cough, and vice versa.

Variable c is a common cause of both symptoms a and b .

6.3.3 Head-to-tail connection

In a head-to-tail connection, variable a is a parent of variable c , which is a parent of variable b . It is called head-to-tail because c has one incoming edge (head) from a and one outgoing edge (tail) to b .

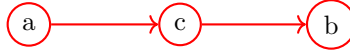


Figure 10: Head-to-tail connection.

The joint distribution can be expressed as:

$$p(a, b, c) = p(b | c)p(c | a)p(a)$$

Note that, using Bayes' theorem, we can rewrite $p(c | a)p(a)$ as $\frac{p(a|c)p(c)}{p(a)}$, obtaining:

$$p(a, b, c) = p(b | c)p(a | c)p(c)$$

This is exactly the same form as the tail-to-tail case 6.2. For completeness, we will repeat the independency analysis for clarity.

- **Independent:** we need to check that $P(a, b) = P(a)P(b)$;

$$P(a, b) = \sum_c P(a, b, c) = \sum_c p(b | c)p(c | a)p(a) \neq P(a)P(b)$$

So a and b are **not independent**.

- **Conditionally independent:** we need to check that $P(a, b | c) = P(a | c)P(b | c)$;

$$P(a, b | c) = \frac{P(a, b, c)}{P(c)} = \frac{p(b | c)p(c | a)p(a)}{P(c)} = P(a | c)P(b | c)$$

So a and b are **conditionally independent** given c .

Intuitively, if we already know the value of c , knowing a provides no additional information about b , and vice versa. Unlike the tail-to-tail case, here c is an intermediary variable between a and b .

Example: A classic example of head-to-tail connection is the relationship between cloud coverage a , rain c , and getting wet b . If we observe whether it is raining, knowing whether there is cloud coverage provides no additional information about whether we get wet. For both cloudy and clear skies, if it is raining we will get wet.

6.3.4 Head-to-head connection

In a head-to-head connection, variables a and b are both parents of variable c . It is called head-to-head because c has two incoming edges (heads) from a and b .

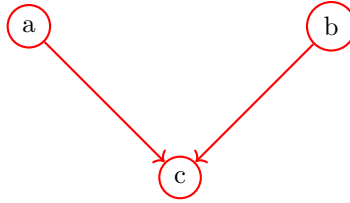


Figure 11: Head-to-head connection.

The joint distribution can be expressed as:

$$p(a, b, c) = p(c | a, b)p(a)p(b)$$

As before, we want to understand the independency relationships between a and b :

- **Independent:**

$$P(a, b) = \sum_c P(a, b, c) = \sum_c p(c | a, b)p(a)p(b)$$

Remember that summing over all possible values of c gives 1, so:

$$P(a, b) = p(a)p(b)$$

So a and b are **independent**.

- **Conditionally independent:**

$$P(a, b | c) = \frac{P(a, b, c)}{P(c)} = \frac{p(c | a, b)p(a)p(b)}{P(c)} \neq P(a | c)P(b | c)$$

So a and b are **not conditionally independent** given c .

Intuitively, a and b are independent. Observing c creates a dependency between a and b . This phenomenon is known as "explaining away": if we know that c has occurred (both a and b could have caused it), observing one of the causes makes the other cause less likely.

Example: A classic example of head-to-head connection is the relationship between burglary a , earthquake b , and alarm ringing c . Burglary and earthquake are independent events and are both causes of the alarm ringing. However, if we know that the alarm has rung, observing a burglary makes an earthquake less likely.

General head-to-head case A *descendant* of a node x is any node that can be reached by following directed edges starting from x . A head-to-head node c unblocks the dependency path between its parents a and b if either c or any of its descendants are observed.

This can be better understood with an example:

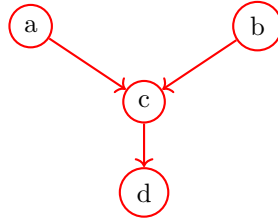


Figure 12: General head-to-head case.

In this case, if either c or d are observed, the dependency path between a and b is unblocked, making them dependent. For instance, in the previous example of burglary, earthquake, and alarm ringing, adding a new variable d representing a call from the alarm company. If we know either that the alarm has rung or that we received a call from the alarm company, observing a burglary makes an earthquake less likely and vice versa.

This rule is valid for any descendant of a head-to-head node, but the further down the descendant is in the graph, the weaker the dependency between the parents becomes.

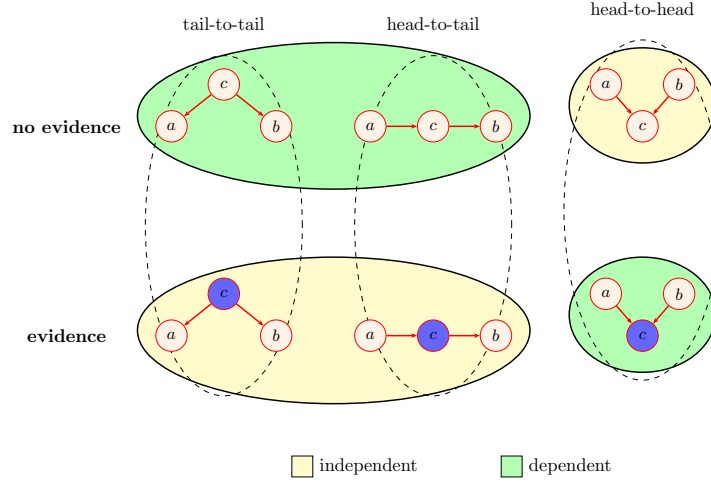


Figure 13: D-separation rules summary.

6.3.5 General D-separation criterion

D-separation

- Let $\mathcal{G}$ be a BN structure over a set of random variables $\mathbf{X}$;
- Given three disjoint subsets of variables $\mathbf{A}, \mathbf{B}, \mathbf{C} \subseteq \mathbf{X}$;
- We say that $\mathbf{A}$ and $\mathbf{B}$ are **D-separated** given $\mathbf{C}$ in $\mathcal{G}$ if all paths from any node in $\mathbf{A}$ to any node in $\mathbf{B}$ are blocked (no information flow possible).
- This can also be written as:

$$\text{D-sep}(\mathbf{A}, \mathbf{B} \mid \mathbf{C})$$

A path is considered blocked if it contains at least one node that meets one of the following conditions:

- The arrows on the path meet either in a tail-to-tail or head-to-tail configuration, and the node is in $\mathbf{C}$ (observed);
- The arrows on the path meet in a head-to-head configuration, and neither the node nor any of its descendants are in $\mathbf{C}$ (not observed).

In other words, a path is blocked if the nodes $\mathbf{A}$ and $\mathbf{B}$ cannot influence each other given the observed variables in $\mathbf{C}$ ($\mathbf{A}$ and $\mathbf{B}$ are independent given $\mathbf{C}$).

If A and B are D-separated given C , then they are conditionally independent given C .

Examples: Consider the following BN structure:

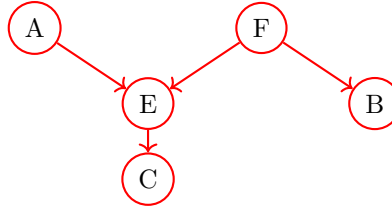


Figure 14: Example of Bayesian Network structure for D-separation.

Consider the two following examples:

1. Are A and B D-separated given C ?
 - Node F is in a tail-to-tail connection and is not observed. This implies that the path is not blocked;
 - Node E is in a head-to-head connection and is not observed. One of its children (C) is observed, meaning that they are dependent (the path is not blocked);
 - Therefore, there exists an unblocked path from A to B , so they are **not D-separated** given C .
2. Are A and B D-separated given F ?
 - Node F is in a tail-to-tail connection and is observed. This implies that the path is blocked;
 - Therefore, all paths from A to B are blocked, so they are **D-separated** given F .

6.4 Bayesian Network Independencies

A Bayesian Network structure $\mathcal{G}$ encodes both a set of local independencies $\mathcal{I}_l(\mathcal{G})$ (see eq 6.1) and a set of global independencies $\mathcal{I}(\mathcal{G})$. Global independencies are all the independencies that can be derived using the D-separation criterion. We can formally define them as:

Global independencies

$$\mathcal{I}(\mathcal{G}) = \{(A \perp B \mid C) : \text{D-sep}(A, B \mid C)\}$$

6.5 Equivalence Classes

Different BN structures can represent the same set of independencies assumptions.

I-Equivalence

We say that two BN structures $\mathcal{G}, \mathcal{G}'$ are **I-equivalent** if they encode the same set of independencies:

$$\mathcal{I}(\mathcal{G}) = \mathcal{I}(\mathcal{G}')$$

As we have previously seen, for a structure $\mathcal{G}$ to be an I-map for a distribution p it is not necessary that $\mathcal{I}(\mathcal{G}) = \mathcal{I}(p)$ (does not need to encode all independencies in p). As a consequence, a fully connected graph (no independencies) is an I-map for any distribution p over the same set of variables. Such a graph is obviously not very useful, as it does not provide any information about independencies in p .

To overcome this limitation, we can define a **minimal I-map**.

Minimal I-Map

A BN structure $\mathcal{G}$ is a **minimal I-map** for a distribution p if:

- $\mathcal{G}$ is an I-map for p ;
- Removing any edge from $\mathcal{G}$ would result in a graph that is not an I-map for p .

This means it is impossible to remove any edge from $\mathcal{G}$ without introducing independencies that do not hold in p . Note that a minimal I-map does not necessarily encode all independencies in p . For this, we need a stronger definition, namely the **perfect I-map**.

Perfect I-Map

We say that a BN structure $\mathcal{G}$ is a **perfect I-map** for a distribution p if it captures all and only the independencies that hold in p :

$$\mathcal{I}(\mathcal{G}) = \mathcal{I}(p)$$

Unfortunately, not all distributions p admit a perfect I-map, some cannot be perfectly represented by any BN structure.

6.6 Building Bayesian Networks

Here is a list of practical guidelines to build a BN structure:

- Get together with domain experts. This allows to gather prior knowledge about the relationships between variables;
- Define relevant variables for both observable entities and quantities you wish to predict. Also, including latent variables can help to better represent the dependencies in the data;
- When adding edges, follow causal relationships when possible. For example, if variable A causes variable B , add a directed edge from A to B ;
- Avoid assigning zero probabilities to conditional probability distributions.
- If data is available, we can use structure learning algorithms to learn the BN structure from data.

7 Learning Bayesian Networks

Learning Bayesian Networks (BNs) from data involves two main tasks: parameter learning and structure learning. In this section, we will discuss both tasks in detail.

7.1 Parameter Learning

For parameter learning, we assume that the structure G of the Bayesian Network is given. We are given a dataset $D = \{\mathbf{x}(1), \mathbf{x}(2), \dots, \mathbf{x}(N)\}$, where each $\mathbf{x}_i$ is a configuration of all (**complete data**) or some (**incomplete data**) of the variables in the network.

The goal is to estimate the parameters θ of the model (one parameter per variable) from the data. Again, the simplest approach consists in finding the parameters that maximize the likelihood of the data (MLE):

$$\theta^* = \arg \max_{\theta} P(D|\theta) = \arg \max_{\theta} \mathcal{L}(D, \theta)$$

7.1.1 Complete Data

Given the complete data D , since every datapoint is *i.i.d.* (independent and identically distributed), we can express the likelihood of the data as:

$$P(D|\theta) = \prod_{i=1}^N P(\mathbf{x}(i)|\theta) \quad (7.1)$$

Instead of working with the entire structure G for each data point, we can exploit the factorization property of Bayesian Networks. This allows us to express $p(\mathbf{x}(i)|\theta)$ as:

$$P(D|\theta) = \prod_{i=1}^N \prod_{j=1}^m P(\mathbf{x}_j(i)|\text{Pa}_j(i), \theta) \quad (7.2)$$

Where:

- $\mathbf{x}_j(i)$ is the value of variable X_j in the i -th data point;
- $\text{Pa}_j(i)$ is the set of values of the parent variables of X_j in the i -th data point.

Graphically, we can represent the likelihood as follows:

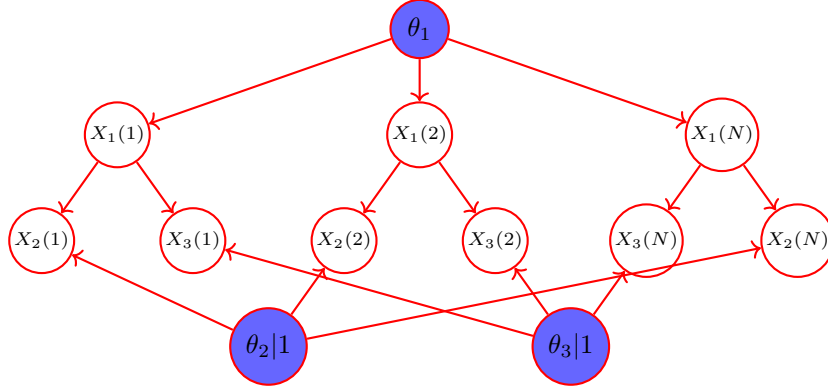


Figure 15: Complete data representation

The probability $p(\mathbf{x}_j(i)|pa_j(i), \theta)$ only depends on the parameters $\theta_{X_j|Pa_j}$, not on the parameters of other variables. Because the Conditional Probability Distributions (CPDs) are independent, we can rearrange the likelihood as:

$$P(D|\theta) = \prod_{j=1}^m \prod_{i=1}^N P(x_j(i)|pa_j(i), \theta_{X_j|Pa_j}) \quad (7.3)$$

This allows us to decompose the MLE problem into m independent subproblem, one for each variable X_j :

$$\theta_{X_j|Pa_j}^{\max} = \arg \max_{\theta_{X_j|Pa_j}} \underbrace{\prod_{i=1}^N P(x_j(i)|pa_j(i), \theta_{X_j|Pa_j})}_{\mathcal{L}(\theta_{X_j|Pa_j}, \mathcal{D})} \quad (7.4)$$

When working with discrete variables, we can estimate the parameters using frequency counts from the data. A discrete CPD $P(X|U)$ can be represented as a table:

- Each row corresponds to a specific configuration of X ;
- Each column corresponds to a specific configuration of the parent variables U ;
- Each table entry $\theta_{x|u}$ represents the probability of the configuration $X = x$ and $U = u$.

From equation 7.4, we can replace $P(x_j(i)|pa_j(i))$ with the corresponding parameter $\theta_{x(i)|u(i)}$. The likelihood for a single CPD X becomes:

$$\mathcal{L}(\theta_{X|U}, D) = \prod_{i=1}^N \theta_{x(i)|u(i)}$$

$$= \prod_{u \in \text{Val}(U)} \left[\prod_{x \in \text{Val}(X)} \theta_{x|u}^{N_{x,u}} \right]$$

Where $N_{x,u}$ is the count of occurrences in the data where $X = x$ and $U = u$. Now, each column in the CPD contains a multinomial distribution over the values of X , given a specific configuration of its parents $U = u$. This implies that each column must sum to 1:

$$\sum_{x \in \text{Val}(X)} \theta_{x|u} = 1$$

Parameters of different columns are independent, allowing us to estimate them independently. Thus, we can decompose the MLE problem for each column u as follows:

$$\theta_{x|u}^{\max} = \frac{N_{u,x}}{\sum_x N_{u,x}}$$

This means that the MLE estimate for each parameter $\theta_{x|u}$ is simply the relative frequency of $X = x$ given $U = u$ in the data.

7.1.2 Adding priors

The maximum likelihood estimation can lead to overfitting on the training data, especially when the dataset is small. Also, configuration of variables that do not appear in the training data will have zero probability. A common solution to these problems is to introduce a prior distribution over the parameters θ , based on prior knowledge or assumptions. This way, we now want to maximize the posterior distribution of the parameters given the data:

$$\theta^* = \arg \max_{\theta} P(\theta|D)$$

Applying Bayes' theorem, we can rewrite this as:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)}$$

Since $P(D)$ is constant with respect to θ , we can simplify the optimization problem to:

$$\theta^* = \arg \max_{\theta} P(D|\theta)P(\theta) \quad (7.5)$$

This approach is known as Maximum A Posteriori (MAP) estimation.

The conjugate prior for the multinomial distribution is the Dirichlet distribution, with parameters $\alpha_{x|u}$, for each possible value of x . The resulting MAP estimate for each parameter $\theta_{x|u}$ becomes:

$$*\theta_{x|u}^{\max} = \frac{N_{u,x} + \alpha_{x|u}}{\sum_x (N_{u,x} + \alpha_{x|u})} \quad (7.6)$$

Where $\alpha_{x|u}$ can be interpreted as imaginary samples with configuration $X_j = x$ and $Pa_j = u$.

7.1.3 Incomplete Data

With incomplete data, some examples may have missing values for certain variables. In such cases, counts $N_{x|u}$ cannot be directly computed from the data. To address this, we can use an approximate method called the Expectation-Maximization (EM) algorithm.

EM algorithm In a nutshell, as we have said before, sufficient statistics (like counts) are not directly computable from incomplete data. The EM algorithm iteratively estimates these statistics based on the current parameter estimates, and then updates the parameters using these estimated statistics.

The algorithm consists of two main steps:

- **Expectation step (E-step):** compute the expected sufficient statistics for the complete data, with respect to the joint distribution for $\mathbf{X}$ conditioned on the current value of θ (current estimates of the parameters) and the known data $\mathcal{D}$:

$$E[N_{ijk}] = \sum_{l=1}^n P(X_i(l) = x_k, Pa_i(l) = pa_j | \mathbf{X}_l, \theta) \quad (7.7)$$

Where:

- N_{ijk} is the count of occurrences where variable X_i takes value x_k and its parents Pa_i take configuration pa_j ;
- $\mathbf{X}_l$ is the l -th data point in the dataset $\mathcal{D}$ (observed values);
- if $X_i(l)$ and $Pa_i(l)$ are observed for X_i in the dataset $\mathcal{D}$, then the probability is either 0 or 1;
- Otherwise, we run Bayesian inference to compute the probability.
- **Maximization step (M-step):** compute the parameters maximizing likelihood of the complete dataset D_c , using the expected counts from the E-step:

$$\theta_{ijk}^{\max} = \frac{E[N_{ijk}]}{\sum_k E[N_{ijk}]} \quad (7.8)$$

This is the same formula as in the complete data case, but using expected counts instead of actual counts.

As previously seen in the complete data case, also in this case we can use a MAP estimation approach by introducing a prior distribution over the parameters. This is not different from the complete data case, for the only exception being the use of expected counts instead of actual counts.

7.2 Structure Learning

There are different approaches to learn the structure of a Bayesian Network from data. Some of this approaches include:

-
- **Constraint-based methods:** these methods use statistical tests to identify conditional independencies between variables. Based on these independencies, a graph structure is proposed.
 - **Score-based methods:** these methods define a scoring for each possible structure based on how well it fits the data. It then searches the space of possible structures to find the one that maximizes the score.
 - **Model-averaging methods:** these methods consider multiple possible structures and average their predictions, weighted by their probabilities.

8 Naive Bayes

Naive Bayes is a simple classification algorithm based on Bayesian networks.

Setting

- Each instance x is represented by a set of attributes values $\langle a_1, a_2, \dots, a_m \rangle$;
- The target feature y is the class label we want to predict. It can take any value from a finite set $\mathcal{Y}$;
- The task is predicting the Maximum A Posteriori (MAP) estimate for the class label y given the attribute values. In other words, we want to predict the class $\hat{y}$ that maximizes the posterior probability $P(y|a_1, a_2, \dots, a_m)$. More formally:

$$\hat{y} = \arg \max_{y_i \in \mathcal{Y}} P(y_i | a_1, a_2, \dots, a_m) \quad (8.1)$$

Using Bayes' theorem, we can rewrite this as:

$$\hat{y} = \arg \max_{y_i \in \mathcal{Y}} \frac{P(a_1, a_2, \dots, a_m | y_i) P(y_i)}{P(a_1, a_2, \dots, a_m)} \quad (8.2)$$

Since the denominator is constant with respect to y_i , we can simplify this to:

$$\hat{y} = \arg \max_{y_i \in \mathcal{Y}} P(a_1, a_2, \dots, a_m | y_i) P(y_i) \quad (8.3)$$

8.1 Naive Bayes assumption

As the number of attributes increases, estimating the joint probability $P(a_1, a_2, \dots, a_m | y_i)$ becomes increasingly difficult. To simplify this, Naive Bayes makes the assumption that all attributes are independent of each other, given the class label y_i . We can simplify the MAP estimate to:

$$p(a_1, \dots, a_m | y_i) = \prod_{j=1}^m P(a_j | y_i) \quad (8.4)$$

This heavily reduces the number of parameters we need to estimate, making the model more efficient and less prone to overfitting.

8.2 Naive Bayes classifier

Given the assumption of conditional independence, the MAP estimate for the class label becomes:

$$\hat{y} = \arg \max_{y_i \in \mathcal{Y}} \prod_{j=1}^m P(a_j | y_i) P(y_i) \quad (8.5)$$

We assume that each value of attribute A_j is modeled using the same type of distribution. The probability of an attribute value given the class label $P(a_j | y_i)$

can for example be modeled using a multinomial distribution over the k possible values of attribute A_j :

$$P(a_j|y_i) = \prod_{k=1}^K \theta_{ky_i}^{z_k(a_j)} \quad (8.6)$$

Where:

- θ_{ky_i} is the parameter representing the probability of attribute a_j taking value k , given class label y_i ;
- $z_k(a_j)$ is an indicator function that is 1 if attribute a_j takes value k , and 0 otherwise.

8.2.1 Parameter estimation

We need to estimate both the class prior probabilities $P(y_i)$ and the conditional probabilities $P(a_j|y_i)$ from the training data.

- The class prior probabilities $p(y_i)$ can be easily learned as the fraction of training instances having each class label;
- The maximum likelihood estimate for the parameters θ_{ky_i} can be computed as the relative frequency of attribute A_j taking value k among all instances with class label y_i . More formally:

$$\theta_{ky_i}^{\max} = \frac{N_{ky_i}}{N_{y_i}} \quad (8.7)$$

As previously mentioned, we can also introduce Dirichlet priors over the parameters to avoid zero probabilities for unseen attribute values:

$$\theta_{ky_i}^{\max} = \frac{N_{ky_i} + \alpha_{ky_i}}{N_{y_i} + \alpha_c} \quad (8.8)$$

8.3 Example - Text Classification

We want to classify documents into one of C possible classes, based only the words they contain. Let V be the vocabulary of all possible words and D a dataset of labeled documents.

- We can easily compute the prior probabilities for each class y_i as:

$$P(y_i) = \frac{|D_i|}{|D|}$$

- We can model attributes with a multinomial distribution over $K = |V|$ possible words.

- The parameters θ_{ky_i} can be estimated as:

$$\theta_{ky_i}^{\max} = \frac{\sum_{x \in D_i} \sum_{j=1}^{|x|} z_k(x_j)}{\sum_{x \in D_i} N_{y_i}(x)}$$

Where:

- x is a document in the dataset;
- D_i is the subset of documents in class y_i ;
- x_j is the j -th word in document x ;
- $\sum_{x \in D_i} \sum_{j=1}^{|x|} z_k(x_j)$ is the total number of occurrences of word k in all documents of class y_i ;
- $\sum_{x \in D_i} N_{y_i}(x)$ is the total number of words in all documents of class y_i .

We can then classify a new document by computing the posterior probability for each class and selecting the class with the highest probability.

$$*\hat{y} = \arg \max_{y_i \in \mathcal{Y}} \prod_{j=1}^{|x|} P(x_j|y_i)P(y_i) \quad (8.9)$$

Substituting the multinomial model for $P(x_j|y_i)$ and the prior $P(y_i)$, we get:

$$\hat{y} = \arg \max_{y_i \in \mathcal{Y}} \prod_{j=1}^{|x|} \prod_{k=1}^K \theta_{ky_i}^{z_k(x_j)} \cdot \frac{|D_i|}{|D|} \quad (8.10)$$

9 Linear Discriminant Functions

Before deep diving into specific learning algorithms, it is important to understand the difference between **generative models** and **discriminative models**.

- **Generative models** aim to model the **distribution** governing the data. They learn the joint probability distribution $P(x, y)$ and can generate new data points, according to the learned distribution. Some examples of generative models are **Naive Bayes**, **Gaussian Mixture Models**.
- **Discriminative models** aim to model the **discriminative function** directly, rather than the underlying data distribution. For example, for classification tasks, they directly model the **decision boundary** that separates different classes.

9.1 Advantages and disadvantages of discriminative models

Pros:

- When data is complex, modelling the underlying distribution can be very challenging (E.g., high resolution images);
- If **data discrimination** is the final goal, then there is no need to model the entire data distribution;
- Discriminative models can focus on learning the parameters that are most relevant for learning the decision boundary, potentially leading to better performance.

Cons:

- The learned model is less flexible in its usage;
- It doesn't allow to perform arbitrary inference tasks on the data. In example, it is not possible to generate new data points from the learned model.

9.2 Definition

A **linear discriminant model** uses a **linear function** to make predictions. The **decision boundary** (discriminant function) is a linear combination of the input features:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 \quad (9.1)$$

where:

- $\mathbf{x}$ is the input feature vector;

- $\mathbf{w}$ is the weight vector, representing the importance of each feature;
- w_0 is the bias term, which allows to shift the decision boundary.

Multiplying $\mathbf{w}^T$ by $\mathbf{x}$ computes a weighted sum of the input features, and adding w_0 shifts the result.

The linear discriminant function is the simplest form of discriminant function. Depending on the specific problem and data, more complex forms of discriminant functions can be used, but linear functions should always be considered as a baseline.

9.3 Linear Binary Classifier

In a **binary classification** problem, the linear discriminant function can be used to classify data points into two classes. Since the basic output of the discriminant function is a continuous value, we need to define a threshold to convert it into a binary prediction. This is usually done by taking the sign of the discriminant function:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0) = \begin{cases} +1 & \text{if } \mathbf{w}^T \mathbf{x} + w_0 \geq 0 \\ -1 & \text{if } \mathbf{w}^T \mathbf{x} + w_0 < 0 \end{cases} \quad (9.2)$$

The **decision boundary**, which is a **hyperplane** H in the feature space, is defined by the equation:

$$f(\mathbf{x}) = 0 \quad (9.3)$$

The weight vector $\mathbf{w}$ is **orthogonal** to the decision boundary H .

Proof. We take two points on the decision boundary, $\mathbf{x}_1$ and $\mathbf{x}_2$, such that:

$$f(\mathbf{x}_1) = f(\mathbf{x}_2) = 0$$

Based on equation (9.1), we can rewrite this as:

$$\mathbf{w}^T \mathbf{x}_1 + w_0 = \mathbf{w}^T \mathbf{x}_2 + w_0 = 0$$

$$\mathbf{w}^T \mathbf{x}_1 - \mathbf{w}^T \mathbf{x}_2 = 0$$

$$\mathbf{w}^T (\mathbf{x}_1 - \mathbf{x}_2) = 0$$

This shows that the vector $\mathbf{w}^T$ is orthogonal to the vector $(\mathbf{x}_1 - \mathbf{x}_2)$, which lies on the decision boundary. $\square$

Functional margin

The value $f(\mathbf{x})$ of the discriminative function for a certain point $\mathbf{x}$ is called **functional margin**. It can be seen as the **confidence** of the model in the prediction for that point. A larger absolute value of the functional margin indicates a higher confidence in the prediction.

The functional margin is not normalized, meaning that its value depends on the scale of the weight vector $\mathbf{w}$. For example, if we multiply $\mathbf{w}$ and w_0 by a constant factor $\alpha > 1$, the functional margin will also increase by the same factor, even though the decision boundary remains unchanged. Without normalization, we could arbitrarily increase the functional margin by scaling the weights, which does not reflect a true increase in confidence.

To address this, we introduce the concept of **geometric margin**, which normalizes the functional margin by the norm of the weight vector.

Geometric margin

The distance from a point $\mathbf{x}$ to the decision boundary H is called **geometric margin**.

$$r^x = \frac{f(\mathbf{x})}{\|\mathbf{w}\|} \quad (9.4)$$

The distance from the origin to the decision boundary is given by:

$$r_0 = \frac{|w_0|}{\|\mathbf{w}\|} \quad (9.5)$$

Proof. A generic point $\mathbf{x}$ can be expressed by its **projection** on H plus its **distance** from H times the vector in that direction:

$$\mathbf{x} = \mathbf{x}^P + r^x \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

where:

- $\mathbf{x}^P$ is the projection of $\mathbf{x}$ on H ;
- r^x is the distance from $\mathbf{x}$ to H ;
- $\frac{\mathbf{w}}{\|\mathbf{w}\|}$ is the unit vector in the direction of $\mathbf{w}$.

Substituting this into the discriminant function:

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{w}^T \left(\mathbf{x}^P + r^x \frac{\mathbf{w}}{\|\mathbf{w}\|} \right) + w_0 \\ &= \mathbf{w}^T \mathbf{x}^P + w_0 + r^x \frac{\mathbf{w}^T \mathbf{w}}{\|\mathbf{w}\|} \end{aligned}$$

We can notice that $\mathbf{w}^T \mathbf{x}^P + w_0 = 0$ since $\mathbf{x}^P$ lies on the decision boundary H . Therefore:

$$f(\mathbf{x}) = r^x \|\mathbf{w}\|$$

Rearranging this gives us the formula for the geometric margin as in equation (9.4). $\square$

9.4 Multiclass Classification

In a **multiclass classification** problem with K classes, we can extend the linear binary classifier by using **one-vs-all** (OvA) or **one-vs-one** (OvO) strategies.

9.4.1 One-vs-All (OvA)

The idea is to train K binary classifiers, one for each class. Each classifier $f_k(\mathbf{x})$ is trained to distinguish class k from all other classes. This way, we have a model with K **hyperplanes**, each separating one class from the rest. At prediction time, we evaluate all K classifiers and assign the input $\mathbf{x}$ to the class with the maximum **functional margin**.

9.4.2 One-vs-One (OvO)

The idea is to train a binary classifier for every pair of classes. For K classes, we need to train $\frac{K(K-1)}{2}$ classifiers. The model predicts a new example $\mathbf{x}$ in the class winning the most pairwise comparisons, as in a tournament.

9.4.3 One-vs-One (OvO) vs One-vs-All (OvA)

- OvA requires to train K classifiers, each on the entire dataset;
- OvO requires to train $\frac{K(K-1)}{2}$ classifiers, each on a smaller subset of the data (only the samples belonging to the two classes being compared);

If the number of examples is high, OvO can be more efficient, since each classifier is trained on a smaller dataset.

10 Perceptron

Perceptrons are one of the simplest types of artificial neural networks, based on the concept of linear classifiers (9.2). They are based on the idea of simulating the behavior of a biological neuron.

10.1 Biological Motivation

The human brain is composed of densely interconnected neurons. They process information based on the following principles:

- Electrochemical reactions allow signals to propagate through the neuron;
- Synapses connect neurons together, they can either **excite** or **inhibit** the receiving neuron;
- When the incoming signals exceed a certain threshold, the neuron **fires** and transmits the signal to the next neuron.

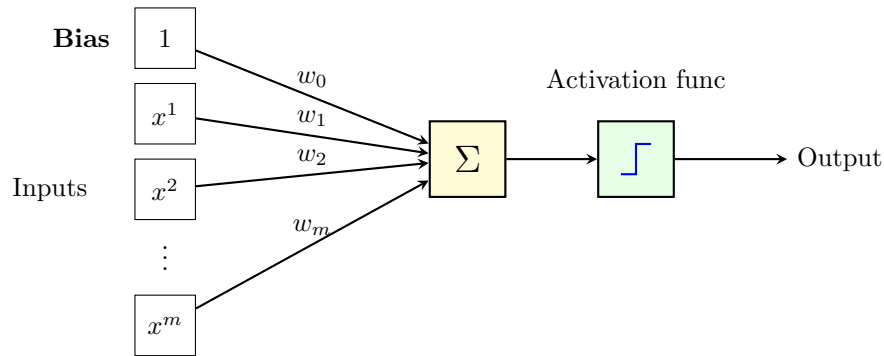
Artificial neural networks aim to mimic this behavior using mathematical models. The threshold is modeled using an **activation function**, while the synapses are represented by **weights**.

10.2 Single Neuron Architecture

A single neuron perceptron is a specific learning model that implements a linear binary classifier. It can be represented as:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0) \quad (10.1)$$

The function $f(\mathbf{x})$ is simply a **linear combination** of the input features $\mathbf{x}$, with weights $\mathbf{w}$ and an activation function (sign function).



10.2.1 Augmented feature / weight vector

To simplify the notation, we can introduce an **augmented feature vector**. We can rewrite the equation (10.1) as:

$$f(\mathbf{x}) = \text{sign}(\hat{\mathbf{w}}^T \hat{\mathbf{x}}) \quad (10.2)$$

where the **bias term** is included in the weight vector and the feature vector as follows:

$$\hat{\mathbf{w}} = \begin{bmatrix} w_0 \\ \mathbf{W} \end{bmatrix}, \quad \hat{\mathbf{x}} = \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix}$$

In the following sections, we will skip the hat notation for simplicity.

10.2.2 Representational power

The single neuron perceptron can only represent **linearly separable** functions. Examples of this are the **primitive boolean functions** such as **AND**, **OR**, and **NOT**. This implies that any logic formula can be represented by a network of a 2 level perceptron in **disjunctive normal form** (DNF) or **conjunctive normal form** (CNF). For example, the DNF formula:

$$(A \wedge B) \vee (C \wedge D)$$

The first layer of the network computes the conjunctions $A \wedge B$ and $C \wedge D$ using two perceptrons, while the second layer computes the disjunction of their outputs using a third perceptron.

However, this representation require an **exponential** number of perceptrons in the worst case.

10.3 Parameter learning

Similar to what we saw for **maximum likelihood estimation**, we need to find a **function** of the parameters to be optimized.

In this case, a reasonable choice is to minimize the **measure of error** on the training set $\mathcal{D}$. This is called **loss function** ℓ and it compares the predicted output $f(\mathbf{x}_i)$ with the ground truth y_i .

We can define **training error** E as:

$$E(\mathbf{w}, D) = \sum_{(\mathbf{x}, y) \in \mathcal{D}} \ell(f(\mathbf{x}), y) \quad (10.3)$$

This is simply the sum of the losses over all training examples.

10.3.1 Gradient descent

This is an optimization problem where we want to find the weights $\mathbf{w}$ that minimize the error function E . There is no closed-form solution for this problem,

so we need to use an iterative optimization algorithm.

The common approach is to minimize the error function using a **gradient descent** approach:

1. Initialize the weights $\mathbf{w}$ randomly or with zeros.
2. Iterate until gradient is approximately zero, updating the weights as:

$$\mathbf{w}' = \mathbf{w} - \eta \nabla E(\mathbf{w}, \mathcal{D}) \quad (10.4)$$

where η is the learning rate and controls the amount of movement at each step.

Since the gradient $\nabla E(\mathbf{w}; D)$ points in the direction of **maximum increase** of the error function, we multiply it by -1 to move in the direction of **maximum decrease**. This approach is guaranteed to converge to a local optimum of the error function $E(\mathbf{w}; D)$ for small enough learning rates η .

- too **low** learning rate: slow convergence;
- too **high** learning rate: the algorithm may **diverge** (oscillate around the minimum).

There are techniques to adaptively change the learning rate during training, allowing for faster convergence.

Perceptron training rule

In order to apply a **gradient descent** approach, we need to define a **loss function** which is **differentiable** (smooth). For example, the misclassification error defined in (10.3) is not differentiable, as it is piecewise constant.

As a consequence, a different training rule is adopted. It's defined as follows:

$$E(\mathbf{w}; \mathcal{D}) = \sum_{(\mathbf{x}, y) \in \mathcal{D}_E} -yf(\mathbf{x}) \quad (10.5)$$

where $\mathcal{D}_E$ is the set of **misclassified** samples in the training set $\mathcal{D}$, for which it holds that:

$$yf(\mathbf{x}) \leq 0$$

The error is the **sum** of the **functional margins** (confidences) of the misclassified samples.

Applying gradient descent to the error function in (10.5), we obtain:

$$\nabla E(\mathbf{w}; \mathcal{D}) = \nabla \sum_{(\mathbf{x}, y) \in \mathcal{D}_E} -yf(\mathbf{x})$$

We can substitute $f(\mathbf{x})$ with the linear function $\mathbf{w}^T \mathbf{x}$:

$$= \nabla \sum_{(\mathbf{x}, y) \in \mathcal{D}_E} -y(\mathbf{w}^T \mathbf{x})$$

The derivative with respect to $\mathbf{w}$ is:

$$= \sum_{(\mathbf{x}, y) \in \mathcal{D}_E} -y\mathbf{x}$$

Thus, the amount of change in the weights is:

$$-\eta \nabla E(\mathbf{w}; \mathcal{D}) = \eta \sum_{(\mathbf{x}, y) \in \mathcal{D}_E} y\mathbf{x}$$

10.3.2 Stochastic Gradient Descent

In practice, the **batch gradient descent** approach we discussed so far is rarely used, since it requires to compute the gradient over the entire training set $\mathcal{D}$ at each iteration.

A more common approach is to use **stochastic gradient descent** (SGD). It works as follows:

1. Initialize the weights $\mathbf{w}$ randomly;
2. Iterate until all examples are correctly classified:
 - (a) For each incorrectly classified example $(\mathbf{x}, y)$ in the training set $\mathcal{D}$, update the weights as:

$$\mathbf{w} = \mathbf{w} + \eta y \mathbf{x} \quad (10.6)$$

With this approach, the weights are updated after each misclassified example, rather than after processing the entire training set. Each gradient step is **very fast** and can sometimes avoid local minima.

10.4 Perceptron Regression

Linear models can also be used for **regression** tasks, where the goal is to predict a continuous output variable y given an input feature vector $\mathbf{x}$. The problem can be modeled as:

- Let $X \in \mathbb{R}^{n \times d}$ be the input training matrix (i.e. $X = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^T$), where each row is a d -dimensional feature vector and $n = |\mathcal{D}|$;
- Let $\mathbf{y} \in \mathbb{R}^n$ be the output training Matrix (i.e. $\mathbf{y}_i$ is the target value for the i -th training example);
- **Regression learning** can be stated as a set of **linear equations**:

$$f(\mathbf{x}) = y \implies X\mathbf{w} = \mathbf{y}$$

Giving as a solution:

$$\mathbf{w} = X^{-1}\mathbf{y} \quad (10.7)$$

However this approach has a few problems:

- The matrix X is usually not square, so the inverse X^{-1} is not defined;
- System of equations is overdetermined (more equations than unknowns)
 $\implies$ no exact solution exists.

10.4.1 Error function

We can again resort to loss minimization to find an approximate solution. Our goal is not to solve the system exactly, but to find the weights $\mathbf{w}$ that minimize the **mean squared error** (MSE) on the training set:

$$E(\mathbf{w}; \mathcal{D}) = \sum_{(\mathbf{x}, y) \in \mathcal{D}} (y - f(\mathbf{x}))^2$$

This can be rewritten in matrix form as:

$$E(\mathbf{w}; \mathcal{D}) = (\mathbf{y} - X\mathbf{w})^T (\mathbf{y} - X\mathbf{w}) \quad (10.8)$$

This is a **convex** function, so it has a single global minimum with a closed-form solution. **Gradient descent** can still be faster than computing the closed-form solution, especially for large datasets.

10.4.2 Closed form solution

To find the weights $\mathbf{w}$ that minimize the mean squared error, we can set the gradient of the error function (10.8) to zero and solve for $\mathbf{w}$:

$$\begin{aligned} \nabla E(\mathbf{w}; \mathcal{D}) &= \nabla (\mathbf{y} - X\mathbf{w})^T (\mathbf{y} - X\mathbf{w}) \\ 2(\mathbf{y} - X\mathbf{w})^T (-X) &= 0 \\ -2\mathbf{y}^T X + 2\mathbf{w}^T X^T X &= 0 \end{aligned}$$

Rearranging the terms and dividing by 2, we get:

$$\mathbf{w}^T X^T X = \mathbf{y}^T X$$

Taking the transpose of both sides:

$$X^T X \mathbf{w} = X^T \mathbf{y}$$

Assuming that $X^T X$ is invertible, we can solve for $\mathbf{w}$:

$$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y} \quad (10.9)$$

NOTE:

- $(X^T X)^{-1} X^T$ is a pseudoinverse called the **left inverse** of X ;
- If X is square, then the left inverse is equal to the regular inverse X^{-1} , and this corresponds to the exact solution of the system of equations;
- The left inverse exists only if the columns of X are linearly independent. This means that each feature is independent from all the others. If this is not the case, we can use remove redundant features using techniques such as **Principal Component Analysis** (PCA), discussed in the Appendix A

10.4.3 Gradient descent

In case of large datasets, computing the closed-form solution can be computationally expensive. Also, we are not always interested in finding the exact minimum of the error function, but rather a good enough approximation. In such cases, we can use **gradient descent** to iteratively update the weights until convergence. Again, given the gradient of the error function, we can compute the update of the weights as:

$$\mathbf{w}' = \mathbf{w} - \eta \nabla E(\mathbf{w}; \mathcal{D})$$

To compute the gradient, we start from the error function defined in (10.8):

$$\nabla E(\mathbf{w}; \mathcal{D}) = \nabla \sum_{(\mathbf{x}, y) \in \mathcal{D}} (y - f(\mathbf{x}))^2$$

We can rewrite the error function in matrix form:

$$\begin{aligned} &= \nabla (\mathbf{y} - X\mathbf{w})^2 \\ &= \nabla (\mathbf{y} - X\mathbf{w})^T (\mathbf{y} - X\mathbf{w}) \\ &= 2(\mathbf{y} - X\mathbf{w})^T (-X) \end{aligned}$$

11 Support Vector Machines

Support Vector Machines (SVMs) are a very popular method for linear (and non-linear) classification and regression tasks. They are based on the following key concepts:

- **Large Margin Classifier:** SVMs aim to find the hyperplane that maximizes the margin between different classes in the feature space. This way each prediction is made with high confidence;
- **Support Vectors:** only a subset of data points influence the position of the decision boundary. These points are called **support vectors**, and we will see how they are determined during the training process;
- **Sound Theoretical Foundation:** the **large margins** help to improve generalization performance;
- **Kernel machines:** SVMs can be extended to non-linear decision boundaries using kernel functions.

11.1 Maximum Margin Classifier

SVMs learn the **central hyperplane** that separates the classes with the **largest margin**. Intuitively, the margin is the distance between the hyperplane and the closest data points from each class (the **support vectors**). To formally define this concept, we first need to introduce the notion of **classifier margin**.

Classifier Margin

Given a training set $\mathcal{D}$, a classifier confidence margin is:

$$\rho = \min_{(\mathbf{x}, y) \in \mathcal{D}} yf(\mathbf{x}) \quad (11.1)$$

This is the minimal confidence of the classifier ($yf(\mathbf{x})$) in a correct prediction. It has to be positive for all training points to be correctly classified. As previously seen with the perceptron, the geometric margin is the same value normalized by the norm of the weight vector:

$$\frac{\rho}{\|\mathbf{w}\|} = \min_{(\mathbf{x}, y) \in \mathcal{D}} \frac{yf(\mathbf{x})}{\|\mathbf{w}\|}. \quad (11.2)$$

Intuitively, we want to learn a classifier that learns correct predictions with high confidence (large margin). To do this we first need to remove the ambiguity in the definition of the hyperplane. In fact, there are infinitely many equivalent formulations for the same hyperplane:

$$\mathbf{w}^T \mathbf{x} + w_0 = 0$$

$$\alpha(\mathbf{w}^T \mathbf{x} + w_0) = 0 \quad \forall \alpha \neq 0$$

To remove this ambiguity, we can define the **canonical hyperplane** by imposing the constraint that the closest points to the hyperplane satisfy:

Canonical Hyperplane

The canonical hyperplane is defined such that the functional margin of the closest points to the hyperplane is equal to 1:

$$\rho = \min_{(\mathbf{X}, y) \in \mathcal{D}} yf(\mathbf{x}) = 1$$

Its geometric margin then becomes:

$$\frac{\rho}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

This way, the points closest to the hyperplane (support vectors) will always distance $\frac{1}{\|\mathbf{w}\|}$ from the hyperplane.

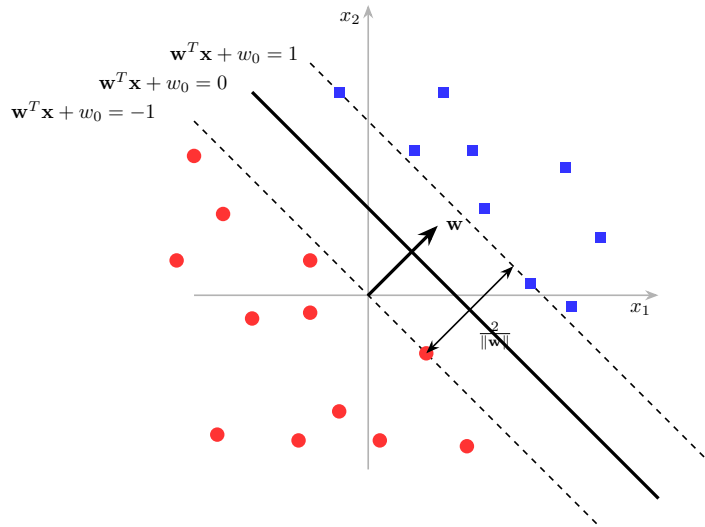


Figure 16: Example of a Maximum Margin Classifier

11.2 Hard Margin SVM

Hard Margin SVMs assume that the training data is linearly separable and that we can find a hyperplane that classifies **all training points correctly**. Let's see how we can formulate this as a learning problem.

11.2.1 Learning problem

The learning objective of SVMs is to find the canonical hyperplane that maximizes the margin, which is equivalent to minimizing $\|\mathbf{w}\|$ since the margin is inversely proportional to it. This leads to the following optimization problem:

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 \quad (11.3)$$

subject to the constraints:

$$y_i(\mathbf{w}^T \mathbf{x}_i + w_0) \geq 1, \quad \forall (\mathbf{x}_i, y_i) \in \mathcal{D}$$

Note that:

- The objective function $\frac{1}{2} \|\mathbf{w}\|^2$ is used instead of $\|\mathbf{w}\|$ simply for mathematical convenience. The squared function is easier to differentiate and optimize and does not change the location of the minimum;
- The constraint guarantees that all points are correctly classified with a margin of at least 1.
- This is a convex **quadratic optimization**, with linear constraints and can be solved efficiently using Lagrange multipliers.

Let's first see how to handle this kind of constrained optimization problems in general.

11.2.2 Karush-Kuhn-Tucker (KKT) Approach

A constrained optimization problem can be addressed by converting it into an unconstrained one with the same solution.

Let's take in example the following general optimization problem:

$$\min_z f(z)$$

subject to the constraint:

$$g_i(z) \geq 0, \quad \forall i$$

We can introduce a non-negative Lagrange multiplier α_i for each constraint $g_i(z)$, and define the **Lagrangian** function:

$$L(z, \boldsymbol{\alpha}) = f(z) - \sum_i \alpha_i g_i(z)$$

The solution of the original constrained problem can be found by solving the following **saddle point** problem:

$$\min_z \max_{\boldsymbol{\alpha} \geq 0} L(z, \boldsymbol{\alpha})$$

The optimal solutions z^* for this problem are the same as the one from the **original constrained problem**. This formulation works because:

- If, for a given $\hat{z}$, a constraint $g_i(\hat{z})$ is **not satisfied** (i.e., $g_i(\hat{z}) < 0$), maximizing over α_i will lead to an **infinite value**. In fact, we are subtracting a negative value multiplied by an unbounded positive value, which can grow indefinitely;
- If all constraints are satisfied, maximizing over α will set all α_i to zero. The sum will be zero, and we will simply minimize $f(z)$, without any constraints.

11.2.3 Lagrangian of the SVM problem

Applying this to the SVM optimization problem (11.3), we can define the relative Lagrangian:

$$L(\mathbf{w}, w_0, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1) \quad (11.4)$$

where:

- $\alpha_i \geq 0$ are the Lagrange multipliers associated to each constraint;
- $m = |\mathcal{D}|$ is the number of training examples. We have one constraint per training example.

The lagrangian is minimized with respect to $\mathbf{w}$ and w_0 , and maximized with respect to the non-negative multipliers α_i .

We now try to solve the **Lagrangian equivalent form**. To do this, we first compute the partial derivatives of L with respect to $\mathbf{w}$ and w_0 , and set them to zero.

Starting with $\mathbf{w}$:

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} \left(\frac{\mathbf{w}^T \mathbf{w}}{2} \right) - \frac{\partial}{\partial \mathbf{w}} \sum_{i=1}^m \alpha_i y_i \mathbf{w}^T \mathbf{x}_i \\ &= \mathbf{w} - \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \end{aligned}$$

Setting this to zero gives:

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \quad (11.5)$$

Similarly, for w_0 :

$$\frac{\partial L}{\partial w_0} = - \sum_{i=1}^m \alpha_i y_i$$

Setting this to zero gives:

$$\sum_{i=1}^m \alpha_i y_i = 0 \quad (11.6)$$

We can now substitute these expressions back into the Lagrangian (11.4) to eliminate the parameters $\mathbf{w}$ and w_0 . This gives us the **dual formulation** of the SVM optimization problem:

$$\frac{1}{2} \sum_{i,j=1}^m \alpha_i y_i \mathbf{x}_i^T \alpha_j y_j \mathbf{x}_j - \sum_{i,j=1}^m \alpha_i y_i \mathbf{x}_i^T \alpha_j y_j \mathbf{x}_j - \underbrace{\sum_{i=1}^m \alpha_i y_i w_0}_{=0} + \sum_{i=1}^m \alpha_i =$$

Applying the simplifications, our objective becomes:

$$L(\alpha) = \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i y_i \mathbf{x}_i^T \alpha_j y_j \mathbf{x}_j \quad (11.7)$$

We did get rid of the primal parameters $\mathbf{w}$ and w_0 , and now we have to maximize $L(\alpha)$ with respect to the multipliers α_i .

Our new optimization problem is:

$$\begin{aligned} \max_{\alpha \in \mathbb{R}^m} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i y_i \mathbf{x}_i^T \alpha_j y_j \mathbf{x}_j \\ \text{subject to} \quad & \begin{cases} \sum_{i=1}^m \alpha_i y_i = 0 \\ \alpha_i \geq 0, \quad \forall i \end{cases} \end{aligned} \quad (11.8)$$

This is a **dual formulation** of the original SVM optimization problem, since it is expressed in terms of the Lagrange multipliers α_i instead of the primal parameters $\mathbf{w}$ and w_0 . It is also a convex quadratic optimization problem, but the constraints are now much simpler than in the primal formulation. It's important to note that this formulation has m variables (the α_i), equal to the number of training examples, while the primal formulation has $d + 1$ variables (size of the weight vector + bias term w_0). One could choose to solve either the primal or the dual formulation depending on which one is more efficient for the given problem.

We can now rewrite the **decision function** of the SVM in terms of the Lagrange multipliers:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + w_0 \quad (11.9)$$

11.2.4 Observations

At this point, we can make some important observations. At the **saddle point** each **constraint** must be equal to zero: This can happen in two cases:

1. If $\alpha_i = 0$, then the corresponding data point does not contribute to the decision function;
2. If $\alpha_i > 0 \implies y_i(\mathbf{w}^T \mathbf{x}_i + w_0) - 1 = 0 \implies y_i(\mathbf{w}^T \mathbf{x}_i + w_0) = 1$, meaning that the data point lies exactly on the margin boundary.

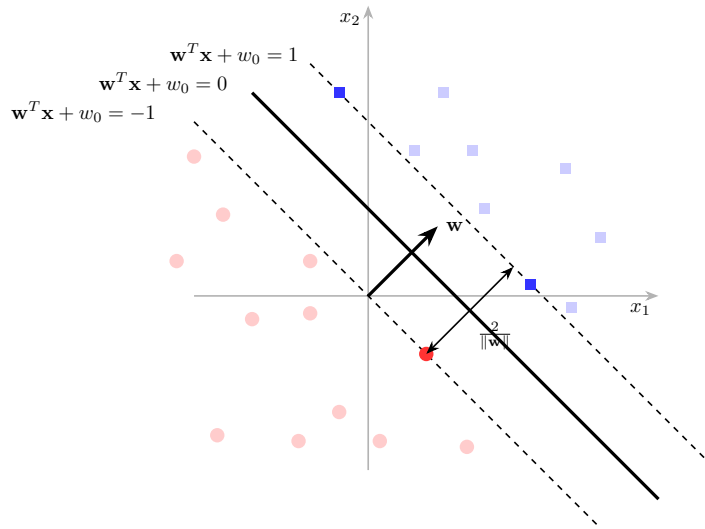


Figure 17: Highlighting Support Vectors in hard Margin SVM

The points that satisfy the second condition are called **support vectors**, as they are the only points that influence the position of the decision boundary. All the other points have $\alpha_i = 0$ and do not affect the decision function.

The bias term w_0 can be computed using any of the support vectors, using the condition that they lie on the margin boundary. This is usually done by averaging the values obtained from all support vectors to improve numerical stability.

11.3 Soft Margin SVM

In practice, there is one big problem with the assumption made by hard margin SVMs: **assuming that all data can be correctly classified** is often unrealistic and can lead to overfitting.

To address this issue, we introduce the concept of **soft margin SVMs**, which allow for some misclassifications in the training data. This is done by introducing **slack variables** to the optimization problem.

Slack variables

Slack variables $\xi_i \geq 0$ are introduced for each training point to allow for some misclassification.

They represent the **penalty** for the example i being on the wrong side of the margin.

The **sum of the slack variables** $\sum_{i=1}^m \xi_i$ has to be minimized along with the norm of the weight vector to find a balance between maximizing the margin and minimizing the misclassification error.

11.3.1 Learning problem

In the soft margin SVM, we still want to maximize the margin (minimize the norm of $\mathbf{w}$), but we also want to penalize the misclassifications. This can be done by adding **slack variables** to the constraints, leading to the following optimization problem:

$$\begin{aligned} \min_{\mathbf{w} \in X, w_0 \in \mathbb{R}, \boldsymbol{\xi} \in \mathbb{R}^m} & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \\ \text{subject to} & \begin{cases} y_i(\mathbf{w}^T \mathbf{x}_i + w_0) \geq 1 - \xi_i, & \forall i \\ \xi_i \geq 0, & \forall i \end{cases} \end{aligned} \quad (11.10)$$

where $C > 0$ is a regularization parameter that controls the **trade-off** between **maximizing the margin** and **minimizing the misclassification error**. If we set C to a very large value (e.g., $C \rightarrow \infty$), we recover the hard margin SVM formulation, as misclassifications are heavily penalized.

11.3.2 Regularization theory

The problem in (11.10) can be interpreted from a regularization perspective. The objective function can be seen as a combination of two terms:

- The first term, $\frac{\|\mathbf{w}\|^2}{2}$, encourages the model to have a small norm, which corresponds to a large margin;
- The second term, $C \sum_{i=1}^m \xi_i$, penalizes the misclassifications, with C controlling the strength of this penalty.

This can be seen as a **Regularized loss minimization**. We can substitute the slack variables ξ_i with a suitable loss function $L(y_i, f(\mathbf{x}_i))$ that measures the error made by the classifier on each training example, removing the explicit constraints on ξ_i .

$$\min_{\mathbf{w} \in X, w_0 \in \mathbb{R}, \boldsymbol{\xi} \in \mathbb{R}^m} \frac{\|\mathbf{w}\|^2}{2} + C \sum_{i=1}^m L(y_i, f(\mathbf{x}_i)) \quad (11.11)$$

We need to define a suitable loss function $L(y_i, f(\mathbf{x}_i))$ that captures the misclassification error. Considering the constraints in (11.10), we can define the **hinge loss**.

Hinge Loss

The hinge loss is defined as:

$$\begin{aligned} L(y_i, f(\mathbf{x}_i)) &= |1 - y_i f(\mathbf{x}_i)|_+ \\ &= |1 - y_i(\mathbf{w}^T \mathbf{x}_i + w_0)|_+ \\ &= \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + w_0)) \end{aligned} \quad (11.12)$$

The hinge loss penalizes predictions that are either incorrect or within the margin.

- For correct predictions outside the margin ($y_i f(\mathbf{x}_i) > 1$), the loss is zero.
- The more the prediction deviates from the margin ($0 < y_i f(\mathbf{x}_i) \leq 1$), the higher the loss.

This is exactly equivalent to the slack variable formulation.

11.3.3 Lagrangian of the Soft Margin SVM

We can now derive the Lagrangian for the soft margin SVM optimization problem (11.10). The Lagrangian is defined as:

$$L = \underbrace{\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i}_{f(x)} - \underbrace{\sum_{i=1}^m \alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + w_0) - 1 + \xi_i)}_{g_1(x)} - \underbrace{\sum_{i=1}^m \beta_i \xi_i}_{g_2(x)} \quad (11.13)$$

In this case the primal variables are $\mathbf{w}$, w_0 and $\boldsymbol{\xi}$, while the dual variables are the Lagrange multipliers $\alpha_i \geq 0$ and $\beta_i \geq 0$.

We can now compute the partial derivatives of L with respect to the primal variables and set them to zero.

- Starting with $\mathbf{w}$:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i$$

Setting this to zero, we get the same expression as in the hard margin case:

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i$$

- For w_0 :

$$\frac{\partial L}{\partial w_0} = - \sum_{i=1}^m \alpha_i y_i$$

Setting this to zero, we again get:

$$\sum_{i=1}^m \alpha_i y_i = 0$$

- Finally, for ξ_i :

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \beta_i$$

Setting this to zero, we get:

$$\alpha_i + \beta_i = C$$

Substituting these expressions back into the Lagrangian (11.13), we obtain the dual formulation of the soft margin SVM optimization problem:

$$\begin{aligned} \max_{\alpha \in \mathbb{R}^m} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i y_i \mathbf{x}_i^T \alpha_j y_j \mathbf{x}_j \\ \text{subject to} \quad & \begin{cases} \sum_{i=1}^m \alpha_i y_i = 0 \\ 0 \leq \alpha_i \leq C, \quad \forall i \end{cases} \end{aligned} \quad (11.14)$$

The only difference with respect to the hard margin case is the additional upper bound constraint on the Lagrange multipliers α_i . These are called **box constraints** and ensure that the multipliers do not exceed the regularization parameter C . This derives from the fact that $\alpha_i = C - \beta_i$ and $\beta_i \geq 0$. Since we are removing from C a non-negative value, α_i cannot be greater than C .

11.3.4 Observations

Starting from equation 11.13, at the saddle point, it must hold that:

$$\begin{cases} \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + w_0) - 1 + \xi_i) = 0 & \forall i \\ \beta_i \xi_i = 0 & \forall i \end{cases} \quad (11.15)$$

Thus, **support vectors** ($\alpha_i > 0$) are examples for which:

$$y_i (\mathbf{w}^T \mathbf{x}_i + w_0) \leq 1 \quad (11.16)$$

We can distinguish some cases:

- If $0 < \alpha_i < C$, then $C - \alpha_i - \beta_i = 0$ and $\beta_i \xi_i = 0$ imply that $\xi_i = 0$ and the point lies exactly on the margin boundary: $y_i (\mathbf{w}^T \mathbf{x}_i + w_0) = 1$;
- If $\alpha_i = C$, then $\xi_i > 0$ and the point is either inside the margin or misclassified: $y_i (\mathbf{w}^T \mathbf{x}_i + w_0) < 1$;
- If $\alpha_i = 0$, then $\xi_i = 0$ and the point is correctly classified and outside the margin. It doesn't count

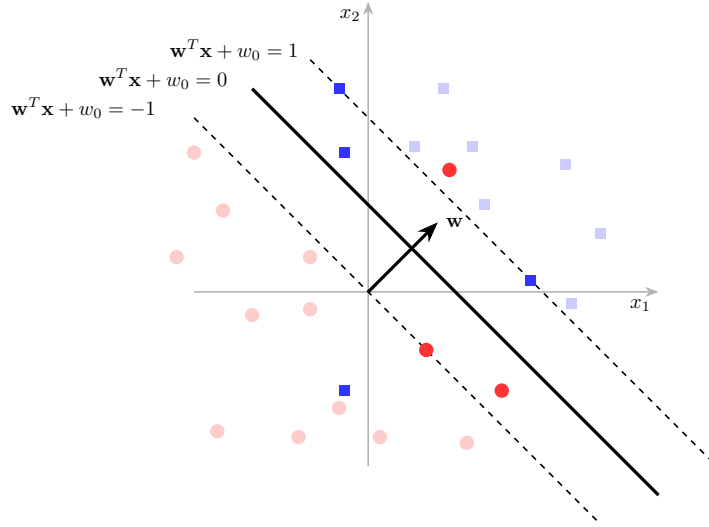


Figure 18: Highlighting Support Vectors in Soft Margin SVM

11.4 Large Scale SVM

When considering large datasets, the standard SVM training algorithms can become computationally expensive due to the quadratic programming involved. To address this, several large-scale SVM training methods have been developed, such as **stochastic gradient descent (SGD)**.

When using SGD, we can reformulate the SVM optimization problem as:

$$\min_{\mathbf{w} \in X} \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{m} \sum_{i=1}^m |1 - y_i \langle \mathbf{w}, \mathbf{x}_i \rangle|_+ \quad (11.17)$$

Using SGD, we can iteratively update the weight vector $\mathbf{w}$ based on the gradient of the loss function computed on a single training example:

$$E(\mathbf{w}; \mathbf{x}_i, y_i) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + |1 - y_i \langle \mathbf{w}, \mathbf{x}_i \rangle|_+ \quad (11.18)$$

There is a problem with this approach: the hinge loss is not differentiable at the point where $1 - y_i \langle \mathbf{w}, \mathbf{x}_i \rangle = 0$. To handle this, we can use the concept of **subgradients**, which generalizes the notion of gradients to non-differentiable functions. The gradients then become:

$$\nabla_{\mathbf{w}} E(\mathbf{w}; \mathbf{x}_i, y_i) = \lambda \mathbf{w} - \mathbb{I}[y_i \langle \mathbf{w}, \mathbf{x}_i \rangle < 1] y_i \mathbf{x}_i \quad (11.19)$$

where $\mathbb{I}[\cdot]$ is the indicator function which can be written as:

$$\mathbb{I}[y_i \langle \mathbf{w}, \mathbf{x}_i \rangle < 1] = \begin{cases} 1 & \text{if } y_i \langle \mathbf{w}, \mathbf{x}_i \rangle < 1 \\ 0 & \text{otherwise} \end{cases}$$

11.5 Margin Error Bound

The performance of SVMs can be theoretically analyzed using the concept of **margin error bounds**. The probability of misclassification on unseen data is bound from above by:

$$v + \sqrt{\frac{c}{m} \left(\frac{R^2 \wedge^2}{\rho^2} \ln^2 m + \ln\left(\frac{1}{\delta}\right) \right)} \quad (11.20)$$

Analyzing the equation, we can see that the **probability of error** depends on:

- **number of margin errors** v on the training set (example inside the margin or misclassified);
- **number of training examples** m (error depends on $\sqrt{\frac{\ln^2 m}{m}}$);
- **size of the margin** ρ (error depends on $\frac{1}{\rho^2}$);

12 Kernel Machines

With what we have seen so far about SVMs, we find out that:

- **Hard Margin SVMs** can address linearly separable data only, with no misclassification;
- **Soft Margin SVMs** can address linearly separable problems with some degree of misclassification;
- **Non-linearly separable problems** need higher expressive power to be addressed.

Kernel Machines are an extension of SVMs that can address non-linearly separable problems, maintaining the same underlying principles of SVMs (large margin and theoretical guarantees).

The key idea is to *map* the input data into a higher dimensional feature space, where it is more likely to be linearly separable and perform a linear separation in that space.

12.1 Feature Map

Feature Map

$$\phi : \mathcal{X} \rightarrow \mathcal{H}$$

A feature map ϕ is a function that maps the input space $\mathcal{X}$ to a higher dimensional (possibly infinite-dimensional) feature space $\mathcal{H}$.

All the examples $\mathbf{x}_i$ are replaced by their images $\phi(\mathbf{x}_i)$ in the feature space. This should increase the expressive power of the representation, introducing features that are combinations of the original inputs. In this new space, the data may become linearly separable.

An example of feature map is the following:

Polynomial Feature Map

- **Homogeneous Polynomial Feature Map** of degree 2:

$$\phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ x_1 x_2 \\ x_2^2 \end{pmatrix} \quad (12.1)$$

This feature map takes a 2D input vector $\mathbf{x} = (x_1, x_2)$ and maps it to a 3D feature space by including only polynomial terms of degree 2 (Homogeneous).

• **Non-Homogeneous Polynomial Feature Map** of degree 2:

$$\phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1 \\ x_2 \\ x_1^2 \\ x_1x_2 \\ x_2^2 \end{pmatrix} \quad (12.2)$$

This feature map takes a 2D input vector $\mathbf{x} = (x_1, x_2)$ and maps it into all possible polynomial terms up to degree 2 (Non-Homogeneous).

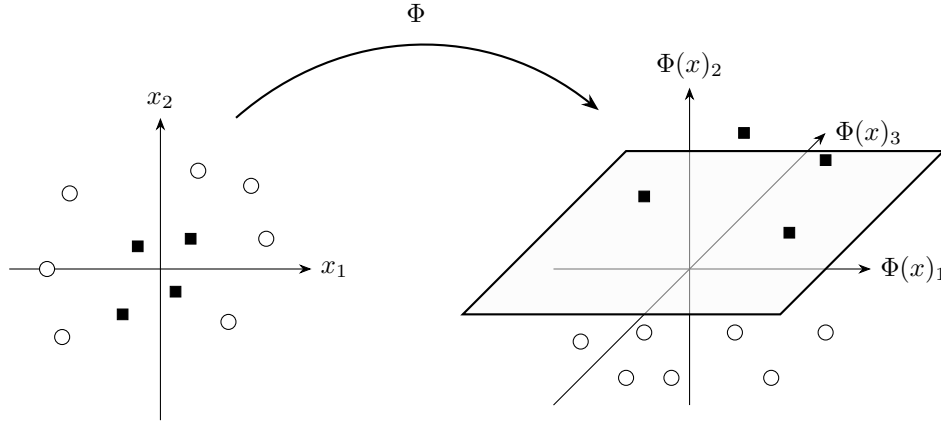


Figure 19: Mapping from 2D input space to 3D feature space using a polynomial feature map of degree 2.

12.2 Linear separation in feature space

Once the data is mapped into the feature space using the feature map ϕ , we can apply SVM algorithms, replacing $\mathbf{x}$ with $\phi(\mathbf{x})$:

$$f(x) = \mathbf{w}^T \phi(\mathbf{x}) + w_0 \quad (12.3)$$

A linear separation in the feature space corresponds to a non-linear separation in the original input space.

E.g. using the homogeneous polynomial feature map of degree 2, the decision function becomes:

$$\phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \text{sgn}(w_1x_1^2 + w_2x_1x_2 + w_3x_2^2 + w_0)$$

The linear separator in the 3D feature space corresponds to a ellipse in the original 2D input space.

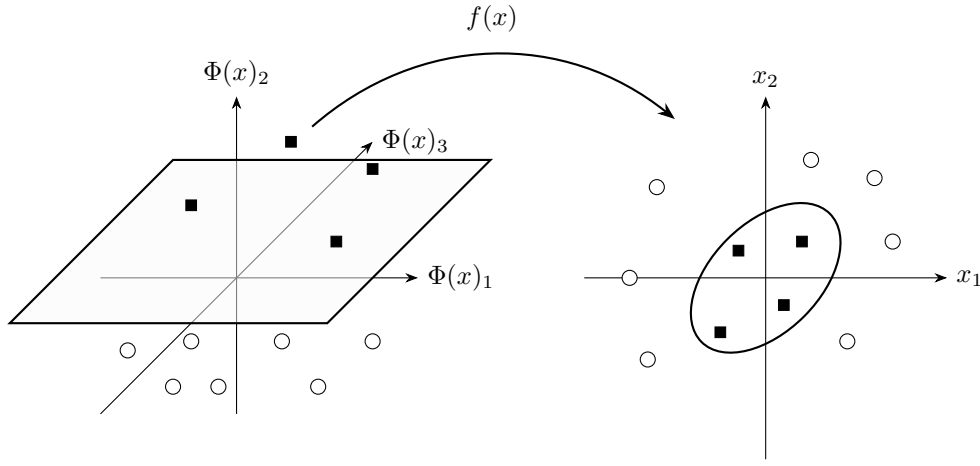


Figure 20: Mapping from 2D input space to 3D feature space using a polynomial feature map of degree 2.

12.3 Kernel trick

Computing the feature map $\phi(\mathbf{x})$ explicitly can be computationally expensive. However, this is not necessary to compute the SVM decision function. In the **dual formulation** of SVMs (equations 11.8, 11.14), the data points appear only in the form of **dot products** $\langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$.

Kernel trick

The **kernel trick** consists in replacing the dot product in the feature space with an equivalent **kernel function**

$$\langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle = \phi(\mathbf{x})^T \phi(\mathbf{x}') = k(\mathbf{x}, \mathbf{x}') \quad (12.4)$$

The **kernel functions** uses examples in the **original input space** $\mathcal{X}$ to compute the dot product in the **feature space** $\mathcal{H}$, without explicitly computing the feature map ϕ .

12.3.1 Examples of kernels

Homogeneous Polynomial Kernel : taking in consideration the equation 12.1, we can derive the following kernel:

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}')^d \quad (12.5)$$

Setting $d = 2$, we have:

$$\begin{aligned} k\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \begin{pmatrix} x'_1 \\ x'_2 \end{pmatrix}\right) &= (x_1x'_1 + x_2x'_2)^2 \\ &= (x_1x'_1)^2 + 2x_1x_2x'_1x'_2 + (x_2x'_2)^2 \\ &= \underbrace{(x_1^2, \sqrt{2}x_1x_2, x_2^2)}_{\phi(\mathbf{x})^T} \underbrace{\begin{pmatrix} x_1'^2 \\ \sqrt{2}x'_1x'_2 \\ x_2'^2 \end{pmatrix}}_{\phi(\mathbf{x}')^T} \end{aligned}$$

This is equivalent to the dot product in the feature space defined by the homogeneous polynomial feature map of degree 2, up to a scaling factor $\sqrt{2}$.

Non-Homogeneous Polynomial Kernel : taking in consideration the equation 12.2, we can derive the following kernel:

$$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + 1)^d \quad (12.6)$$

Setting $d = 2$, we have:

$$\begin{aligned} k\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \begin{pmatrix} x'_1 \\ x'_2 \end{pmatrix}\right) &= (x_1x'_1 + x_2x'_2 + 1)^2 \\ &= (x_1x'_1)^2 + 2x_1x'_1x_2x'_2 + (x_2x'_2)^2 + 2x_1x'_1 + 2x_2x'_2 + 1 \\ &= \underbrace{(1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2)}_{\phi(\mathbf{x})^T} \underbrace{\begin{pmatrix} 1 \\ \sqrt{2}x'_1 \\ \sqrt{2}x'_2 \\ x_1'^2 \\ \sqrt{2}x'_1x'_2 \\ x_2'^2 \end{pmatrix}}_{\phi(\mathbf{x}')^T} \end{aligned}$$

This is equivalent to the dot product in the feature space defined by the non-homogeneous polynomial feature map of degree 2, up to a scaling factor $\sqrt{2}$.

12.4 Valid Kernels

Valid Kernel

A kernel function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is valid if it corresponds to a dot product in some feature space $\mathcal{H}$:

$$k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle \quad (12.7)$$

The kernel generalizes the concept of similarity between data points in arbitrary feature spaces.

Gram Matrix

Given examples $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$, and a kernel function k , the **Gram matrix** K is defined as:

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) \forall i, j \quad (12.8)$$

It can be interpreted as a matrix of pairwise similarities between the examples in the dataset, according to the kernel function k .

Positive definite matrix

A symmetric matrix $K \in \mathbb{R}^{m \times m}$ is positive definite if:

$$\sum_{i,j=1}^m c_i c_j K_{ij} \geq 0 \quad \forall \mathbf{c} \in \mathbb{R}^m \quad (12.9)$$

Equivalent definitions of positive definiteness include:

- All eigenvalues of K are non-negative;
- There exists a matrix A such that $K = A^T A$.

A sufficient and necessary condition for a kernel k to be valid is that the corresponding Gram matrix is positive definite for any choice of examples $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$. There are several ways to check if a kernel is valid, e.g.:

- Prove its positive definiteness;
- Find out a corresponding feature map ϕ (as we did before);
- Use kernel combination properties (e.g. sum, product yield valid kernels).
This is the most common way to build complex kernels from simpler ones.

12.4.1 Basic Kernels

As just mentioned, basic kernels can be useful as building blocks for more complex kernels using kernel combination properties. Some common examples include:

- **Linear Kernel:**

$$k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}' \quad (12.10)$$

Apply no feature map, the kernel corresponds to the standard dot product in the input space.

- **Polynomial Kernel:**

$$k_{d,c}(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^T \mathbf{x}' + c)^d \quad (12.11)$$

The kernel depends on two parameters: the degree d and the constant c . We have already seen this kernel in paragraph 12.3.1.

- **Gaussian Kernel:**

$$k_\sigma(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right) = \exp\left(-\frac{\mathbf{x}^T \mathbf{x} + \mathbf{x}'^T \mathbf{x}' - 2\mathbf{x}^T \mathbf{x}'}{2\sigma^2}\right) \quad (12.12)$$

The kernel depends on a parameter σ that controls the width of the Gaussian function. The smaller the σ , the more localized the influence of each training example.

The Gaussian kernel is an example of **universal kernel**, which means that it can approximate any continuous function on a compact domain given enough data.

12.4.2 Kernel Combinations

Basic kernels can be **combined** using various operations to create more complex kernels from simpler ones. Correctly using combination operators guarantees that the resulting kernel is still positive definite.

We will now analyze some common kernel combination methods.

Kernel Sum

The sum of two kernels corresponds to the concatenation of their feature spaces:

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}') \quad (12.13)$$

Applying the definition of kernel:

$$\begin{aligned} &= \phi_1(\mathbf{x})^T \phi_1(\mathbf{x}') + \phi_2(\mathbf{x})^T \phi_2(\mathbf{x}') \\ &= (\phi_1(\mathbf{x}) + \phi_2(\mathbf{x}))^T (\phi_1(\mathbf{x}') + \phi_2(\mathbf{x}')) \end{aligned}$$

Kernel Product

The product of two kernels corresponds to the **cartesian product** of their features:

$$\begin{aligned} (k_1 \times k_2)(\mathbf{x}, \mathbf{x}') &= k_1(\mathbf{x}, \mathbf{x}') k_2(\mathbf{x}, \mathbf{x}') \quad (12.14) \\ &= \sum_{i=1}^n \phi_{1i}(\mathbf{x}) \phi_{1i}(\mathbf{x}') \sum_{j=1}^m \phi_{2j}(\mathbf{x}) \phi_{2j}(\mathbf{x}') \\ &= \sum_{i=1}^n \sum_{j=1}^m (\phi_{1i}(\mathbf{x}) \phi_{2j}(\mathbf{x})) (\phi_{1i}(\mathbf{x}') \phi_{2j}(\mathbf{x}')) \end{aligned}$$

$$= \sum_{k=1}^{nm} \phi_{12k}(\mathbf{x}) \phi_{12k}(\mathbf{x}') = \phi_{12}(\mathbf{x})^T \phi_{12}(\mathbf{x}')$$

where $\phi_{12}(\mathbf{x}) = \phi_1(\mathbf{x}) \times \phi_2(\mathbf{x})$ is the cartesian product of the two feature maps. The product can also be between kernel in different spaces.

Linear combination of kernels

A kernel can be rescaled by a positive constant:

$$k_{\beta}(\mathbf{x}, \mathbf{x}') = \beta k(\mathbf{x}, \mathbf{x}')$$

We can define a linear combination of kernels as:

$$k_{sum}(\mathbf{x}, \mathbf{x}') = \sum_{k=1}^K \beta_k k_k(\mathbf{x}, \mathbf{x}')$$

Since understanding what kernel works best for a given problem can be challenging, combining multiple kernels allows us to leverage the strengths of each individual kernel. With **kernel learning** the weights β_k of the combination can be learned from data, allowing the model to adaptively combine multiple kernels.

Kernel Normalization

Kernel values can be influenced by the dimension of objects. I.E., in text classification, longer documents tend to have higher dot products simply because they contain more words.

To mitigate this effect, we can normalize the kernel:

$$\hat{k}(\mathbf{x}, \mathbf{x}') = \frac{k(\mathbf{x}, \mathbf{x}')}{\sqrt{k(\mathbf{x}, \mathbf{x})k(\mathbf{x}', \mathbf{x}')}} \quad (12.15)$$

This is a **cosine normalization**, which takes into account the angle between the feature vectors rather than their magnitudes.

12.5 Kernel on Graphs

Kernels on graphs heavily rely on the Weisfeiler-Lehman (WL) test for graph isomorphism.

12.5.1 Graph Isomorphism

To understand this algorithm, we need to define the concept of **graph isomorphism**.

Graph Isomorphism

Two graphs $G = (V, E)$ and $G' = (V', E')$ are isomorphic if there exists a bijection $f : V \rightarrow V'$ such that:

$$(u, v) \in E \iff (f(u), f(v)) \in E' \quad (12.16)$$

In a nutshell, two graphs are isomorphic if they have the same structure, but possibly different node labels.

12.5.2 Weisfeiler-Lehman Test

If isomorphism is easy to check on vectorial data, it is much more challenging on graph data. This is in fact an NP-hard problem.

The WL test is an approximate algorithm that can efficiently determine if two graphs are non-isomorphic, but it may fail to distinguish some non-isomorphic graphs (false positives).

Algorithm: We are now going to present the WL algorithm on two graphs with same node labels:

- Given two graphs $G = (V, E)$ and $G' = (V', E')$ with $|V| = |V'| = n$;
- Let $L(G) = l(v)|v \in V$ be the set of node labels in G . $l(v)$ is a function returning the label of node v ;
- Let $L(G') = L(G)$;
- Let $label(s)$ be a function assigning a unique label to a string.

The algorithm is defined as follows:

Algorithm 3 Weisfeiler-Lehman Test

```

Set  $l_0(v) \leftarrow l(v), \forall v \in V$ 
for  $i = 1$  to  $n - 1$  do
  for each node  $v \in G$  do
    Let  $M_i(v) = \{l_{i-1}(u) | u \in \text{neigh}(v)\}$ 
    Concatenate the sorted labels of  $M_i(v)$  into  $s_i(v)$ 
    Let  $l_i(v) = label(l_{i-1}(v) \oplus s_i(v))$ 
  end for
  If  $L_i(G) \neq L_i(G')$ , return not isomorphic
end for
Return possibly isomorphic

```

Where:

- $M_i(v)$: the multiset of labels of the neighbors of node v at iteration i ;

- $l_i(v)$: the new label assigned to node v at iteration i . It is computed by assigning a unique label to the concatenation of the previous label $l_{i-1}(v)$ and the sorted labels of its neighbors $s_i(v)$.

12.5.3 Weisfeiler-Lehman Kernel

- Let $G_1, G_2, \dots, G_h = (V, E, l_0), (V, E, l_1), \dots, (V, E, l_h)$ be a set of resulting graphs after applying h iterations of the WL algorithm on graph G .
- Let $k : G \times G \rightarrow \mathbb{R}$ be any kernel on graphs;

The **Weisfeiler-Lehman kernel** between two graphs G and G' is defined as:

$$K_{WL}^h(G, G') = \sum_{i=0}^h k(G_i, G'_i) \quad (12.17)$$

13 Neural Networks

Neural networks are needed to resolve the limitations of other seen models. More in depth:

- **Perceptron:** are a linear model. They can only learn linearly separable functions;
- **Kernel Machines:** add the concept of non-linearity, but the choice of the appropriate kernel is not trivial;

13.1 Multi Layer Perceptron

A Multi Layer Perceptron (MLP) is a network of interconnected neurons, organized in layers. Neurons from one layer sends their output to the neurons of the next layer. The first layer is called *input layer*, one or more *hidden layers* are in the middle, and the last layer is called *output layer*.

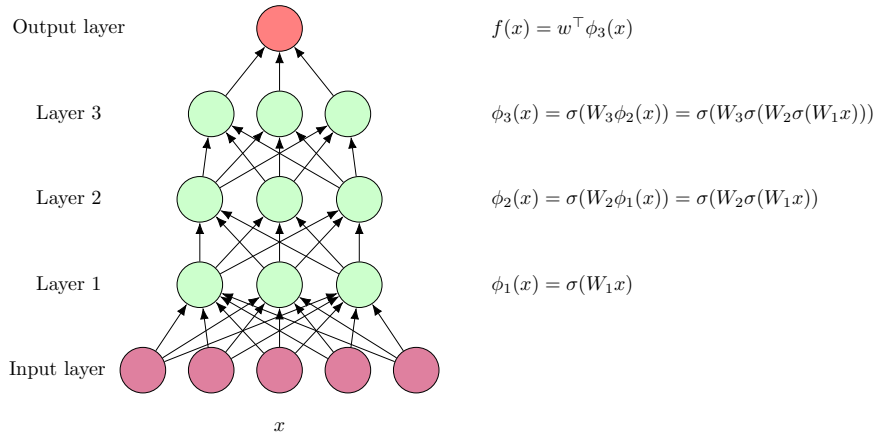


Figure 21: Multi Layer Perceptron with 3 hidden layers

Where:

- x is the input vector;
- $\phi_i(x)$ is the output of layer i , learned from the data during training.

$$\phi_i(x) = \sigma(W_i^T x)$$

- W_i is the weight matrix of layer i . Its dimensions depend on the number of neurons in layer i . Each row contains the weights associated to a neuron. Computing $W_i^T \mathbf{x}$ will return a vector which contains the weighted sum of the inputs for each neuron in layer i .
- $\sigma(\cdot)$ is a non-linear activation function, applied element-wise.

The final output is computed as a linear combination in the new feature space created by the hidden layers. Differently from SVMs, where the kernel is chosen *a priori*, in MLPs the transformation is learned from data. The only downside is that training requires more data and more computational power.

13.1.1 Activation Functions

Activation functions introduce non-linearity in the model. We will analyze some proposed functions:

Threshold activation: the threshold activation, already seen in the perceptron model, is defined as:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$$

It's cannot be used in MLPs, since it's derivative is zero almost everywhere, apart from the discontinuity in zero (not differentiable). This makes impossible to use gradient-based optimization methods for training.

Sigmoid activation: The sigmoid activation function is defined as:

$$f(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

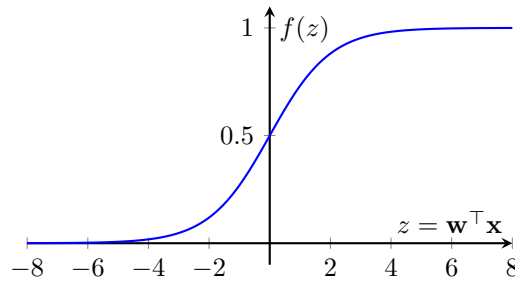


Figure 22: Sigmoid activation function

It represents a smooth approximation of the threshold function. It is approximately linear around zero, which helps in gradient-based optimization. However, for large positive or negative values of z , the function saturates (approaches 0 or 1), causing the gradient to vanish.

13.1.2 Output Layer

The output layer of a neural network produces the final predictions. The function used in this layer can be very different from the one used in hidden layers and heavily depends on the task at hand.

We will now analyze the activation functions used in the output layer for different tasks.

Binary Classification: for binary classification tasks, only one output neuron $o(\mathbf{x})$ is needed. The output activation function is usually the sigmoid activation function, which maps the output to the range $(0, 1)$. It is defined as:

$$f(\mathbf{x}) = \sigma(o(\mathbf{x})) = \frac{1}{1 + e^{-o(\mathbf{x})}}$$

This output can be interpreted as the probability of belonging to the positive class. To obtain the final class label, a threshold (usually 0.5) is applied:

$$y^* = \text{sign}(f(\mathbf{x}) - 0.5)$$

Multi-class Classification: for multi-class classification, similar to what was done with single layer perceptron, a one vs all approach can be used. This requires K output neurons, one for each class. The softmax activation function is commonly used in this case, defined as:

$$f_i(\mathbf{x}) = \frac{\exp(o_i(\mathbf{x}))}{\sum_{j=1}^K \exp(o_j(\mathbf{x}))} \quad \text{for } i = 1, 2, \dots, K$$

Where $o_i(\mathbf{x})$ is the raw output score of the i -th output neuron.

This function converts the raw output scores into probabilities that sum to 1 across all classes. This is not activated element-wise, but it is a layer-wise activation function. The predicted class is the one with the highest probability:

$$y^* = \arg \max_i f_i(\mathbf{x})$$

Regression: for regression tasks, where the output is a continuous value, the decision is the value of the output neuron itself.

$$f(\mathbf{x}) = o(\mathbf{x})$$

13.1.3 Representational power of MLPs

In this section we will analyze the representational power of MLPs on different tasks.

- **Boolean functions:** any boolean function can be represented by a MLP with two layers of units. This can be done using CNF or DNF, but it would require an exponential number of hidden units in the worst case;

- **Continuous functions:** any bounded continuous function can be approximated arbitrarily well by a MLP with two layers of units (using sigmoid activations).
- **Arbitrary functions:** any function can be approximated arbitrarily well by a MLP with three layers of units (using sigmoid activations).

13.2 Training Multi Layer Perceptrons

Similarly to other models, MLPs are trained by minimizing a loss function on the training data. Different loss functions can be used depending on the task at hand.

13.2.1 Loss Functions

Binary Classification: For binary classification tasks, the binary cross-entropy loss is commonly used. It is defined as:

$$L(y, f(\mathbf{x})) = -(y \log(f(\mathbf{x})) + (1 - y) \log(1 - f(\mathbf{x})))$$

This can be interpreted as a switch function between two log-likelihoods, depending on the true label y . In fact:

$$\begin{cases} L(y, f(\mathbf{x})) = -\log(f(\mathbf{x})) & \text{if } y = 1 \\ L(y, f(\mathbf{x})) = -\log(1 - f(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

This is needed because, as mentioned before, $f(\mathbf{x})$ represents the predicted probability of belonging to the positive class.

By putting the minus sign, we convert the maximization problem into a minimization one (from a likelihood to a loss). This way, making the argument of the log function closest to 1 (correct prediction) will minimize the loss.

Multi-class Classification: For multi-class classification tasks, the cross-entropy loss is redefined for multiple classes. It is defined as:

$$L(y, \mathbf{f}(\mathbf{x})) = -\log f_y(\mathbf{x})$$

Where $f_y(\mathbf{x})$ is the predicted probability for the true class y .

Regression: For regression tasks, the mean squared error (MSE) loss is commonly used. It is defined as:

$$L(y, f(\mathbf{x})) = (y - f(\mathbf{x}))^2$$

13.2.2 Stochastic Gradient Descent

In order to train MLPs, we use Stochastic Gradient Descent (SGD) to minimize the chosen loss function. As example, choosing mean squared error (MSE) as loss function, we would have:

$$E(W) = \frac{1}{2}(y - f(\mathbf{x}))^2$$

Where W represents all the weights in the network. Note that $\frac{1}{2}$ is added to simplify the derivative computation.

To minimize the loss function, we need to compute the gradient of the loss with respect to each weight in the network. The relative gradient update is given by:

$$w_{lj} = w_{lj} - \eta \frac{\partial E(W)}{\partial w_{lj}}$$

Where η is the learning rate, w_{lj} is the weight connecting neuron j (source node) in the previous layer, to neuron l (target node) in the current layer.

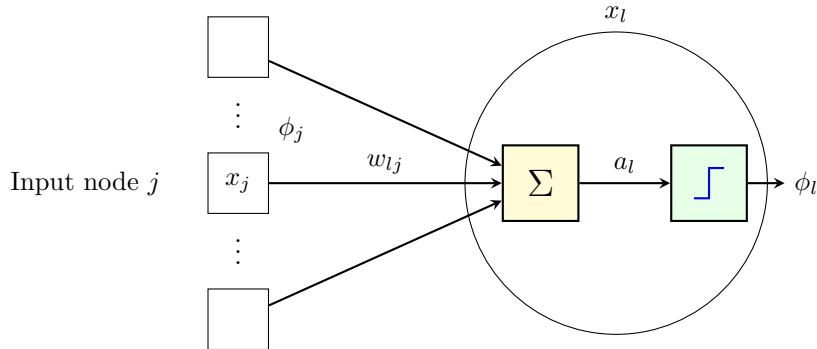
To compute the partial derivative can be quite complex when the weights are really far from the output layer, where we actually compute the loss (we need the output of the network to compute the loss).

To solve this problem, we use the Backpropagation algorithm, which efficiently computes the gradients using the chain rule of differentiation.

13.2.3 Backpropagation

The idea behind backpropagation is to compute the gradients layer by layer, starting from the output layer and moving backwards to the input layer. Since each neuron's output depends on the outputs of the previous layer, which in turn depend on their own weights, we can apply the chain rule to compute the gradients.

Let's now zoom in on a single node. Suppose we want to compute the gradient of the loss function with respect to a weight w_{lj} connecting neuron j in the previous layer to neuron l in the current layer.



Remembering that the chain rule is defined as:

$$y = f(z), z = g(x) \Rightarrow \frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$$

Applying it to our case, we have that the weight w_{lj} affects the loss function through the input of the activation function of neuron l , which in turn affects the loss function. Therefore, we can write:

$$\frac{\partial E(W)}{\partial w_{lj}} = \underbrace{\frac{\partial E(W)}{\partial a_l}}_{\delta_l} \frac{\partial a_l}{\partial w_{lj}} = \delta_l \phi_j$$

where:

- δ_l represents the error term for neuron l . Represents how much the output of neuron l contributes to the final error;
- ϕ_j is the output of neuron j in the previous layer. It is one of the inputs of the node l ;
- $\frac{\partial a_l}{\partial w_{lj}}$ is ϕ_j . Since $a_l = \sum_i w_{li} \phi_i$, if we derive with respect to w_{lj} all terms are constant except for the one with $i = j$, which gives ϕ_j .

The final update rule for weight w_{lj} is simply the error term δ_l multiplied by the input ϕ_j coming from the previous layer.

While ϕ_j is easily computed during the forward pass, δ_l is more complex to compute, since it depends on the error of the output layer and on the structure of the network. Also, the computation of δ_l depends on whether neuron l is in the output layer or in a hidden layer. We will analyze both cases.

Output layers: Since neuron l is in the output layer, we can directly compute the derivative of the loss function with respect to the output of the network. Considering the MSE loss function, we have:

$$\delta_o = \frac{\partial E(W)}{\partial a_o} = \frac{\partial \frac{1}{2}(y - f(\mathbf{x}))^2}{\partial a_o}$$

Since we do not apply a non-linear activation function in the output layer for regression tasks, we have that $f(\mathbf{x}) = a_o$, so:

$$= \frac{\partial \frac{1}{2}(y - a_o)^2}{\partial a_o} = -(y - a_o)$$

Hidden layers: Here the node l is in a hidden layer. To compute δ_l , we need to use again the chain rule. In this case, the output of neuron l affects the final loss through all possible paths from l to the output layer. Therefore, we

need to consider all the neurons in the next layer that are connected to neuron l (children of l), and sum their contributions to the error.

$$\delta_l = \frac{\partial E(W)}{\partial a_l} = \sum_{k \in \text{children}(l)} \frac{\partial E(W)}{\partial a_k} \frac{\partial a_k}{\partial a_l}$$

Where k are the neurons in the next layer connected to neuron l .

The term $\frac{\partial E(W)}{\partial a_k}$ is simply δ_k , already computed in the next layer. We derive that:

$$\delta_l = \sum_{k \in \text{children}(l)} \delta_k \frac{\partial a_k}{\partial a_l}$$

Remembering that $a_k = \sum_m w_{km} \phi_m$ (the weighted sum of all inputs of node k), we have that a change in a_l will affect a_k through the output of neuron l ϕ_l . Therefore, we can write:

$$\frac{\partial a_k}{\partial a_l} = \frac{\partial a_k}{\partial \phi_l} \frac{\partial \phi_l}{\partial a_l}$$

The first term is simply the weight w_{kl} . The second term is the derivative of the activation function $\sigma(\cdot)$ used in neuron l . This gives us the final formula for δ_l :

$$\delta_l = \sum_{k \in \text{children}(l)} \delta_k w_{kl} \frac{\partial \sigma(a_l)}{\partial a_l}$$

Remembering that we are using the sigmoid activation function, we can compute its derivative as:

$$\frac{\partial \sigma(a_l)}{\partial a_l} = \sigma(a_l)(1 - \sigma(a_l))$$

Substituting this back into the formula for δ_l , we get:

$$\delta_l = \sum_{k \in \text{children}(l)} \delta_k w_{kl} \sigma(a_l)(1 - \sigma(a_l))$$

13.3 Modular Structure of Neural Networks

Neural networks can be seen as a combination of different modules, each with a specific function. Each of these modules has an interface to communicate with other modules. Each layer j takes as input ϕ_{j-1} and its weights W_j , and produces as output $\phi_j = F(\phi_{j-1}, W_j)$. Once we get to the output layer, we can compute the loss function and the gradients using backpropagation.

$$\frac{\partial E(W)}{\partial W_j} = \frac{\partial E(W)}{\partial \phi_j} \frac{\partial \phi_j}{\partial W_j} = \frac{\partial E(W)}{\partial \phi_j} \frac{\partial F_j(\phi_{j-1}, W_j)}{\partial W_j}$$

Each layer computes:

$$\frac{\partial E(W)}{\partial \phi_{j-1}} = \frac{\partial E(W)}{\partial \phi_j} \frac{\partial \phi_j}{\partial \phi_{j-1}} = \frac{\partial E(W)}{\partial \phi_j} \frac{\partial F_j(\phi_{j-1}, W_j)}{\partial \phi_{j-1}}$$

Each module must be able to compute:

- the derivative of its output with respect to its weights:

$$\frac{\partial \phi_j}{\partial W_j} = \frac{\partial F_j(\phi_{j-1}, W_j)}{\partial W_j}$$

- the derivative of its output with respect to its input:

$$\frac{\partial \phi_j}{\partial \phi_{j-1}} = \frac{\partial F_j(\phi_{j-1}, W_j)}{\partial \phi_{j-1}}$$

13.4 Remarks on Training Neural Networks

The error surface of neural networks is non-convex. This implies that there can be multiple local minima and backpropagation is only guaranteed to find a local minimum. There are many techniques to improve the training of neural networks, that we will cover later in this section.

13.4.1 Stopping Criteria

Overtraining the network can increase the possibility of overfitting the training data. When the network is initialized, the weights are set to small random values. In this phase, we have a very simple surface, which is easy to optimize. As the weights are updated, the surface becomes more complex, and the risk of overfitting increases. To avoid this, a separate validation set is used to monitor the performance of the network during training. When the performance on the validation set starts to degrade, training is stopped.

13.4.2 Vanishing gradients

A common problem when training deep neural networks is the vanishing gradient problem. During backpropagation, at each step the gradient is multiplied by the derivative of the activation function (sigmoid in our case). Since the saturated regions of the sigmoid have very small derivatives, the gradients can become very small as they are propagated back through the layers. A few simple suggestions to mitigate this problem are:

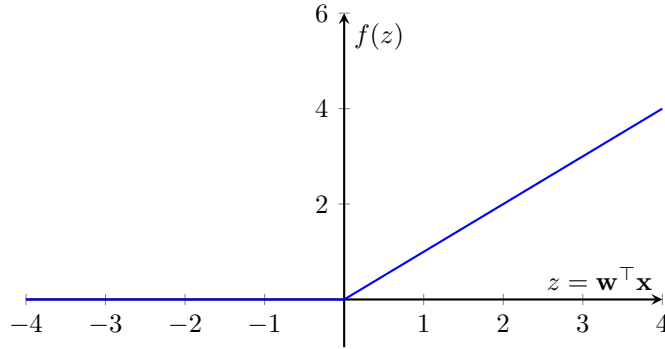
- **Weight initialization:** initialize the weights to small random values;
- **Normalization:** normalize the input data to have zero mean and unit variance ($X' = \frac{X - \mu_x}{\sigma_x}$)

Other tricks of the trade will be covered in the next paragraph.

Activations functions: Since the main problem generating vanishing gradients is the use of the sigmoid activation function, a new activation function has been proposed: the ReLU (Rectified Linear Unit). It is defined as:

$$f(x) = \max(0, \mathbf{w}^T \mathbf{x})$$

Where $\mathbf{w}^T \mathbf{x}$ represents the weighted sum of inputs to the neuron.



The ReLU activation function does not saturate for positive values, and its derivative is constant (1) in that region. This helps to mitigate the vanishing gradient problem.

Regularization: Two common regularization techniques used in training neural networks are:

- **2-norm regularization:** adding a penalty term to the loss function proportional to the euclidean norm of the weights.

$$J(W) = E(W) + \lambda ||W||_2$$

In essence, this encourages less relevant weights to be small, reducing overfitting.

- **1-norm regularization:** adding a penalty term to the loss function proportional to the absolute value of the weights. This encourages less relevant weights to be exactly zero.

$$J(W) = E(W) + \lambda |W|$$

Initialization: Random initialization of weights is crucial for breaking symmetry between neurons. If all weights are initialized to the same value, all neurons in a layer will learn the same features. A common approach is to initialize weights randomly, from a uniform distribution in the range:

$$W_{ij} \sim U\left(-\frac{\sqrt{6}}{\sqrt{n+m}}, \frac{\sqrt{6}}{\sqrt{n+m}}\right)$$

Where n is the number of input units and m is the number of output units.

Gradient descent:

- **Batch Gradient Descent:** updates weights after seeing the entire training set. This can be computationally expensive for large datasets;

- **Stochastic Gradient Descent:** updates weights after seeing each training example. The final objective function is noisy and may be different from the true objective;
- **Mini-batch Gradient Descent:** updates weights after seeing a small batch of training examples. The size of the batch is usually associated with the hardware capabilities (e.g., GPU memory).

Momentum: Momentum is a technique used to accelerate gradient descent and escape local minima. The idea is to update the weights using a combination of the current gradient and the previous updates, similar to a ball rolling down a hill. The update rule with momentum is given by:

$$v_{ji}^{(\text{new})} = \alpha v_{ji}^{(\text{old})} - \eta \frac{\partial E(W)}{\partial w_{ji}}$$

$$w_{ji}^{(\text{new})} = w_{ji}^{(\text{old})} + v_{ji}^{(\text{new})}$$

Where:

- α is the momentum coefficient. $0 \leq \alpha < 1$
- v_{ji} is the velocity term for weight w_{ji} ;

Adaptive Learning Rates: Adaptive learning rate methods adjust the learning rate during training based on the progress made. Some popular methods are:

- **Decreasing learning rate:** gradually decrease the learning rate over time.

$$\eta_t = \begin{cases} (1 - \frac{t}{\tau})\eta_0 + \frac{t}{\tau}\eta_{\min} & \text{if } t < \tau \\ \eta_{\min} & \text{if } t \geq \tau \end{cases}$$

The idea is to start with a high learning rate to explore the error surface, and then decrease to avoid oscillations around minima. The learning rate is decreased linearly from η_0 to $\eta_{\min}$ over τ iterations. After that, it remains constant at $\eta_{\min}$;

- **AdaGrad:** adapts the learning rate for each weight based on the historical gradients.

$$r_{ji}^{(\text{new})} = r_{ji}^{(\text{old})} + \left(\frac{\partial E(W)}{\partial w_{ji}} \right)^2$$

$$w_{ji}^{(\text{new})} = w_{ji}^{(\text{old})} - \frac{\eta}{\sqrt{r_{ji}^{(\text{new})}}} \frac{\partial E(W)}{\partial w_{ji}}$$

Where r_{ji} accumulates the squared gradients for weight w_{ji} . This results in smaller updates for weights with large gradients (steep directions) and larger updates for weights with small gradients (flat directions); This has a drawback: the accumulated squared gradients can grow indefinitely.

- **RMSPprop:** addresses the drawback of AdaGrad by using an exponentially decaying average of squared gradients.

$$r_{ji}^{(\text{new})} = \rho r_{ji}^{(\text{old})} + (1 - \rho) \left(\frac{\partial E(W)}{\partial w_{ji}} \right)^2$$

$$w_{ji}^{(\text{new})} = w_{ji}^{(\text{old})} - \frac{\eta}{\sqrt{r_{ji}^{(\text{new})} + \epsilon}} \frac{\partial E(W)}{\partial w_{ji}}$$

Where ρ allows to exponentially decay the influence of past gradients.

Batch Normalization: To mitigate the problem of internal covariate shift, batch normalization is used. It normalizes the input of each activation function on its mini-batch.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sigma_B}$$

Where μ_B and σ_B are the mean and standard deviation of the mini-batch B , x is the activation of an arbitrary layer. This normalization helps to stabilize the learning process and allows for higher learning rates.

13.5 Popular architectures

Different architectures of neural networks have been proposed to tackle specific problems. In this section we will briefly describe some of the most popular ones.

Autoencoders

Autoencoders are a type of neural network used for unsupervised learning (learning with no labelled data). They consist of an encoder, that maps the input data to a lower-dimensional latent space, and a decoder, that reconstructs the input data from the latent representation. This is useful for dimensionality reduction, feature learning, and data compression.

Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are specialized neural networks designed for processing grid-like data, such as images. They use convolutional filters to extract local features from the input data. Pooling layers are used to compose higher-level features from lower-level ones. After several convolutional and pooling layers, fully connected layers are used to perform the final classification.

Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) learns to generate items from random noise. They consist of two neural networks: a generator, that generates new data, and a discriminator, that tries to distinguish between real and generated data. The two networks are trained simultaneously, with no supervision needed.

Transformers

Use attention mechanisms to learn input words encodings, that depends on the context of the sentence. They can also learn output words encodings, that depends on the context of the already generated words and on the input sentence.

Graph Neural Networks (GNNs)

Allow to learn from graph-structured data. Allow to propagate information from a node to its neighbors, learning a representation that takes into account the graph structure.

14 Unsupervised Learning - Clustering

What we have seen so far are all examples of **supervised learning**, which required a labeled dataset to learn from. The process of obtaining labeled data is usually extremely expensive, and it could happen that we don't have access to labeled data at all.

In such cases, we can resort to **unsupervised learning** techniques, which can be used to group data points into **clusters**.

14.1 K-Means Clustering

One of the most popular clustering algorithms is **K-Means Clustering**. The setting is the following:

- We have a dataset of n points $\{x_1, x_2, \dots, x_n\}$ in $\mathbb{R}^d$.
- We assume that examples should be grouped into k clusters, we will see later how to choose k ;
- Each cluster i is represented by its **centroid** (mean) μ_i ;

Algorithm

The K-Means algorithm works as follows:

1. Initialize k cluster means $\mu_1, \mu_2, \dots, \mu_k$ (randomly or using some heuristic);
2. Repeat until convergence:
 - **Assignment step:** Assign each point x_j to the nearest cluster mean;
 - **Update step:** Update cluster means according to the assigned points;

The algorithm converges when the assignments no longer change or the cluster means stabilize.

Distance Metrics

To compute the distance between points and cluster means, we have to choose a distance metric, the most common choices are:

- Euclidean distance in $\mathbb{R}^d$;

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{i=1}^d (x_i - x'_i)^2}$$

- Manhattan distance:

$$d(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^d |x_i - x'_i|$$

- Cosine similarity:

$$d(\mathbf{x}, \mathbf{x}') = 1 - \frac{\mathbf{x}^T \mathbf{x}'}{\|\mathbf{x}\| \|\mathbf{x}'\|}$$

14.2 Quality of Clustering

Since we are working in an unsupervised setting, we don't have true labels to evaluate the quality of our clustering. However, we can define the following metric:

Sum-of-Squared error criterion

The sum-of-squared error criterion is defined as:

- let n_i be the number of points assigned to cluster i (D_i);
- let μ_i be the centroid of cluster i . It can be computed as:

$$\mu_i = \frac{1}{n_i} \sum_{\mathbf{x} \in D_i} \mathbf{x}$$

- the sum-of-squared error criterion is defined as:

$$E = \sum_{i=1}^k \sum_{\mathbf{x} \in D_i} \|\mathbf{x} - \mu_i\|^2$$

It measures the squared error incurred in representing each point by its cluster centroid.

14.3 Gaussian Mixture Models (GMM)

The idea behind Gaussian Mixture Models is to model the data as a mixture of several Gaussian distributions. Each cluster is represented by a Gaussian distribution with its own parameters (mean and covariance). It assumes that the number of clusters K is given and that each data point is generated from one of the K Gaussian distributions.

Parameter Estimation

If we wanted to estimate the parameters of the Gaussians (means and covariances), we could use Maximum Likelihood Estimation (MLE).

$$\theta^* = \arg \max_{\theta} \mathcal{L}(\theta | \mathcal{D}) = \arg \max_{\theta} p(\mathcal{D} | \theta)$$

where θ are the parameters of the Gaussians and $\mathcal{D}$ is the dataset. Assuming that the data points are independent, we can write the likelihood as:

$$\mathcal{L}(\theta|\mathcal{D}) = \prod_{i=1}^N p(x_i|\theta)$$

Taking the logarithm (to simplify the optimization), we get the log-likelihood:

$$\log \mathcal{L}(\theta|\mathcal{D}) = \sum_{i=1}^N \log p(x_i|\theta)$$

In the case of Gaussian Mixture Models, the probability of a data point x_i is given by the sum of the probabilities of the K Gaussians:

$$p(x_i|\theta) = \sum_{k=1}^K \pi_k p(x_i|\theta_k)$$

where θ_k are the parameters of the k -th Gaussian and π_k is the prior probability of cluster k .

Substituting this into the log-likelihood, we get:

$$\log \mathcal{L}(\theta|\mathcal{D}) = \sum_{i=1}^N \log \left(\sum_{k=1}^K \pi_k p(x_i|\theta_k) \right)$$

However, since cluster assignments are latent, we cannot directly maximize this log-likelihood. Instead, we can apply an iterative approach called the **Expectation-Maximization (EM)**.

Expectation-Maximization (EM) Algorithm

The EM algorithm consists of two main steps:

1. **E-step (Expectation step)**: Compute the expected cluster assignments given the current parameters;
2. **M-step (Maximization step)**: Update the parameters of the Gaussians given the expected cluster assignments;
3. Repeat until convergence.

Example: estimating the mean of k univariate Gaussians

Here, we assume to have a dataset of $x_1, x_2, \dots, x_N$ examples. We assume that the data is generated from k univariate Gaussian distributions with unknown means $\mu_1, \mu_2, \dots, \mu_k$ and known variance σ^2 . We want to estimate the means of the Gaussians using the EM algorithm, allowing us to assign points to clusters probabilistically.

The algorithm works as follows:

1. **Initialization:** randomly initialize the means $h = \langle \mu_1, \mu_2, \dots, \mu_k \rangle$;
2. **Iterate until convergence:** convergence is checked by measuring the change in Maximum Log-Likelihood:
 - **E-step:** compute the posterior probability that a datapoint x_i belongs to cluster k , given the current means h .

$$\gamma_{ik} = \frac{\pi_k p(x_i | \mu_k)}{\sum_{j=1}^K \pi_j p(x_i | \mu_j)}$$

We are basically computing the probability that x_i belongs to cluster k and normalizing it over all clusters for each data point. This way, summing over all clusters gives 1. The term π_j represents the prior probability of cluster j .

- **M-step:** calculate a new hypothesis h' assuming values of latent variables are their expected values. The maximum likelihood estimate of the mean μ_k is given by the weighted sample mean, where each instance is weighted by its probability of belonging to cluster k :

$$\mu'_k = \frac{\sum_{i=1}^N \gamma_{ik} x_i}{\sum_{i=1}^N \gamma_{ik}}$$

14.4 Choose the number of clusters k

Choosing the right number of clusters k is a crucial step in clustering. Increasing the number of clusters will always improve the fit to the data, but it may lead to overfitting. We will now introduce different methods to choose k .

14.4.1 Elbow Method

The Elbow Method tries to trade-off the quality of the clustering with quantity. The number of clusters k should stop increasing when the advantage of adding another cluster is limited. The approach is composed of the following steps:

- Run the clustering algorithm for increasing values of k ;
- For each k , compute the sum-of-squared error criterion $E(k)$;
- Plot $E(k)$ against k ;
- Choose k at the "elbow" point, where the decrease in $E(k)$ starts to slow down.

Optimal number of clusters

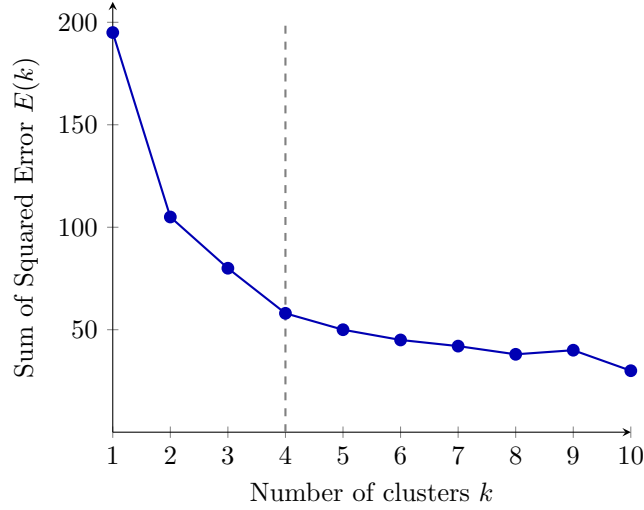


Figure 23: Elbow Method example plot. The elbow point is at $k = 4$.

This method is really intuitive, but can be ambiguous, since the elbow point is not always clear. For example, in the plot above, one could argue that $k = 2$ or $k = 3$ are also valid choices.

14.4.2 Silhouette Method

Increasing the number of clusters k will always decrease the sum-of-squared error criterion, but it will also increase the similarity between different clusters. The Silhouette Method tries to balance these two aspects. The silhouette score for a single point x_i is defined as:

- $a(i)$: average distance between x_i and all other points in the same cluster C :

$$a(i) = \frac{1}{|C|} \sum_{j \in C} d(i, j)$$

- $b(i)$: minimum average distance between x_i and all points in any other cluster C' :

$$b(i) = \min_{C' \neq C} \frac{1}{|C'|} d(i, C')$$

- The silhouette score $s(i)$ is then defined as:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

To choose the number of clusters k , we compute the average silhouette score over all points for different values of k . As before, we plot the average silhouette score against k and choose the value of k that maximizes the score.

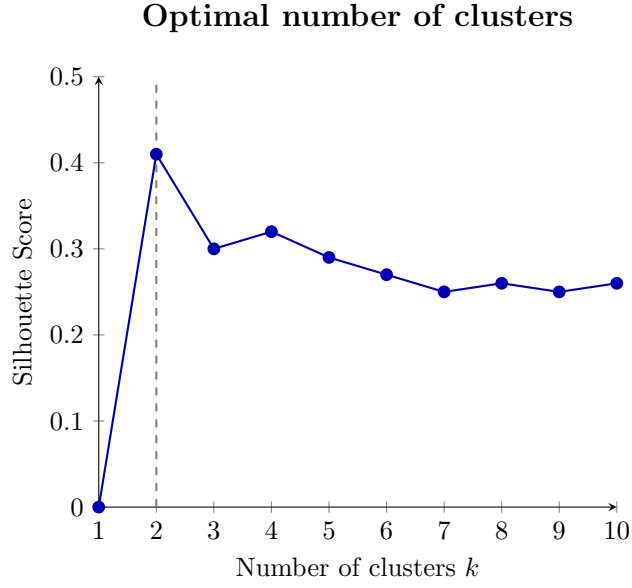


Figure 24: Silhouette Method example plot. The optimal number of clusters is at $k = 2$.

14.4.3 Hierarchical Clustering

Hierarchical Clustering is an alternative approach, that assumes that the data is organized in a hierarchy of clusters. This structure can be built from examples in two ways:

- **Top-down approach:** start with all data points in a single cluster and recursively split clusters into smaller clusters;
- **Bottom-up approach:** start with each data point in its own cluster and recursively merge clusters into larger clusters.

The algorithm for the Bottom-up approach is defined as follows:

- Initialize:
 - Set the target number of clusters k ;
 - Set the initial number $\hat{k}$ of clusters to n ;
 - Initialize clusters as individual data points: $D_i = \{x_i\}, \forall i \in [1, n]$
- Repeat until the number of clusters $\hat{k}$ is equal to k :

- Find the two closest clusters D_i and D_j ;
- Merge the clusters: $D_i \leftarrow D_i \cup D_j$;
- Update the number of clusters: $\hat{k} \leftarrow \hat{k} - 1$;

The similarity between clusters can be computed in different ways:

- **Nearest neighbor:** distance between the two closest points in the clusters:

$$d_{min}(D_i, D_j) = \min_{\mathbf{x} \in D_i, \mathbf{x}' \in D_j} \|\mathbf{x} - \mathbf{x}'\|$$

- **Farthest neighbor:** distance between the two farthest points in the clusters:

$$d_{max}(D_i, D_j) = \max_{\mathbf{x} \in D_i, \mathbf{x}' \in D_j} \|\mathbf{x} - \mathbf{x}'\|$$

- **Average linkage:** average distance between all points in the clusters:

$$d_{avg}(D_i, D_j) = \frac{1}{|D_i||D_j|} \sum_{\mathbf{x} \in D_i} \sum_{\mathbf{x}' \in D_j} \|\mathbf{x} - \mathbf{x}'\|$$

The first two methods are sensitive to outliers, while the average linkage is more robust. All result complex to compute. A more efficient method is the **centroid method**, which computes the distance between the centroids of the clusters:

$$d_{centroid}(D_i, D_j) = \|\mu_i - \mu_j\|$$

where μ_i and μ_j are the centroids of clusters D_i and D_j respectively.

15 Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning algorithm where an agent learns to make decisions by taking actions in an environment to maximize cumulative rewards.

15.1 Learning Settings

- The learner (agent) is provided with a set of possible states S , and for each state, a set of possible actions A , moving it to a new state.
- In performing action a , from state s , the agent receives an immediate reward $r(s, a)$.
- The goal is to learn a policy $\pi : S \rightarrow A$ that maximizes the overall reward over time.
- The learner has to deal with problems of delayed reward, where the consequences of an action may not be immediately apparent and trade-offs between **exploration** (trying new actions) and **exploitation** (choosing actions known to yield high rewards).

The agent needs to take a sequence of decisions, maximizing some utility function over time. The result of a decision may not be always deterministic.

More formally, the setting of reinforcement learning can be modeled as a Markov Decision Process (MDP).

15.2 Markov Decision Process (MDP)

An MDP is defined by:

- A set of **states** S , in which the agent can be;
- A set of **terminal states** $S_G \subseteq S$, where the process ends. Might also be empty;
- A set of **actions** A available to the agent;
- A **transition model**, providing the probability of going to state s' when taking action a in state s :

$$P(s'|s, a) \quad s, s' \in S, a \in A$$

- A **reward function** $R(s, a, s')$, providing the immediate reward received after transitioning from state s to state s' due to action a .

The agent's objective is to find a policy $\pi(s)$ that maximizes an utility function, taking into account the uncertainty in state transitions and rewards.

15.3 Utilities

Utilities are defined over **environment histories**. An environment history is a sequence of states $[s_0, s_1, s_2, \dots]$, visited by the agent over time. When calculating the utility, we assume an **infinite horizon**, meaning the agent will continue to interact with the environment indefinitely. We also assume **stationary preferences**, meaning the agent's preferences do not change over time.

We can define the utility in two ways:

- **Additive rewards:** values immediate and future rewards equally.

$$U([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \dots$$

- **Discounted utility:** prefers immediate rewards over future rewards.

$$U([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$$

where $0 \leq \gamma < 1$ is the discount factor.

Here we are assuming that rewards only depend on the current state, not the action taken or the next state.

15.4 Policy

A policy π is a mapping from states to actions: $\pi : S \rightarrow A$. It defines the action a that the agent should take when in a given state s .

Expected Utility of a policy Given a policy π , we define the expected utility of π as the utility of an environment, taken in expectation over all possible state sequences generated by following policy π . In other words, we are multiplying the utility of that history, by the probability of that history occurring under policy π .

Optimal Policy

An optimal policy π^* is a policy that maximizes the expected utility. In other words, it is the best policy the agent can follow to achieve the highest cumulative reward over time.

Utility of states

Given a policy π , the utility of a state s is defined as:

$$U^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(S_t) | S_0 = s \right]$$

Where S_t is the state reached after t steps, using policy π , starting from state $S_0 = s$. This is the expected utility of the environment history, starting from state s and following policy π , obtained by summing the discounted rewards over time.

The **true utility** of a state s is defined as the utility of that state under the optimal policy:

$$U^*(s) = U^{\pi^*}(s) \quad (15.1)$$

Given the true utility of each state, we can easily derive the optimal policy by choosing the action that maximizes the expected utility:

$$\pi^*(s) = \arg \max_{a \in A} \sum_{s' \in S} P(s'|s, a) U^*(s') \quad (15.2)$$

This definition creates a circular dependency between the optimal policy and the true utility, which can be resolved using the Bellman equations.

15.4.1 Bellman Equations

The Bellman equation for state s is defined as:

$$U(s) = R(s) + \gamma \max_{a \in A} \sum_{s' \in S} P(s'|s, a) U(s')$$

This equation states that the utility of a state s is equal to the immediate reward $R(s)$ obtained by being in that state, plus the discounted expected utility of the next state, assuming the agent takes the optimal action.

We can define a system of Bellman equations, one for each state in the MDP. Solving this system will yield the true utilities for all states, which can then be used to derive the optimal policy. Directly solving this system can be computationally expensive, since they are non-linear equations (due to the max operator). Instead, we can use iterative methods such as *value iteration* or *policy iteration* to approximate the solution.

15.4.2 Value Iteration

Value iteration, also known as Utility Iteration, works by initializing the utility of all states to arbitrary values (for example zero) and then repeatedly updating the utilities using the Bellman equation until convergence.

Algorithm 4 Value Iteration

- 1: Initialize $U_0(s) \leftarrow 0 \quad \forall s \in S$
- 2: **repeat**
- 3: Do bellman update for all states s :

$$U_{i+1}(s) \leftarrow R(s) + \gamma \max_{a \in A} \sum_{s' \in S} P(s'|s, a) U_i(s')$$

- 4: $i \leftarrow i + 1$
 - 5: **until** convergence
 - 6: **return** $U(s)$
-

At each iteration, we update the utility of each state based on the current estimates of the utilities of the successor states. This process continues until the utilities converge to stable values. Since we are assuming that the found utilities are the true utilities, the optimal policy can then be derived from the converged utilities using the formula for the optimal policy 15.4.

15.4.3 Policy Iteration

Contrary to what happens in value iteration (where the policy is derived from the utilities), in policy iteration we start with an arbitrary policy and then iteratively improve it.

Algorithm 5 Policy Iteration

- 1: Initialize π_0 randomly
- 2: **repeat**
- 3: **Policy Evaluation:** solve the set of linear equations:

$$U_i(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s, \pi_i(s)) U_i(s') \quad \forall s \in S$$

where $\pi_i(s)$ is the action taken in state s under policy π_i .

- 4: **Policy Improvement:** update the policy using the new utilities:

$$\pi_{i+1}(s) = \arg \max_{a \in A} \sum_{s' \in S} P(s'|s, a) U_i(s') \quad \forall s \in S$$

- 5: **until** convergence
 - 6: **return** $\pi(s)$
-

In the policy evaluation step, we compute the utilities of the states under the current policy by solving a system of linear equations. Using the current policy, instead of the max operator from the Bellman equation, we made the equations linear, making it easier to solve. In the policy improvement step, we update the policy using equation 15.4, based on the newly computed utilities.

15.5 Dealing with Partial Knowledge

Value iteration and policy iteration both assume that the agent has complete knowledge of the MDP (environment, transition model, reward function). In most real-world scenarios, the agent does not have access to the full transition model or reward function, but it aims to learn them through interaction with the environment.

There are two main approaches to deal with this partial knowledge:

- **Policy Evaluation:** the policy is given, environment is learned through experience (passive agent). The agent simply learns the transition model while following a fixed policy.
- **Policy Improvement:** both the policy and the environment are learned through experience (active agent).

15.6 Policy Evaluation

15.6.1 Adaptive Dynamic Programming (ADP)

In ADP, the agent maintains counts of useful transitions to estimate the transition model:

- N_{sa} : number of times action a has been taken in state s ;
- $N_{s'|sa}$: number of times action a has led to state s' from state s .

Algorithm 6 Adaptive Dynamic Programming

repeat

 Initialize s

repeat

 receive reward r , set $R(s) = r$

 choose action $a \leftarrow \pi(s)$

 execute action a , observe next state s'

 update counts:

$$N_{sa} \leftarrow N_{sa} + 1$$

$$N_{s'|sa} \leftarrow N_{s'|sa} + 1$$

 update transition model:

$$P(s''|s, a) = \frac{N_{s''|sa}}{N_{sa}} \quad \forall s'', s \in S, a \in A$$

 update utilities using policy evaluation

$U \leftarrow \text{PolicyEvaluation}(\pi, U, p, R, \gamma)$

until s is terminal

until convergence

Remark:

The outer loop is needed to ensure any significant results. Each iteration of the outer loop is called an **Episode**.

The algorithm performs maximum likelihood estimation of the transition model based on observed counts. It then updates the utilities of each state, based on the current learned transition model, using standard policy Evaluation. This approach is really expensive, since it requires computing the policy evaluation at every step.

15.6.2 Temporal-Difference Learning (TD Learning)

TD Learning tries to approximate the utility of states, without needing to run full policy evaluation at every step. This is done by assuming a deterministic transition model (no real transition model is learned). If the agent is in state s and takes action a , arriving in state s' , we assume that s' is the only possible successor of s :

- If s' was always the successor of s , then we can update the utility of s as:

$$U(s) = R(s) + \gamma U(s')$$

- The temporal-difference updates the utility of s incrementally towards this target at each step:

$$U(s) \leftarrow U(s) + \alpha (R(s) + \gamma U(s') - U(s))$$

Where α is the learning rate, controlling how much new information overrides old information. It should be decreasing over time, to ensure convergence. Basically, the idea is to adjust the utility of the current state s , making it closer to the approximation of the utility based on the observed transition to s' .

Algorithm 7 TD Learning

repeat

 Initialize s

repeat

 Receive reward r

 Choose next action $a \leftarrow \pi(s)$

 Execute action a , observe next state s'

 update local utility estimates:

$$U(s) \leftarrow U(s) + \alpha (r + \gamma U(s') - U(s))$$

$s \leftarrow s'$

until s is terminal

until convergence

15.6.3 Differences

- TD Learning does not learn a transition model (simply estimates the utility), while ADP does (it uses maximum likelihood estimation);
- Each step of TD Learning is computationally cheaper than ADP, since it does not require running policy evaluation;
- TD Learning takes longer to converge than ADP.

We can say that TD Learning is a rough approximation of ADP.

15.7 Policy Improvement

As we have said before, in policy improvement both the optimal policy and the environment are learned through experience.

15.7.1 A simple solution

A simple solution to this problem consists in using a variation of ADP, where the policy evaluation is replaced with the computation of the optimal policy (both value and policy iteration can be used).

$$U(s) = R(s) + \gamma \max_{a \in A} \sum_{s' \in S} P(s'|s, a) U(s')$$

This approach theoretically works, but it has two main issues:

1. It is computationally expensive, since it requires computing the optimal policy at every step;
2. This is a greedy approach, which may lead to suboptimal policies. The space is not explored enough, and the agent tend to overfit to the current knowledge of the environment.

To resolve the exploration vs exploitation trade-off, non entirely greedy algorithms should be defined, to encourage exploration of the state space.

15.7.2 Trade off strategies

There are two main strategies to deal with the exploration vs exploitation trade-off:

- **ϵ -greedy**: with probability $1 - \epsilon$, the agent chooses the action that maximizes the expected utility (the greedy action). With probability ϵ , the agent chooses a random action (exploration).
- **Define a new Bellman equation**: assign **higher utilities estimates** to less explored states, to encourage the agent to visit them.

$$U^+(s) = R(s) + \gamma \max_{a \in A} f\left(\sum_{s' \in S} P(s'|s, a) U^+(s'), N_{sa}\right)$$

Where the function $f(u, n)$ is directly proportional to u and inversely proportional to n (the number of times action a has been taken in state s).

Both these strategies help with the suboptimal policies issue, but still require computing the optimal policy at every step, which is computationally expensive.

15.7.3 Utilities of an action:

To resolve the computational issue we could follow the idea used in section 15.6 and use TD Learning to approximate the utilities of states. However, TD Learning cannot be directly applied in this context. Here, we are learning both the policy and the environment.

As we have seen in equation 15.4, the optimal policy depends on the transition model, which is not computed in TD Learning.

$$\pi^*(s) = \arg \max_{a \in A} \underbrace{\sum_{s' \in S} \overbrace{P(s'|s, a)}^{\text{Not computed}} U^*(s')}_{Q(s, a)} \quad (15.3)$$

We could use TD Learning to directly learn an **action utility function** $Q(s, a)$, which represents the utilities of taking action a in state s .

Given this, we can derive the optimal policy as:

$$\pi^*(s) = \arg \max_{a \in A} Q(s, a)$$

There are 2 algorithms that use this idea to compute the optimal policy: SARSA (On-policy) and Q-Learning (Off-policy).

SARSA (On-policy)

SARSA (State-Action-Reward-State-Action) is an on-policy TD learning algorithm. It chooses the next action based on a ϵ -greedy policy derived from the current utility Q . It then takes that action, to reach state s' . It uses again the ϵ -greedy policy to choose the next action a' in state s' .

Algorithm 8 SARSA

```

repeat
  Initialize  $s$ 
  repeat
    Receive reward  $r$ 
    Choose action  $a \leftarrow \pi^\epsilon(s)$ 
    Execute action  $a$ , observe reward  $r$  and next state  $s'$ 
    Choose next action  $a' \leftarrow \pi^\epsilon(s')$ 
    Update utility estimates:
      
$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a))$$

     $s \leftarrow s'$ 
     $a \leftarrow a'$ 
  until  $s$  is terminal
until convergence

```

Since it is an on-policy algorithm, SARSA updates the utility action $Q(s, a)$ using the current policy π^ϵ . This means that the action a' taken in state s' is also influenced by the exploration strategy, which can lead to more conservative updates.

Q-Learning (Off-policy)

Q-Learning is an off-policy TD learning algorithm. It uses different policies for exploration and for learning the utilities of actions (the greedy policy).

Algorithm 9 Q-Learning

```

repeat
  Initialize  $s$ 
  repeat
    Receive reward  $r$ 
    Choose action  $a \leftarrow \pi^\epsilon(s)$ 
    Execute action  $a$ , observe reward  $r$  and next state  $s'$ 
    Update utility estimates:
      
$$Q(s, a) \leftarrow Q(s, a) + \alpha \left( r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

     $s \leftarrow s'$ 
  until  $s$  is terminal
until convergence

```

15.8 Scaling to Large Problems

All the techniques we have seen so far assume a tabular representation of the utility functions. This is not feasible for large state or action spaces, since the tables would become too large to store and update efficiently.

The solution is to use function approximation techniques: we can approximate the utility functions, using parametric functions (such as neural networks). The problem now shifts to learning the parameters of these functions, instead of maintaining large tables. This can be done using gradient descent methods.

15.8.1 TD learning: State Utilities

First, to do gradient descent, we need to define an error function we need to minimize.

$$E(s, s') = \frac{1}{2} (R(s) + \gamma U_\theta(s') - U_\theta(s))^2 \quad (15.4)$$

Where $U_\theta(s)$ is the approximated utility function, parameterized by θ . This error function measures the distance between the current value function $U_\theta(s)$ and the target value $R(s) + \gamma U_\theta(s')$.

We can then compute the gradient of this error function with respect to the parameters θ :

$$\nabla_\theta E(s, s') = - (R(s) + \gamma U_\theta(s') - U_\theta(s)) \nabla_\theta U_\theta(s) \quad (15.5)$$

The stochastic gradient descent update rule for the parameters θ is then given by:

$$\theta \leftarrow \theta - \alpha \nabla_\theta E(s, s') \quad (15.6)$$

Where α is the learning rate.

15.8.2 TD learning: Action Utilities

Similarly, we can define an error function for the action utilities:

$$E(s, a, s', a') = \frac{1}{2} (R(s) + \gamma Q_\theta(s', a') - Q_\theta(s, a))^2 \quad (15.7)$$

This error function measures the distance between the current action-value function $Q_\theta(s, a)$ and the target value $R(s) + \gamma Q_\theta(s', a')$.

The gradient of this error function with respect to the parameters θ is:

$$\nabla_\theta E(s, a, s', a') = - (R(s) + \gamma Q_\theta(s', a') - Q_\theta(s, a)) \nabla_\theta Q_\theta(s, a) \quad (15.8)$$

The stochastic gradient descent update rule for the parameters θ is then given by:

$$\theta \leftarrow \theta + \alpha \nabla_\theta E(s, a, s', a') \quad (15.9)$$

Where α is the learning rate.

16 Ensemble methods

Ensemble methods are techniques that combine multiple ML models to improve the overall performance compared to individual models. The idea is based on the principle that group of individuals can make better decisions than a single individual, if they have diverse perspectives.

Ensemble methods need to diversify their predictions. They can be categorized into three main types:

1. **Bagging**: diversifies the training sets, while keeping the model architecture the same;
2. **Stacking**: diversifies the model architectures being trained;
3. **Boosting**: diversifies the weights of the single training example, focusing on the "hard" ones.

16.1 Bagging

In a nutshell, Bagging is based on this simple tasks:

- Take a learning algorithm A ;
- Extract m different datasets $D^{(i)}$ from the original training set D , using **bootstrap resampling** (explained later);
- Train one base model on each dataset:

$$f^{(i)} = A(D^{(i)})$$

- Combine predictions of different base models:

$$\hat{y} = \text{COMBINE}(f^{(1)}(x), \dots, f^{(m)}(x))$$

Where $\hat{y}$ is the final prediction. The various methods to combine will be discussed later.

16.1.1 Bootstrap resampling

The easiest way to create different datasets from the original would be to split it into m disjoint subsets. This would however reduce the amount of data available for training each model, which is not ideal.

Bootstrap resampling instead extracts $N = |D|$ samples from D **with replacement**, meaning that the same sample can be extracted multiple times. This procedure is then repeated m times to create m different datasets, of the same size as D .

We can show that each bootstrap sample contains on average 63% of the unique samples from the original dataset.

Proof. Consider a single sample from the original dataset. The probability that it is selected in one draw is:

$$P(\text{selected}) = \frac{1}{N}$$

We derive that the probability that it is not selected in one draw is:

$$P(\text{not selected}) = 1 - \frac{1}{N}$$

Since we draw N times (with replacement), the probability that it is never selected in the bootstrap sample is:

$$P(\text{never selected}) = \left(1 - \frac{1}{N}\right)^N$$

As N approaches infinity, this expression converges to $e^{-1} \approx 0.3679$. $\square$

The instances that are not selected in a bootstrap sample are called **out-of-bag** (OOB) instances, and can be used to estimate the test performance of the base model.

16.1.2 Combining methods

There are 3 main methods to combine the predictions of the base models:

- **Majority voting:** for classification tasks, the final prediction is the class with the most votes:

$$\hat{y} = \arg \max_y \sum_{i=1}^m \delta(y, \hat{y}^{(i)})$$

where $\delta(a, b)$ is 1 if $a = b$, 0 otherwise, and $\hat{y}^{(i)}$ is the prediction of the i -th base model;

- **Soft voting:** for multiclass classification tasks, the final prediction is the class with the highest average predicted probability. This assumes that the base models can output class probabilities.

$$\hat{y} = \arg \max_y \frac{1}{m} \sum_{i=1}^m f_y^{(i)}(x)$$

where $f_y^{(i)}(x)$ is the predicted probability of class y for the i -th base model;

- **Mean:** for regression tasks, predict the mean of the based models' predictions:

$$\hat{y} = \frac{1}{m} \sum_{i=1}^m \hat{y}^{(i)}$$

Given this, we can now formally define **Random Forests**. A Random Forest is an ensemble of decision trees trained using bagging, which introduce stochasticity not only by bootstrap resampling the data, but also by randomly selecting a subset of features at each split in the tree.

16.2 Stacking

Stacking is an ensemble method that combines multiple different base models by training a meta-model to make final predictions based on the outputs of the base models. The process can be summarized in the following steps:

- Train multiple base models $f^{(i)}$ on the original training set D , using m different algorithms $A^{(i)}$:

$$f^{(i)} = A^{(i)}(D)$$

- Use a **meta-learner** to learn a combination of the base models' predictions (e.g., a linear combination):

$$g = A_{\text{META}}([f^{(1)}, \dots, f^{(m)}], D')$$

- Use the meta-model to make final predictions:

$$\hat{y} = g([f^{(1)}(x), \dots, f^{(m)}(x)])$$

The **meta-model** should be trained on a separate dataset D' or it will simply focus on learning the best performing model on D .

16.3 Boosting

Boosting is an ensemble method that combines multiple weak learners to create a strong learner. The key idea is to train models sequentially, with each new model focusing on the errors made by the previous ones. The process can be summarized in the following steps:

- Take a learner A and train it on D ;
- Reweigh examples in D based on their accuracy according to the trained model (incorrectly classified examples get higher weights);
- Train A again on the reweighed dataset;
- Repeat the procedure for m times;
- Combine the learned models into the final model.

This procedure is particularly effective with weak learners.

Weak learner

A weak learner is a model that performs slightly better than random guessing. A weak learner is easy to implement and train. Applying boosting with weak learners as the base models, allows to turn them into a strong learner. This may come out easier than training a single complex model.

One of the most popular boosting algorithms is **AdaBoost** (Adaptive Boosting).

16.3.1 AdaBoost

Algorithm 10 AdaBoost

```

1:  $d^{(0)} = (\frac{1}{N}, \dots, \frac{1}{N})$  ▷ initialize uniform important weights
2: for  $i = 1, \dots, m$  do
3:    $f^{(i)} \leftarrow \mathcal{A}(D, d^{(i-1)})$  ▷ train  $i^{\text{th}}$  model on weighted data
4:    $\hat{y}_n \leftarrow f^{(i)}(x_n), \forall n$  ▷ collect model predictions
5:    $\hat{\varepsilon}^{(i)} \leftarrow \sum_n d_n^{(i-1)} \mathbb{I}[y_n \neq \hat{y}_n]$  ▷ compute weighted training error
6:    $\alpha^{(i)} \leftarrow \frac{1}{2} \log \left( \frac{1 - \hat{\varepsilon}^{(i)}}{\hat{\varepsilon}^{(i)}} \right)$  ▷ compute adaptive parameter
7:    $d_n^{(i)} \leftarrow \frac{1}{Z} d_n^{(i-1)} \exp(-\alpha^{(i)} y_n \hat{y}_n), \forall n$  ▷ re-weight examples
8: end for
9: return  $f(x) = \text{sgn}(\sum_i \alpha^{(i)} f^{(i)}(x))$ 

```

Where $d^{(i)}$ are the importance weights of the training examples at iteration i :

$$d_n^{(i)} = \frac{1}{Z} d_n^{(i-1)} \exp(-\alpha^{(i)} y_n \hat{y}_n)$$

- Correctly classified examples ($y_n * \hat{y}_n = 1$) will have their weight decreased;
- Incorrectly classified examples ($y_n * \hat{y}_n = -1$) will have their weight increased;
- Z is a normalization factor to ensure that the weights sum to 1.

A Principal Component Analysis

Principal Component Analysis (PCA) is a widely used technique for dimensionality reduction and feature extraction in machine learning and statistics. Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ be a data matrix, with correlated coordinates. PCA is a linear transformation that transforms the data into a new coordinate system of uncorrelated variables.

A.1 PCA Algorithm

The PCA algorithm can be summarized in the following steps:

1. Compute the **mean** of the data:

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \quad (\text{A.1})$$

2. Center the data by subtracting the mean from each data point:

$$\mathbf{X} = \mathbf{X} - \bar{\mathbf{x}} \quad (\text{A.2})$$

We are basically defining a new matrix, where each row is the original data point minus the mean.

3. Compute the **covariance matrix** of the centered data:

$$\mathbf{C} = \frac{1}{n} \mathbf{X}^T \mathbf{X} \quad (\text{A.3})$$

Since $x_i = x_i - \bar{x}$, this is equivalent to the usual definition of covariance. In fact, we have:

$$\mathbf{X}^T \mathbf{X} = \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T = \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})^2$$

4. Compute the **eigenvalues** and **eigenvectors** of the covariance matrix $\mathbf{C}$. The eigenvectors represent the directions of maximum variance in the data, while the eigenvalues indicate the amount of variance in the direction of each eigenvector.
5. We can then select the top k eigenvectors corresponding to the largest eigenvalues to form a new feature space of reduced dimensionality.